

SISTEM DETEKSI DAN HITUNG OBJEK KENTANG MULTI-SKALA SECARA *REAL-TIME*



AUDY FEBRYANTI
D121211005



PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
MAKASSAR
2026

**SISTEM DETEKSI DAN HITUNG OBJEK KENTANG MULTI-SKALA SECARA
*REAL-TIME***

**AUDY FEBRYANTI
D121211005**



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**

**SISTEM DETEKSI DAN HITUNG OBJEK KENTANG MULTI-SKALA SECARA
*REAL-TIME***

AUDY FEBRYANTI

D121211005

Skripsi

sebagai salah satu syarat untuk mencapai gelar sarjana

Program Studi Teknik Informatika

pada

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**

SKRIPSI
SISTEM DETEKSI DAN HITUNG OBJEK KENTANG MULTI-SKALA SECARA
REAL-TIME

AUDY FEBRYANTI
D121211005

Skripsi,

telah dipertahankan di depan Panitia Ujian Sarjana Teknik Informatika pada 01 April
2026 dan dinyatakan telah memenuhi syarat kelulusan
pada



Program Studi Teknik Informatika
Departemen Teknik Informatika
Fakultas Teknik
Universitas Hasanuddin
Makassar

Mengesahkan:
Pembimbing Tugas Akhir,

Mengetahui:
Ketua Program Studi,

Prof. Dr. Ir. Indrabayu, S.T., M.T.,
M.Bus.Sys., IPM., ASEAN Eng.
NIP: 19750716 200212 1 004

Prof. Dr. Ir. Indrabayu, S.T., M.T.,
M.Bus.Sys., IPM., ASEAN Eng.
NIP: 19750716 200212 1 004

PERNYATAAN KEASLIAN SKRIPSI DAN PELIMPAHAN HAK CIPTA

Dengan ini saya menyatakan bahwa, skripsi berjudul “Sistem Deteksi dan Hitung Objek Kentang Multi-Skala secara *Real-Time*” adalah benar karya saya dengan arahan dari pembimbing Prof. Dr. Ir. Indrabayu, M.T., M.Bus.Sys. sebagai Pembimbing Utama. Karya ilmiah ini belum diajukan dan tidak sedang diajukan dalam bentuk apa pun kepada perguruan tinggi mana pun. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan maupun tidak diterbitkan dari penulis lain telah disebutkan dalam teks dan dicantumkan dalam Daftar Pustaka skripsi ini. Apabila di kemudian hari terbukti atau dapat dibuktikan bahwa sebagian atau keseluruhan skripsi ini adalah karya orang lain, maka saya bersedia menerima sanksi atas perbuatan tersebut berdasarkan aturan yang berlaku.

Dengan ini saya melimpahkan hak cipta (hak ekonomis) dari karya tulis saya berupa skripsi ini kepada Universitas Hasanuddin.

Makassar, 01 April 2026

AUDY FEBRYANTI
NIM D121211005

UCAPAN TERIMA KASIH

Puji dan syukur senantiasa penulis panjatkan ke hadirat Allah SWT atas segala rahmat, karunia, dan kemudahan-Nya, sehingga penulis dapat menyelesaikan skripsi yang berjudul “Sistem Deteksi dan Hitung Objek Kentang Multi-Skala secara *Real-Time*”. Penulis menyampaikan rasa terima kasih yang sebesar-besarnya kepada kedua orang tua, kakak, adik, nenek, dan almarhum kakek penulis atas cinta kasih yang tak terhingga, doa yang tiada henti, serta dukungan moral maupun material yang senantiasa diberikan kepada penulis. Penulis menyadari bahwa penyelesaian skripsi ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, pada kesempatan ini penulis menyampaikan terima kasih yang tulus kepada:

1. Prof. Dr. Ir. Indrabayu, S.T., M.T., M.Bus.Sys., IPM., ASEAN. Eng., selaku dosen pembimbing atas waktu, ilmu, dan arahan yang diberikan selama proses penyelesaian skripsi ini.
2. Prof. Dr. Ir. Zahir Zainuddin, M.Sc. dan Ir. Anugrayani Bustamin, S.T., M.T., selaku dosen penguji atas masukan dan arahan yang membangun demi penyempurnaan skripsi ini.
3. Pak Arbi, selaku pemilik kebun kentang yang menyediakan media berupa tanah perkebunan kentang.
4. Sahabat penulis yakni Andini, Annisa, Nadya, Nadia, Fathanah, Faysa, dan Fira atas persahabatan yang tulus, motivasi, serta kesediaan menjadi tempat berbagi keluh kesah di tengah padatnya proses pengerjaan tugas akhir.
5. Pihak-pihak yang kebersamai penulis di Laboratorium AI (*Artificial Intelligence*) yakni Nadya, Jessica, Tasya, Misna, Fatur, Radya, Syawal, Wira, Shobur, Dzaky, Fathur, Kak Sofyan, Kak Herlina, Kak Abdi, Ibu Syarifah, Kak Erwin, Kak Ulil, Kak Syahid, Ica, Rafiq, Nabila, Suci, atas ruang diskusi yang produktif, kolaborasi teknis, serta suasana kekeluargaan yang meringankan beban penelitian. Tak lupa pula untuk rekan-rekan lain yang tidak dapat penulis sebutkan satu per satu.
6. Teman-teman Kisah A (Dillah, Icha, Ardria, dan Astri) atas kebersamaan dan dukungan yang diberikan selama masa studi.
7. Algo21thm atas kontribusi dan bantuan akademis yang telah diberikan kepada penulis selama masa studi.

Akhir kata, penulis berharap semoga skripsi ini dapat memberikan manfaat dan kontribusi yang positif bagi perkembangan ilmu pengetahuan dan bagi para pembaca.

Penulis

Audy Febryanti

ABSTRAK

AUDY FEBRYANTI. **Sistem Deteksi dan Hitung Objek Kentang Multi-Skala secara *Real-Time*** (dibimbing oleh Indrabayu).

Latar belakang. Proses penghitungan hasil panen kentang di lapangan umumnya masih dilakukan secara manual sehingga memerlukan waktu yang lama dan rentan terhadap kesalahan pencatatan. Perkembangan teknologi visi komputer dan *deep learning* membuka peluang untuk mengotomatisasi proses deteksi dan penghitungan objek secara *real-time*. **Tujuan.** Penelitian ini bertujuan mengembangkan sistem deteksi dan hitung objek kentang multi-skala secara *real-time* serta menganalisis kinerjanya pada berbagai kondisi operasional. **Metode.** Sistem dibangun menggunakan YOLOv5 untuk deteksi objek dan ByteTrack untuk pelacakan objek. Penelitian dilakukan melalui seleksi lima varian YOLOv5, pengujian konvergensi model, pengujian *preprocessing* menggunakan *unsharp masking*, serta evaluasi sistem *counting* berdasarkan variasi ketinggian kamera, sudut pengambilan gambar, dan kecepatan pergerakan alat. **Hasil.** Varian YOLOv5s terpilih sebagai model terbaik dengan *precision* 0.996, *recall* 0.990, *F1-score* 0.993, *mAP50* 0.995, dan *mAP50-95* 0.917, serta *inference time* 5.9 ms. Penerapan *unsharp masking* meningkatkan kemampuan deteksi, terutama pada *recall* dan *mAP50-95*, tetapi meningkatkan waktu *preprocessing* dari 1.9 ms menjadi 111.2 ms dan menurunkan performa dari 19.06 FPS menjadi 16.11 FPS. Pada pengujian *counting*, konfigurasi terbaik tanpa *unsharp masking* diperoleh pada ketinggian 110–120 cm, sudut 80°, dan kecepatan 0.25 m/s, sedangkan dengan *unsharp masking* diperoleh pada ketinggian 90 cm, sudut 80°, dan kecepatan 0.25 m/s. **Kesimpulan.** Sistem yang dikembangkan mampu mendeteksi dan menghitung objek kentang secara *real-time*, tetapi performa *counting* sangat dipengaruhi oleh guncangan alat yang mengganggu kestabilan *tracking* objek.

Kata kunci: visi komputer, deteksi objek, *counting*, *unsharp masking*, *real-time*

ABSTRACT

AUDY FEBRYANTI. **A Real-Time Multi-Scale Potato Object Detection and Counting System** (supervised by Indrabayu).

Background. The process of counting potato harvest yields in the field is generally still performed manually, which requires considerable time and is prone to recording errors. Advances in computer vision and deep learning technologies provide opportunities to automate object detection and counting processes in real time. **Aim.** This study aims to develop a real-time multi-scale potato detection and counting system and evaluate system performance against variations in operational data acquisition parameters. **Method.** This study aims to develop a real-time multi-scale potato object detection and counting system and to analyze its performance under various operational conditions. **Methods.** The system was developed using YOLOv5 for object detection and ByteTrack for object tracking. The research was conducted through the selection of five YOLOv5 variants, model convergence testing, preprocessing evaluation using unsharp masking, and counting system evaluation based on variations in camera height, camera angle, and tool movement speed. **Results.** The YOLOv5s variant was selected as the best model, achieving a precision of 0.996, recall of 0.990, F1-score of 0.993, mAP50 of 0.995, and mAP50–95 of 0.917, with an inference time of 5.9 ms. The application of unsharp masking improved detection performance, particularly in recall and mAP50–95, but increased preprocessing time from 1.9 ms to 111.2 ms and reduced system performance from 19.06 FPS to 16.11 FPS. In the counting experiments, the best configuration without unsharp masking was obtained at a camera height of 110–120 cm, an angle of 80°, and a speed of 0.25 m/s, while with unsharp masking the best performance was achieved at a camera height of 90 cm, an angle of 80°, and a speed of 0.25 m/s. **Conclusion.** The developed system is capable of detecting and counting potato objects in real time; however, counting performance is strongly affected by mechanical vibrations that disrupt the stability of object tracking.

Keywords: computer vision, object detection, counting, unsharp masking, real-time.

DAFTAR ISI

Halaman

ABSTRAK	i
ABSTRACT	ii
DAFTAR ISI	iii
DAFTAR TABEL	v
DAFTAR GAMBAR	vi
DAFTAR LAMPIRAN	viii
DAFTAR ISTILAH/SINGKATAN/SIMBOL	ix
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Landasan Teori	3
1.2.1 Kentang	3
1.2.2 Citra Digital	4
1.2.3 Visi Komputer	5
1.2.4 <i>Unsharp Masking</i>	6
1.2.5 Deteksi Objek	7
1.2.6 YOLO (<i>You Only Look Once</i>)	8
1.2.7 YOLOv5	9
1.2.8 <i>Object Tracking</i>	11
1.2.9 ByteTrack	14
1.2.10 Metrik Evaluasi	15
1.3 Rumusan Masalah	19
1.4 Tujuan dan Manfaat	19
1.4.1 Tujuan	19
1.4.2 Manfaat	19
1.5 Batasan Masalah	19
BAB II METODE PENELITIAN	20
2.1 Tempat dan Waktu Penelitian	20
2.2 Instrumen Penelitian	20
2.3 Tahapan Penelitian	22
2.4 Teknik Pengambilan Data	23
2.5 Perancangan dan Implementasi Sistem	24

2.5.1 Tahap <i>Training</i>	24
2.5.2 Tahap Testing	38
BAB III HASIL DAN PEMBAHASAN	43
3.1 Hasil Pengujian Model YOLOv5	43
3.1.1 YOLOv5 Varian <i>Nano</i>	43
3.1.2 YOLOv5 Varian <i>Small</i>	45
3.1.3 YOLOv5 Varian <i>Medium</i>	47
3.1.4 YOLOv5 Varian <i>Large</i>	50
3.1.5 YOLOv5 Varian <i>Xtra</i>	52
3.1.6 Perbandingan Model	54
3.2 Hasil Pengujian Konvergensi Model YOLOv5s	59
3.3 Hasil Pengujian <i>Preprocessing</i> dengan <i>Unsharp Masking</i>	63
3.4 Hasil Pengujian <i>Counting</i> Model YOLOv5s	67
3.4.1 Skenario Ketinggian	67
3.4.2 Skenario Sudut	73
3.4.3 Skenario Kecepatan	77
3.5 Hasil Pengujian <i>Counting</i> Model YOLOv5s dengan <i>Unsharp Masking</i>	81
3.5.1 Skenario Ketinggian	82
3.5.2 Skenario Sudut	88
3.5.3 Skenario Kecepatan	92
3.6 Performa <i>Real-Time</i>	96
BAB IV KESIMPULAN	98
DAFTAR PUSTAKA	100
LAMPIRAN	104

DAFTAR TABEL

Nomor urut	Halaman
1. Spesifikasi varian YOLOv5.....	11
2. Parameter skenario pengambilan data.....	24
3. Alur Data pada Arsitektur YOLOv5.....	34
4. Referensi Nilai <i>Channel</i> per Varian YOLOv5.....	37
5. Konfigurasi <i>hyperparameter training</i> untuk seleksi varian.....	37
6. Konfigurasi <i>hyperparameter training</i> untuk konvergensi model.....	38
7. Performa pengujian YOLOv5 varian <i>nano</i>	44
8. Performa pengujian YOLOv5 varian <i>small</i>	47
9. Performa pengujian YOLOv5 varian <i>medium</i>	49
10. Performa pengujian YOLOv5 varian <i>large</i>	51
11. Performa pengujian YOLOv5 varian <i>xtra</i>	53
12. <i>Confusion matrix</i> YOLOv5s 100 <i>epoch</i> dan 1000 <i>epoch</i>	61
13. <i>Confusion matrix</i> YOLOv5s tanpa <i>unsharp masking</i> dan dengan <i>unsharp masking</i>	63
14. Hasil pengujian model YOLOv5s pada ketinggian 80 cm.....	67
15. Hasil pengujian model YOLOv5s pada ketinggian 90 cm.....	68
16. Hasil pengujian model YOLOv5s pada ketinggian 100 cm.....	69
17. Hasil pengujian model YOLOv5s pada ketinggian 110 cm.....	69
18. Hasil pengujian model YOLOv5s pada ketinggian 120 cm.....	70
19. Hasil pengujian model YOLOv5s pada ketinggian 130 cm.....	71
20. Hasil pengujian model YOLOv5s pada ketinggian 140 cm.....	72
21. Perbandingan skenario ketinggian pada model YOLOv5s.....	72
22. Hasil pengujian model YOLOv5s pada sudut 80°.....	73
23. Hasil pengujian model YOLOv5s pada sudut 90°.....	75
24. Perbandingan skenario sudut pada model YOLOv5s.....	76
25. Hasil pengujian model YOLOv5s pada kecepatan 0.25 m/s.....	77
26. Hasil pengujian model YOLOv5s pada kecepatan 0.5 m/s.....	78
27. Hasil pengujian model YOLOv5s pada kecepatan 0.75 m/s.....	79
28. Hasil pengujian model YOLOv5s pada kecepatan 1 m/s.....	80
29. Perbandingan skenario kecepatan pada model YOLOv5s.....	81
30. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada ketinggian 80 cm.....	82
31. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada ketinggian 90 cm.....	83
32. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada ketinggian 100 cm.....	84
33. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada ketinggian 110 cm.....	84
34. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada ketinggian 120 cm.....	85
35. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada ketinggian 130 cm.....	86
36. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada ketinggian 140 cm.....	87
37. Perbandingan skenario ketinggian pada YOLOv5s dengan <i>unsharp masking</i>	88
38. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada sudut 80°.....	88
39. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada sudut 90°.....	90
40. Perbandingan skenario sudut pada YOLOv5s dengan <i>unsharp masking</i>	91
41. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada kecepatan 0.25 m/s.....	92
42. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada kecepatan 0.5 m/s.....	93
43. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada kecepatan 0.75 m/s.....	94
44. Hasil pengujian YOLOv5s dengan <i>unsharp masking</i> pada kecepatan 1 m/s.....	95
45. Perbandingan skenario kecepatan pada YOLOv5s dengan <i>unsharp masking</i>	96

DAFTAR GAMBAR

Nomor urut	Halaman
1. Proses panen kentang secara manual	4
2. Tahapan <i>two-stage detector</i>	7
3. Tahapan <i>single-stage detector</i>	7
4. <i>Timeline</i> perkembangan YOLO	8
5. Mekanisme kerja YOLO	9
6. Arsitektur YOLOv5.....	10
7. <i>Single-object tracking</i>	12
8. <i>Multiple-object tracking</i>	13
9. <i>Online tracking</i>	13
10. <i>Offline tracking</i>	14
11. <i>ByteTrack workflow</i>	15
12. <i>Intersection of Union</i>	16
13. <i>Confusion Matrix</i>	16
14. Lokasi Penelitian: (a) Laboratorium <i>Artificial Intelligence</i> (AI); (b) Kebun kentang Gowa	20
15. Tahapan Penelitian	22
16. Skenario pengambilan data.....	23
17. Diagram Alir Sistem.....	24
18. Tampilan visual kentang: (a) Sesaat setelah dikeluarkan dari tanah; (b) Setelah dikeringkan selama beberapa jam.....	25
19. Distribusi piksel untuk citra kentang: (a) <i>Red channel</i> ; (b) <i>Green channel</i> ; (c) <i>Blue channel</i>	25
20. Kumpulan video dataset	26
21. Hasil ekstraksi video	27
22. Sampel dataset: (a) Citra dengan objek kentang; (b) Citra latar belakang.	28
23. Anotasi dataset menggunakan Roboflow: (a) Citra dengan kelas kentang; (b) Citra tanpa kelas kentang.	29
24. Sampel Dataset.....	29
25. Blok CBS	30
26. Blok C3.....	31
27. Blok SPPF	33
28. Perbandingan citra pada tahap <i>preprocessing</i> : (a) Citra original; (b) Citra dengan <i>unsharp masking</i>	39
29. Area ROI	40
30. Posisi <i>counting line</i>	40
31. Visualisasi hasil deteksi kentang	41
32. Visualisasi ID pada objek hasil deteksi.....	41
33. Ilustrasi <i>event counting</i>	42
34. <i>Confusion matrix</i> YOLOv5 varian <i>nano</i>	43
35. Hasil deteksi YOLOv5 varian <i>nano</i> : (a) Sampel <i>false positive</i> ; (b) Sampel <i>false negative</i>	44
36. <i>Confusion matrix</i> YOLOv5 varian <i>small</i>	46
37. Hasil deteksi YOLOv5 varian <i>small</i> : (a) Sampel <i>false positive</i> ; (b) Sampel <i>false negative</i>	46
38. <i>Confusion matrix</i> YOLOv5 varian <i>medium</i>	48
39. Hasil deteksi YOLOv5 varian <i>medium</i> : (a) Sampel <i>false positive</i> ; (b) Sampel <i>false negative</i>	48
40. <i>Confusion matrix</i> YOLOv5 varian <i>large</i>	50

41. Hasil deteksi YOLOv5 varian <i>large</i> : (a) Sampel <i>false positive</i> ; (b) Sampel <i>false negative</i>	51
42. <i>Confusion matrix</i> YOLOv5 varian <i>xtra</i>	52
43. Hasil deteksi YOLOv5 varian <i>xtra</i> : (a) Sampel <i>false positive</i> ; (b) Sampel <i>false negative</i>	53
44. Perbandingan <i>precision</i>	55
45. Perbandingan <i>recall</i>	55
46. Perbandingan <i>F1-score</i>	56
47. Perbandingan <i>mAP50</i>	56
48. Perbandingan <i>mAP50-95</i>	57
49. Perbandingan <i>model size</i>	57
50. Perbandingan <i>preprocessing time</i>	58
51. Perbandingan <i>inference time</i>	58
52. Perbandingan <i>postprocessing time</i>	59
53. Grafik metrik <i>train object loss</i> YOLOv5s 100 <i>epoch</i>	60
54. Grafik metrik <i>train object loss</i> YOLOv5s 1000 <i>epoch</i>	60
55. Perbandingan hasil deteksi YOLOv5s 100 <i>epoch</i> dan YOLOv5s 1000 <i>epoch</i> : (a) Sampel <i>false positive</i> ; (b) Sampel <i>false negative</i>	62
56. Perbandingan metrik antara YOLOv5s 100 <i>epoch</i> dan YOLOv5s 1000	62
57. Perbandingan hasil deteksi YOLOv5s tanpa <i>unsharp masking</i> dan dengan <i>unsharp masking</i> (a) Sampel <i>false positive</i> ; (b) Sampel <i>false negative</i>	64
58. Perbandingan metrik antara YOLOv5s tanpa <i>unsharp masking</i> dan dengan <i>unsharp masking</i>	65
59. Perbandingan waktu pemrosesan antara YOLOv5s tanpa <i>unsharp masking</i> dan dengan <i>unsharp masking</i>	66

DAFTAR LAMPIRAN

Nomor urut	Halaman
1. Source code	104

DAFTAR ISTILAH/SINGKATAN/SIMBOL

Istilah	Arti dan Penjelasan
<i>Backbone</i>	Bagian utama arsitektur jaringan neural yang bertugas mengekstraksi fitur visual dari citra masukan pada berbagai skala, mulai dari fitur tingkat rendah hingga fitur semantik tingkat tinggi.
Bedengan	Area tanah yang ditinggikan dan dipersiapkan khusus untuk penanaman tanaman.
<i>Bottleneck</i>	Hambatan atau titik kemacetan dalam aliran data sistem yang membatasi kinerja keseluruhan.
<i>Bounding Box</i>	Kotak batas imajiner berbentuk persegi panjang yang membungkus objek yang terdeteksi oleh sistem untuk menandai lokasinya.
ByteTrack	Algoritma <i>Multi-Object Tracking</i> yang mengintegrasikan deteksi dengan skor keyakinan rendah ke dalam proses pelacakan untuk meningkatkan akurasi dan mengurangi kehilangan objek.
<i>Centroid</i>	Titik pusat geometri dari suatu <i>bounding box</i> .
<i>Computer Vision</i>	Bidang ilmu komputer yang mengembangkan sistem agar mampu memproses, menganalisis, dan memahami citra digital atau video layaknya penglihatan manusia.
<i>Confidence Score</i>	Nilai probabilitas atau tingkat keyakinan model terhadap prediksi objek yang dihasilkannya.
<i>Confusion Matrix</i>	Tabel evaluasi yang merangkum hasil prediksi model dengan menampilkan jumlah <i>True Positive</i> , <i>False Positive</i> , <i>False Negative</i> , dan <i>True Negative</i> secara terstruktur.
<i>Counting Line</i>	Garis virtual horizontal yang ditetapkan dalam <i>frame</i> video.
<i>Convolutional Neural Network (CNN)</i>	Jenis jaringan saraf tiruan yang dirancang khusus untuk memproses data berbentuk grid seperti citra, dengan menggunakan operasi konvolusi untuk mengekstraksi fitur secara hierarkis.
<i>Deep Learning</i>	Sub-bidang <i>machine learning</i> yang menggunakan jaringan saraf tiruan berlapis banyak (<i>neural networks</i>) untuk mempelajari fitur data secara hierarkis.
<i>Epoch</i>	Satu siklus penuh pelatihan model di mana seluruh data latih telah digunakan satu kali untuk memperbarui bobot jaringan.
<i>False Negative (FN)</i>	Kondisi di mana model gagal mendeteksi suatu objek yang sebenarnya ada dalam citra.

Istilah	Arti dan Penjelasan
<i>False Positive (FP)</i>	Kondisi di mana model mendeteksi suatu objek padahal sebenarnya bukan objek target.
<i>Feature Extraction</i>	Proses pengambilan representasi fitur penting dari data mentah (citra) oleh jaringan neural untuk digunakan dalam proses klasifikasi atau lokalisasi objek.
<i>Feature Map</i>	Peta fitur yang dihasilkan oleh lapisan konvolusi yang merepresentasikan respons filter terhadap pola tertentu dalam citra pada posisi spasial tertentu.
<i>Ground Truth (GT)</i>	Data kebenaran aktual atau label referensi yang dibuat secara manual (anotasi) sebagai acuan untuk melatih dan menguji model.
<i>Head</i>	Bagian akhir dari arsitektur deteksi objek yang bertugas memprediksi kelas objek, <i>confidence score</i> dan koordinat <i>bounding box</i> .
<i>Hungarian Algorithm</i>	Algoritma optimasi kombinatorial yang digunakan dalam proses asosiasi data pada ByteTrack untuk mencocokkan <i>tracklet</i> dengan deteksi baru secara optimal.
<i>Hyperparameter</i>	Parameter konfigurasi yang nilainya ditentukan sebelum pelatihan model dimulai.
<i>ID Switch</i>	Fenomena dalam <i>object tracking</i> di mana ID unik suatu objek berubah secara tidak tepat ke objek lain, menyebabkan ketidakkonsistensian dalam pelacakan.
<i>Inference</i>	Proses penggunaan model yang telah dilatih untuk melakukan prediksi pada data baru yang belum pernah dilihat sebelumnya.
<i>Intersection over Union</i>	Metrik yang mengukur seberapa akurat tumpang tindih antara area prediksi model dengan area sebenarnya.
<i>Machine Learning</i>	Cabang kecerdasan buatan yang memungkinkan sistem belajar dari data tanpa diprogram secara eksplisit untuk setiap aturan.
<i>Neck</i>	Bagian arsitektur yang menghubungkan <i>Backbone</i> dan <i>Head</i> , berfungsi menggabungkan fitur dari berbagai skala.
<i>Object Detection</i>	Tugas dalam <i>computer vision</i> untuk mengidentifikasi keberadaan, jenis, dan lokasi objek tertentu di dalam citra atau video.
<i>Object Tracking</i>	Proses mengestimasi posisi dan menjaga identitas unik suatu objek di sepanjang urutan <i>frame</i> video.
<i>Occlusion</i>	Kondisi visual di mana suatu objek tertutup sebagian atau seluruhnya oleh objek lain, yang menyulitkan proses deteksi.

Istilah	Arti dan Penjelasan
<i>Overcount</i>	Kondisi di mana sistem menghitung jumlah objek melebihi jumlah aktual yang ada.
<i>Postprocessing</i>	Tahap pasca-pemrosesan data setelah inferensi model, seperti menggambar kotak deteksi atau menghitung objek.
<i>Precision</i>	Rasio prediksi benar positif dibandingkan dengan keseluruhan hasil prediksi positif yang dikeluarkan oleh model.
<i>Preprocessing</i>	Tahap awal pengolahan data mentah sebelum dimasukkan ke dalam model untuk dilatih atau diproses.
<i>Real-Time</i>	Pemrosesan data yang dilakukan secara seketika saat kejadian berlangsung dengan latensi yang sangat rendah.
<i>Recall</i>	Rasio keberhasilan model dalam menemukan kembali seluruh objek target yang sebenarnya ada dalam data.
<i>Region of Interest</i>	Area spesifik pada citra yang dipilih untuk diproses lebih lanjut guna memfokuskan komputasi dan mengurangi noise.
<i>Solanum tuberosum L.</i>	Nama latin (ilmiah) untuk tanaman kentang.
<i>Tracklet</i>	Lintasan parsial suatu objek dalam video yang terdiri dari serangkaian posisi objek pada <i>frame-frame</i> berurutan sebelum lintasan tersebut terhubung penuh.
<i>True Negative (TN)</i>	Kondisi di mana model dengan benar tidak mendeteksi objek pada area yang memang bukan objek target.
<i>True Positive (TP)</i>	Kondisi di mana model berhasil mendeteksi objek yang memang ada dalam citra dengan benar.
<i>Undercount</i>	Kondisi di mana sistem menghitung jumlah objek kurang dari jumlah aktual yang ada.
<i>Unsharp Masking (UM)</i>	Teknik peningkatan kualitas citra yang mempertajam detail tepi dengan mengurangi versi blur dari citra asli, sehingga memperkuat fitur visual objek untuk deteksi yang lebih baik.
YOLOv5	Versi kelima dari keluarga YOLO yang tersedia dalam beberapa varian ukuran seperti <i>nano</i> , <i>small</i> , <i>medium</i> , <i>large</i> , dan <i>xtra</i> dengan <i>trade-off</i> berbeda antara akurasi dan efisiensi komputasi.

Lambang/singkatan	Arti dan Penjelasan
AP	<i>Average Precision</i>
CBS	<i>Convolution with Batch Normalization and SiLU</i>
CNN	<i>Convolutional Neural Network</i>
C3	<i>CSP bottleneck with three convolution layer</i>
CSP	<i>Cross Stage Partial</i>
FN	<i>False Negative</i>
FP	<i>False Positive</i>
FPN	<i>Feature Pyramid Network</i>
FPS	<i>Frame Per Second</i>
GPU	<i>Graphics Processing Unit</i>
GT	<i>Ground Truth</i>
ID	<i>Identifier</i>
IoU	<i>Intersection over Union</i>
MAE	<i>Mean Absolute Error</i>
mAP	<i>Mean Average Precision</i>
MB	<i>Megabyte</i>
MOT	<i>Multi-Object Tracking</i>
NMS	<i>Non-Maximum Suppression</i>
P	<i>Precision</i>
PAN	<i>Path Aggregation Network</i>
R	<i>Recall</i>
RMSE	<i>Root Mean Square Error</i>
ROI	<i>Region of Interest</i>
SGD	<i>Stochastic Gradient Descent</i>
SOT	<i>Single Object Tracking</i>
SPPF	<i>Spatial Pyramid Pooling Fast</i>
UM	<i>Unsharp Masking</i>
TP	<i>True Positive</i>
YOLO	<i>You Only Look Once</i>
YOLOv5s	<i>You Only Look Once versi ke-5</i>

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sektor pertanian berperan penting dalam mendukung pertumbuhan dan ketahanan ekonomi nasional. Dalam kerangka pembangunan pertanian, subsektor hortikultura menjadi salah satu fokus karena berkontribusi pada penyediaan pangan bergizi, penciptaan nilai tambah industri, serta penguatan pendapatan rumah tangga tani. Salah satu komoditas hortikultura unggulan adalah kentang. Kentang (*Solanum tuberosum* L.) memiliki nilai ekonomis tinggi dan berpotensi sebagai bahan baku olahan makanan, sekaligus sumber karbohidrat dengan kandungan gizi penting, termasuk vitamin C dan mineral seperti kalium. Peningkatan produksi kentang secara nasional dipandang strategis untuk memperluas peluang ekspor, mengimbangi impor, serta memenuhi kebutuhan industri dan konsumsi dalam negeri, sehingga diharapkan turut meningkatkan pendapatan petani dan kesejahteraan masyarakat (Maulidiyah, 2024).

Urgensi penguatan komoditas kentang juga tercermin pada kebijakan pemerintah yang menempatkan kentang sebagai komoditas yang perlu diatur dari sisi mutu-keamanan pangan dan tata niaga. Dari perspektif keamanan pangan, Peraturan Badan Pangan Nasional terkait batas maksimal residu pestisida pada pangan segar asal tumbuhan memasukkan kentang sebagai komoditas yang menjadi objek pengawasan residu, yang menegaskan pentingnya standar mutu dan keamanan kentang pada rantai pasok domestik (Badan Pangan Nasional, 2024b). Dari perspektif tata niaga, regulasi impor pemerintah turut mencantumkan kentang dalam pengaturan impor barang pertanian/peternakan serta barang konsumsi, termasuk bentuk segar/dingin maupun olahan tertentu, yang mengindikasikan sensitivitas komoditas kentang terhadap stabilitas pasokan dan dinamika pasar (Kementerian Perdagangan, 2025a, 2025b). Selain itu, kebijakan panganekaragaman pangan melalui standardisasi mutu produk pangan lokal memperlihatkan arah penguatan pangan non-beras, yang secara konseptual relevan sebagai konteks diversifikasi pangan meskipun tidak menempatkan kentang sebagai komoditas yang disebut secara eksplisit di dalam daftar jenis pangan pokok lokal yang distandardisasi (Badan Pangan Nasional, 2024a).

Dari sisi kinerja produksi, data statistik hortikultura menunjukkan adanya fluktuasi produksi kentang dari tahun ke tahun. Publikasi Statistik Hortikultura BPS (ATAP) memuat data produksi kentang dan komoditas hortikultura strategis lainnya yang menjadi rujukan resmi untuk evaluasi kinerja serta perumusan kebijakan (Badan Pusat Statistik, 2023, 2024, 2025). Dengan demikian, dinamika produksi tersebut mengindikasikan adanya tantangan produktivitas dan efisiensi usaha tani kentang yang perlu dijawab melalui inovasi teknologi dan perbaikan manajemen budidaya (Maulidiyah, 2024).

Produktivitas berkaitan dengan efisiensi, utamanya efisiensi teknis dan efisiensi ekonomi. Efisiensi teknis merupakan kemampuan unit produksi untuk mencapai output maksimum pada tingkat teknologi tertentu, sedangkan efisiensi ekonomi dihitung berdasarkan perbandingan antara nilai produk marjinal dengan biaya korbanan marjinal. Namun, fakta di lapangan sering kali menunjukkan bahwa keadaan usaha tani yang efisien secara teknis dan ekonomi belum tercapai (Maulidiyah, 2024). Kondisi ini semakin menegaskan perlunya intervensi teknologi pada tahapan-tahapan kritis produksi kentang yang berkontribusi besar terhadap kehilangan hasil dan biaya.

Salah satu tahap krusial dalam produksi kentang adalah pemanenan, yang dilakukan dengan memisahkan dan mengumpulkan umbi kentang dari tanah. Pada proses ini, sering terjadi ketidakefektifan karena umbi kentang tertinggal di lahan atau mengalami kerusakan mekanis sehingga menimbulkan kerugian hasil (*losses*) (Johnson & Auat Cheein, 2023). Kerugian pada tahap panen tidak hanya mengurangi kuantitas hasil, tetapi juga dapat menurunkan mutu umbi yang berdampak pada harga jual serta kesesuaian bahan baku untuk industri pengolahan.

Metode pemanenan terus berkembang dari waktu ke waktu. Pemanenan manual dengan tangan menggunakan alat seperti cangkul atau garpu tanah, atau bahkan tanpa alat, membutuhkan banyak waktu dan tenaga. Selain itu, pemanenan dengan bantuan hewan berisiko melukai hewan. Untuk mengatasi keterbatasan metode manual dan tenaga hewan, sistem mekanisasi dalam pemanenan mulai dikembangkan. Pemanenan dengan mesin mengumpulkan umbi kentang dalam trolis yang dapat menghemat waktu, energi, dan mengurangi kerugian hasil (*losses*) (Johnson & Auat Cheein, 2023). Namun, pemanenan dengan mesin tetap membutuhkan tenaga kerja, dan proses pengumpulan berpotensi turut mengambil objek non-target seperti tanah, batu, residu tanaman sehingga dapat memengaruhi efisiensi dan kualitas sortasi awal pascapanen.

Saat ini, otomatisasi gencar digunakan dalam berbagai sektor, termasuk pertanian, khususnya dalam proses pemanenan. Dengan meniru kemampuan visual dan kognitif manusia, kombinasi visi komputer, kecerdasan buatan, dan *machine learning* memungkinkan pengambilan keputusan yang lebih tepat untuk suatu tugas. Visi komputer digunakan untuk memproses dan menginterpretasikan informasi visual dari lingkungan sekitar, sementara kecerdasan buatan dan *machine learning* digunakan untuk mengenali pola dan memprediksi tindakan. Otomatisasi dengan kombinasi ketiga teknologi tersebut memungkinkan proses panen yang lebih efisien serta dapat bekerja dengan intervensi manusia minimal dan hasil yang presisi. Selain itu, petani dapat memperoleh informasi seperti estimasi jumlah hasil panen, yang membantu pengambilan keputusan dan pengoptimalan proses pertanian (Ghazal et al., 2024). Penghitungan objek berbasis visi komputer juga mendukung keputusan pertanian berbasis data, prediksi kesehatan tanaman, dan estimasi hasil panen. Penggunaan metode berbasis *deep learning*, terutama *convolutional neural networks* (CNN), telah meningkatkan akurasi penghitungan dan memungkinkan pengumpulan data hasil panen secara cepat dan *real-time*, sehingga mengurangi kendala penghitungan manual yang lambat dan tidak konsisten (Farjon et al., 2023).

Perancangan sistem panen otomatis berbasis visi komputer melibatkan penerapan algoritma deteksi dan pelacakan objek untuk mempermudah proses deteksi dan perhitungan hasil panen secara *real-time*. Dalam pengembangan sistem ini perlu mempertimbangkan kompleksitas model, keterbatasan sumber daya (daya komputasi, memori, dan daya tahan baterai), serta kebutuhan pemrosesan *real-time* (Ghazal et al., 2024). YOLO (*You Only Look Once*) merupakan salah satu pendekatan deteksi objek yang dirancang untuk kebutuhan *real-time* dengan membagi gambar masukan ke dalam matriks grid, serta memprediksi *bounding box* dan probabilitas kelas secara bersamaan. Berdasarkan analisis pada YOLOv1 hingga YOLOv10 di berbagai domain, YOLOv5 dilaporkan memiliki kinerja deteksi yang cepat dan sesuai untuk aplikasi *real-time* (Alif & Hussain, 2024). Untuk mendukung deteksi dan pelacakan multi-objek secara *real-time*, diperlukan algoritma pelacakan yang efisien. ByteTrack merupakan algoritma pelacakan multi-objek dengan kebutuhan perangkat keras relatif rendah dan pemrosesan cepat karena struktur yang sederhana. ByteTrack juga mempertimbangkan *confidence* tinggi hingga rendah untuk menyeimbangkan kecepatan deteksi dan akurasi (Bajraktari & Toylan, 2025).

Penelitian sebelumnya oleh Xiao et al. (2025) mengembangkan sistem metering otomatis menggunakan YOLOv5n yang ditingkatkan dan ByteTrack untuk mendeteksi dan menghitung bibit kentang serta alat pengambil bibit (sendok tanam) dalam proses penanaman secara *real-time*. Metode ini mencapai mAP@0.5 sebesar 98.0% dan mAP@0.5:0.95 sebesar 80.8% dengan akurasi hitung 96.6%. Studi lain oleh (Shen et al., 2023) membahas penggunaan YOLOv5 untuk mendeteksi dan menghitung tandan buah anggur di kebun secara *real-time*, dengan tantangan utama berupa ukuran multi-skala, oklusi, dan warna objek yang serupa dengan lingkungan. Dengan penggunaan YOLOv5 dan SORT (*Simple Online and Realtime Tracking*), sistem mencapai mAP 84.9% dengan kecepatan deteksi 50.4 FPS. Temuan-temuan tersebut menunjukkan bahwa integrasi deteksi dan pelacakan objek berbasis *deep learning* memiliki potensi kuat untuk mendukung otomasi pada komoditas hortikultura, termasuk kentang, terutama pada kondisi lapangan yang menuntut ketahanan terhadap variasi skala, oklusi, dan kompleksitas latar.

Berdasarkan pemaparan tersebut, penelitian ini mengusulkan pengembangan “Sistem Deteksi dan Hitung Objek Kentang Multi-Skala secara *Real-Time*”. Penelitian ini bertujuan mengembangkan model deteksi berbasis YOLOv5 dan ByteTrack yang dapat diintegrasikan dengan sistem robotik pertanian sebagai solusi otomasi proses panen kentang.

1.2 Landasan Teori

1.2.1 Kentang

Kentang (*Solanum tuberosum* L.) merupakan salah satu tanaman hortikultura yang menjadi sumber pangan utama keempat di dunia setelah gandum, beras, dan jagung (Johnson & Auat Cheein, 2023). Di Indonesia, kentang banyak dibudidayakan karena

memiliki nilai ekonomis yang tinggi. Hal ini ditunjukkan oleh harga kentang yang relatif stabil, potensi bisnis yang stabil dan terjamin, dan sifat daya simpan yang lebih tahan lama (Maulidiyah, 2024). Selain berperan sebagai alternatif sumber pangan, kentang memiliki beragam kandungan seperti vitamin C, B6, B1, folate, protein berkualitas tinggi, mineral potassium, fosfor, kalsium dan magnesium, juga kandungan mikronutrien besi dan seng (Sari, 2021).



Gambar 1. Proses panen kentang secara manual

Sumber: Photo by Manuel Parra: <https://www.pexels.com/photo/cultivando-la-tierra-27611557/>

Budidaya kentang umumnya dilakukan di daerah dingin (suhu berkisar 15°C - 20°C) yang memiliki ketinggian di atas 1000 MDPL agar kentang dapat tumbuh subur dan sesuai dengan standar mutu (Sitepu et al., 2021). Tahapan dalam proses budidaya kentang meliputi pengolahan lahan, persiapan bibit, penanaman, pemupukan, penyiangan, penyemprotan hingga panen. Penentuan waktu panen dan penanganan panen memiliki standar yang dapat diketahui melalui perhitungan umur dan visual. Secara perhitungan umur, pemanenan kentang umumnya berkisar 100 – 110 hari tergantung pada varietas/kultivar, cuaca/musim, dan pemeliharaan tanaman. Secara visual, kentang yang sudah siap dipanen akan mengalami perubahan daun dan batang yang mengering lebih dari 75% dengan warna yang kekuningan. Jika perubahan visual tersebut sudah tampak, daun dapat dipangkas dan ditinggalkan minimal 7 hari. Setelahnya, guludan akan dibongkar menggunakan alat seperti cangkul, lalu umbi dikeluarkan dari tanah dan dijemur di bawah sinar matahari. Pemanenan kentang dilakukan pada saat cuaca cerah, tidak saat hujan atau pun menjelang hujan (Sitepu et al., 2021).

1.2.2 Citra Digital

Citra didefinisikan sebagai representasi visual dari suatu objek yang dapat diamati oleh sistem optik, baik itu mata manusia maupun mesin. Secara fundamental, citra dikategorikan menjadi dua jenis utama, yaitu citra analog dan citra digital. Citra analog bersifat kontinu yang menerima sinyal analog dari sistem optik seperti mata

manusia atau kamera analog. Sementara itu, citra digital merupakan representasi numerik yang bersifat diskrit, yang dihasilkan dari konversi gambar analog dua dimensi melalui proses *sampling* dan kuantisasi (F. et al., 2022).

Dalam konteks komputasi, citra digital dipandang sebagai fungsi dua dimensi $f(x, y)$, di mana x dan y merupakan koordinat spasial, dan nilai amplitudo f pada pasangan koordinat tersebut merepresentasikan intensitas cahaya atau tingkat keabuan (*gray-level*) dari citra pada titik tersebut (F. et al., 2022). Citra digital tersusun atas sekumpulan elemen diskrit yang disebut piksel (*picture element*). Kumpulan piksel ini disusun dalam struktur matriks dua dimensi. Jika sebuah citra memiliki dimensi tinggi M dan lebar N , maka resolusi citra tersebut dinyatakan sebagai $M \times N$. Nilai pada setiap sel matriks dibatasi oleh rentang tertentu yang menentukan kedalaman warna atau derajat keabuan L . Menurut (Syahidi, 2024), secara matematis citra digital dapat direpresentasikan sebagai matriks berikut.

$$f(x, y) \approx \begin{bmatrix} f(0,0) & f(0,1) & \dots & f(0, N-1) \\ f(1,0) & f(1,1) & \dots & f(1, N-1) \\ \vdots & \vdots & \dots & \vdots \\ f(M-1,0) & f(M-1,1) & \dots & f(M-1, N-1) \end{bmatrix} \quad (1)$$

Dimana fungsi tersebut harus memenuhi batasan kondisi sebagai berikut.

$$\begin{aligned} 0 &\leq x \leq M-1 \\ 0 &\leq y \leq N-1 \\ 0 &\leq f(x, y) \leq L-1 \end{aligned}$$

Persamaan di atas menunjukkan bahwa setiap elemen matriks memiliki indeks baris x dan kolom y yang spesifik, serta nilai intensitas f yang terbatas pada rentang derajat keabuan L (misalnya 0-255 untuk citra 8-bit). Representasi numerik ini memungkinkan komputer untuk memproses informasi visual melalui operasi aljabar linier yang menjadi dasar bagi algoritma pengolahan citra dan visi komputer.

1.2.3 Visi Komputer

Visi komputer merupakan sub-bidang dari kecerdasan buatan yang bertujuan untuk mengekstrak dan mengambil informasi yang ada pada data visual seperti gambar atau video (Hussien et al., 2021). Secara sederhana, visi komputer dikembangkan untuk meniru kemampuan persepsi manusia dalam memahami konten pada data visual (Jiang et al., 2023). Mekanisme ini mengadopsi prinsip kerja penglihatan manusia yang terdiri dari dua komponen utama, yaitu mata untuk menangkap detail visual dan otak untuk memproses serta mengekstraksi informasi dari objek yang telah dilihat (Syahsir, 2024).

Secara operasional, visi komputer bekerja dengan mengintegrasikan teknik pemrosesan citra digital (*image processing*) dengan algoritma pembelajaran mesin untuk mengekstraksi fitur, mengenali pola, dan menarik kesimpulan dari data input (Chai et al., 2021). Kemampuan ini mendasari beberapa tugas utama dalam ranah visi komputer, meliputi deteksi objek, klasifikasi, *image captioning*, *image recognition*,

dan lain-lain (Joiya, 2022). Seiring waktu, pengaplikasian visi komputer terus meluas ke berbagai industri, seperti bidang astronomi, kesehatan, industri makanan, manufaktur, kendaraan otonom (Singh et al., 2023), serta pertanian cerdas (Zharikova et al., 2021).

1.2.4 Unsharp Masking

Unsharp Masking (UM) merupakan teknik peningkatan kualitas citra yang bertujuan menajamkan detail tepi (*edge*) pada citra digital. Nama teknik ini berasal dari penggunaan sebuah citra negatif yang diburamkan (*unsharp*) sebagai masker terhadap citra asli, yang kemudian digabungkan untuk menghasilkan citra yang lebih tajam dari sebelumnya. Secara matematis, proses UM direpresentasikan melalui persamaan berikut.

$$S = B + \lambda(I - B) \quad (2)$$

atau secara ekuivalen:

$$S = I + \lambda(I - B) \quad (3)$$

Pada persamaan tersebut, I merepresentasikan citra masukan, sedangkan B adalah *base layer* yang diperoleh melalui penerapan filter *low-pass* (umumnya filter Gaussian) pada citra masukan. Operasi pengurangan $(I - B)$ mengekstraksi perbedaan antara citra masukan dan *base layer*, menghasilkan *detail layer* atau *mask* yang memuat informasi komponen frekuensi tinggi, yakni tepi dan tekstur halus dari citra. Variabel λ bertindak sebagai faktor pengali (*gain* atau *amount*) yang berfungsi mengatur intensitas penguatan lapisan detail tersebut. Variabel S merepresentasikan citra keluaran hasil penajaman, yang diperoleh dengan menambahkan lapisan detail yang telah diskalakan kembali ke citra masukan (Wang et al., 2026).

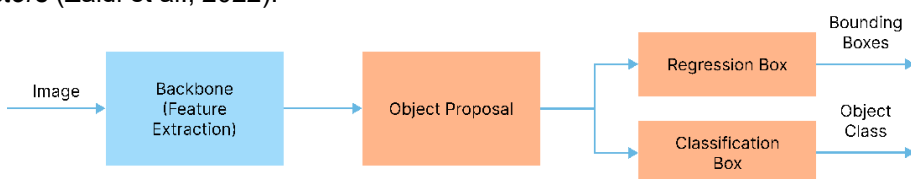
Proses UM melibatkan tiga parameter utama yang dapat dikonfigurasi, yaitu *radius*, *amount*, dan *threshold*. Parameter *radius* menentukan ukuran jangkauan pemburaman (*blur*) yang digunakan untuk membentuk *base layer*, sehingga memengaruhi skala tepi yang akan dipertajam. Parameter *amount* mengontrol kekuatan efek penajaman, di mana nilai yang lebih besar menghasilkan peningkatan kontras yang lebih kuat pada piksel tepi. Sementara itu, parameter *threshold* menetapkan nilai kontras minimum yang diperlukan agar suatu piksel dianggap sebagai tepi dan layak dipertajam, sehingga membantu mencegah penguatan derau (*noise*) pada area yang seragam (Wang et al., 2026).

Meskipun UM terbukti efektif dalam meningkatkan ketajaman citra, teknik ini memiliki keterbatasan berupa sensitivitas tinggi terhadap derau. Nilai *threshold* yang terlalu rendah dapat menyebabkan penguatan derau secara tidak terkendali, sedangkan penajaman yang berlebihan dapat menimbulkan artefak *halo*, yaitu kemunculan pita terang atau gelap di sekitar tepi objek yang dapat menurunkan kualitas visual citra secara keseluruhan.

1.2.5 Deteksi Objek

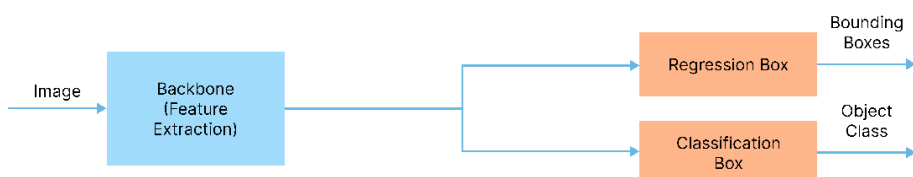
Deteksi objek merupakan ranah fundamental dalam bidang visi komputer yang bertujuan untuk mengidentifikasi dan melokalisasi objek pada gambar atau video secara spesifik (Sun et al., 2024). Dalam prosesnya, deteksi objek harus menyelesaikan dua permasalahan inti yaitu “*what is it*” yang merujuk pada label kelas objek dan “*where is it*” yang menunjukkan koordinat posisi dari objek tersebut (Alhardi & Afeef, 2024). Peran ini menjadikan deteksi objek sebagai dasar dalam banyak tugas visi komputer, seperti *instance segmentation*, *image captioning*, dan *object tracking* (Zou et al., 2023).

Dalam penerapannya, deteksi objek telah digunakan secara luas dalam berbagai sektor di dunia nyata meliputi bidang robotika dan otomasi, kendaraan otonom, sistem pengawasan dan keamanan, kesehatan, *augmented reality*, pemantauan satwa liar, hingga sektor ritel dan pemasaran (Tasnim & Qi, 2023). Meskipun penggunaannya sudah sangat luas, penerapan deteksi objek di lapangan masih menghadapi beberapa tantangan teknis, antara lain penanganan variasi intra-kelas akibat faktor eksternal seperti oklusi dan perubahan pencahayaan, kelangkaan data teranotasi untuk kategori objek yang masif, serta tuntutan efisiensi komputasi agar model dapat beroperasi secara optimal pada perangkat *edge* dengan sumber daya terbatas (Zaidi et al., 2022). Secara umum deteksi objek berbasis *deep learning* terbagi menjadi dua tipe *detector* yaitu *two-stage detectors*, dan *single-stage detectors* (Zaidi et al., 2022).



Gambar 2. Tahapan *two-stage detector*

Two-stage detector. *Two-stage detector* bekerja dengan memisahkan proses deteksi ke dalam dua tahap utama, di mana tahap pertama bertujuan untuk menghasilkan *region proposals* sebagai kandidat lokasi objek, kemudian tahap kedua melakukan klasifikasi objek dan regresi *bounding box* pada proposal yang dihasilkan sebelumnya. Algoritma seperti R-CNN, Faster R-CNN, dan Mask R-CNN termasuk dalam kategori ini. Meskipun unggul dalam akurasi dan presisi lokalisasi (terutama untuk objek kecil), kompleksitas komputasi yang tinggi sering kali membatasi kinerjanya pada aplikasi *real-time* (Zaidi et al., 2022).



Gambar 3. Tahapan *single-stage detector*

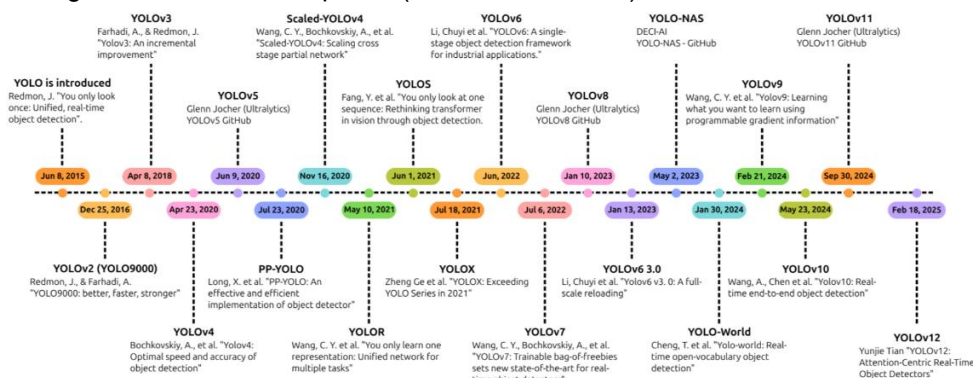
Single-stage detector. *Single-stage detector* menyederhanakan proses deteksi

dengan menggabungkan proses klasifikasi dan lokalisasi dalam satu tahap inferensi. Beberapa algoritma yang merepresentasikan kategori ini meliputi YOLO (*You Only Look Once*), SSD (*Single Shot MultiBox Detector*), dan RetinaNet. Pendekatan ini menawarkan kecepatan inferensi yang lebih tinggi dibandingkan *two-stage detector* sehingga lebih cocok untuk aplikasi *real-time* (Zaidi et al., 2022).

Secara komparatif, *two-stage detectors* cenderung unggul dalam aspek akurasi, sedangkan *single-stage detectors* menonjol dalam kecepatan inferensi. Oleh karena itu, pemilihan arsitektur yang tepat sangat bergantung pada kebutuhan aplikasi, termasuk tuntutan pemrosesan *real-time*, ketersediaan sumber daya komputasi, serta tingkat presisi yang ditargetkan (Zaidi et al., 2022).

1.2.6 YOLO (*You Only Look Once*)

YOLO (*You Only Look Once*) merupakan algoritma detektor satu tahap (*single-stage*) berbasis *Convolutional Neural Network* (CNN) yang memandang deteksi objek sebagai masalah regresi tunggal (Cong et al., 2022). Secara teknis, model ini membagi citra input ke dalam grid $S \times S$, di mana setiap sel pada grid memprediksi *bounding box*, skor keyakinan (*confidence score*), serta probabilitas kelas objek secara bersamaan (Gündüz & Işık, 2023). Mekanisme *end-to-end* ini memungkinkan model memproses keseluruhan konteks citra secara global untuk memprediksi lokasi dan label objek dalam satu kali pemrosesan jaringan. Berkat pendekatan tersebut, YOLO unggul dalam menyeimbangkan kecepatan inferensi *real-time* dan akurasi deteksi (Wei et al., 2025). Seiring perkembangannya, algoritma ini terus berevolusi melalui berbagai iterasi untuk mengatasi keterbatasan arsitektur terdahulu dan meningkatkan efisiensi komputasi (Terven et al., 2023).

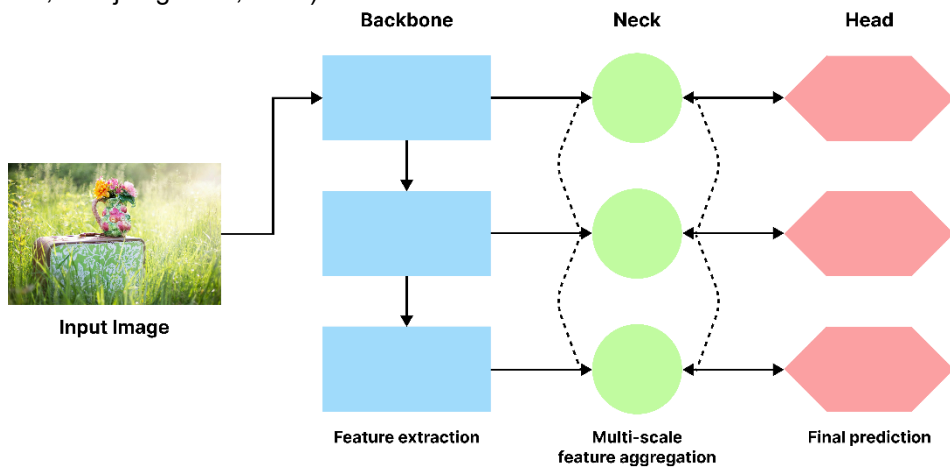


Gambar 4. *Timeline* perkembangan YOLO

Sumber: (Jegham et al., 2025)

Secara arsitektural, mekanisme kerja YOLO terbagi menjadi tiga tahapan utama yaitu *input*, *feature extraction*, dan *prediction*. Pada tahap *input*, citra masukan mengalami pra-pemrosesan dan augmentasi untuk meningkatkan *robustness* model

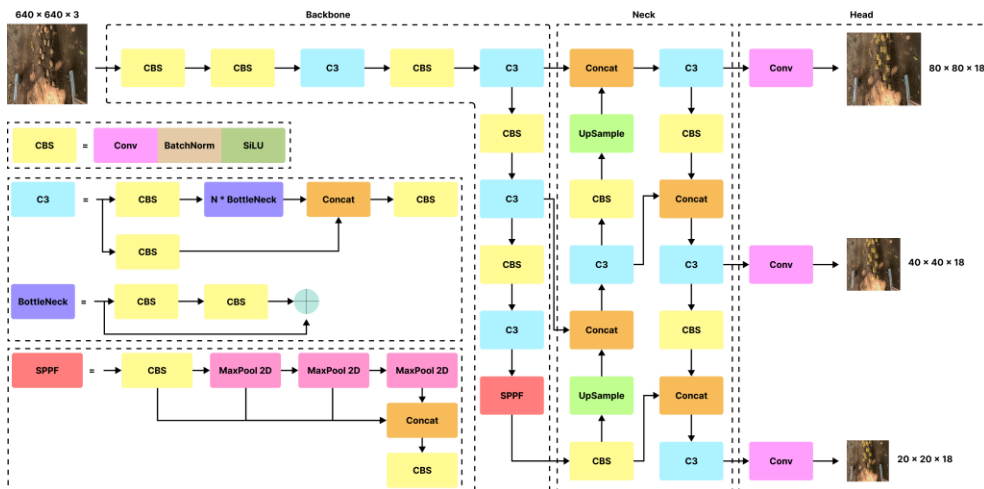
terhadap variasi visual, dengan resolusi input yang dapat disesuaikan. Proses berlanjut ke tahap *feature extraction* yang memanfaatkan *backbone* berbasis CNN untuk mengekstrak fitur visual. Sejak YOLOv2, lapisan *neck* ditambahkan untuk menggabungkan fitur dari berbagai skala guna menangkap pola yang lebih kompleks. Pada tahap *prediction*, model menghasilkan *anchor boxes* dengan berbagai ukuran berdasarkan *feature maps*. Mekanisme *Non-Maximum Suppression* (NMS) kemudian diterapkan untuk mengeliminasi *bounding box* redundan, menghasilkan luaran akhir berupa label kelas objek dan lokasi deteksi (Wei et al., 2025; Yeerjiang et al., 2024).



Gambar 5. Mekanisme kerja YOLO

1.2.7 YOLOv5

YOLOv5 merupakan pengembangan dari keluarga arsitektur *You Only Look Once* (YOLO) yang dirancang untuk menyeimbangkan kecepatan inferensi dan akurasi deteksi. Berdasarkan tinjauan komprehensif arsitektur YOLO oleh Terven et al. (2023), YOLOv5 menawarkan keunggulan signifikan dibandingkan pendahulunya karena dibangun secara *native* menggunakan kerangka kerja PyTorch, yang mempermudah proses *training* dan *deployment* model. Versi 6.0 (v6.0) membawa sejumlah pembaruan arsitektur yang meningkatkan efisiensi komputasi dan kompatibilitas ekspor model secara signifikan (Jocher et al., 2021). Secara struktural, arsitektur YOLOv5 terdiri dari tiga komponen utama yaitu *backbone*, *neck*, dan *head*.



Gambar 6. Arsitektur YOLOv5

Backbone. *Backbone* pada YOLOv5 v6.0 menggunakan struktur CSP-Darknet53 yang berfungsi mengekstraksi fitur citra pada berbagai skala, mulai dari fitur tingkat rendah (*low-level*) pada lapisan dangkal hingga fitur semantik tingkat tinggi (*high-level*) pada lapisan dalam. Struktur *backbone* tersusun atas beberapa modul utama, yaitu CBS, C3, dan SPPF. Pada tahap awal, proses dimulai dengan modul CBS pertama, yang menerapkan lapisan konvolusi (*Convolutional 2D*) berukuran kernel 6×6 dengan *stride 2* dan *padding 2*, menggantikan modul *Focus* pada versi sebelumnya. Penggantian ini dilakukan karena lapisan *Conv2d* 6×6 menawarkan hasil ekstraksi fitur yang setara dengan modul *Focus* namun dengan kompatibilitas yang lebih baik terhadap berbagai *backend* inferensi (Jocher et al., 2021). Lapisan ini mereduksi dimensi spasial citra masukan menjadi setengahnya sekaligus memperluas representasi fitur pada dimensi kanal, yang kemudian dinormalisasi oleh *Batch Normalization* (BN) dan dilewatkan pada fungsi aktivasi SiLU. Selanjutnya, *backbone* terus memproses fitur secara bertahap menggunakan serangkaian modul CBS lainnya. Sejumlah modul CBS disusun dengan koneksi residual untuk membentuk modul C3, di mana peta fitur masukan digabungkan (*concatenated*) dengan keluaran koneksi residual tersebut sehingga membentuk struktur *Cross Stage Partial* (CSP). Tahapan ini bertujuan mengekstraksi informasi untuk fusi fitur pada skala yang berbeda guna meningkatkan penggunaan ulang (*reuse*) fitur dalam jaringan. Pada tahap akhir *backbone*, terdapat modul *Spatial Pyramid Pooling Fast* (SPPF) sebagai pengganti modul SPP pada versi sebelumnya. Modul SPPF menerapkan *max pooling* secara berurutan dengan ukuran kernel yang sama, kemudian menggabungkan seluruh keluarannya, sehingga menghasilkan operasi yang secara fungsional setara dengan SPP namun dengan kecepatan komputasi yang lebih tinggi (Jocher et al., 2021).

Neck. *Neck* berfungsi sebagai penghubung antara *backbone* dan *head* dengan menggabungkan fitur-fitur yang telah diekstraksi. Komponen ini mengombinasikan struktur *Feature Pyramid Network* (FPN) dan *Path Aggregation Network* (PAN) yang pada v6.0 diperbarui menjadi CSP-PAN untuk mengintegrasikan fitur semantik dan fitur posisi secara optimal (Jocher et al., 2021). Proses integrasi dilakukan dengan menggabungkan aliran FPN yang bersifat *top-down* dengan aliran PAN yang bersifat *bottom-up*. Aliran *bottom-up* mengekstraksi peta fitur dari citra asli pada berbagai skala untuk memperoleh fitur posisi yang terperinci (*fine-grained*), sedangkan aliran *top-down* diimplementasikan melalui proses *upsampling* dan koneksi lintas lapisan (*cross-layer connection*). Fusi kedua aliran tersebut menghasilkan serangkaian peta fitur dengan resolusi berbeda, yang dirancang untuk meningkatkan sensitivitas model terhadap objek pada berbagai skala sekaligus meningkatkan akurasi deteksi secara keseluruhan (Huang et al., 2023).

Head. *Head* memproses keluaran dari *neck* untuk menghasilkan prediksi akhir pada tiga skala peta fitur, yakni P3, P4, dan P5, yang berkorespondensi dengan *stride* 8, 16, dan 32 dan merepresentasikan deteksi objek kecil, sedang, dan besar secara berturut-turut. Untuk setiap skala, *head* menghasilkan keluaran berupa vektor yang memuat informasi probabilitas kelas objek, skor keyakinan (*confidence score*), serta koordinat *bounding box*. Penggunaan peta fitur multiskala pada komponen ini sangat krusial untuk meningkatkan sensitivitas model terhadap variasi skala objek sekaligus memastikan akurasi deteksi yang presisi (Huang et al., 2023).

Tabel 1. Spesifikasi varian YOLOv5

Model	Parameter (juta)	mAP 50	mAP 50-95	Waktu CPU (ms)	Waktu GPU (ms)
YOLOv5n	1.9	45.7	28.0	45	6.3
YOLOv5s	7.2	56.8	37.4	98	6.4
YOLOv5m	21.2	64.1	45.4	224	8.2
YOLOv5l	46.5	67.3	49.0	430	10.1
YOLOv5x	86.7	68.9	50.7	766	12.1

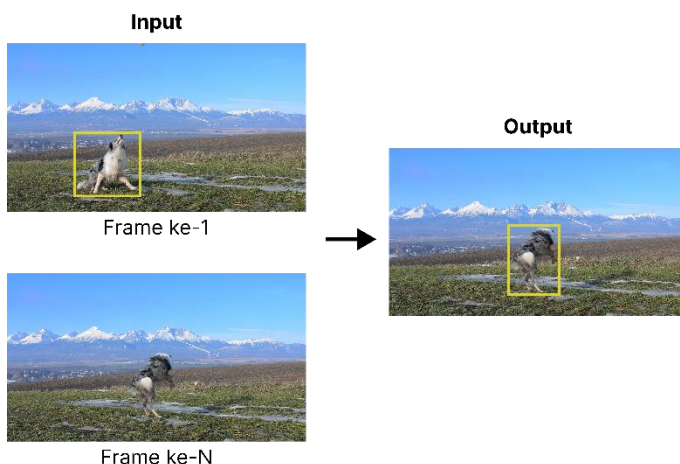
1.2.8 Object Tracking

Object tracking adalah proses estimasi posisi dan lintasan suatu objek di sepanjang *frame* video atau urutan gambar (*image sequence*) (Jiménez-Bravo et al., 2022). *Object tracking* tidak hanya sebatas mengidentifikasi objek di setiap *frame* tetapi juga harus mempertahankan ID unik masing-masing objek meskipun dihadapkan pada

berbagai tantangan seperti oklusi, *motion blur*, variasi pencahayaan dan deformasi objek (Raja et al., 2025).

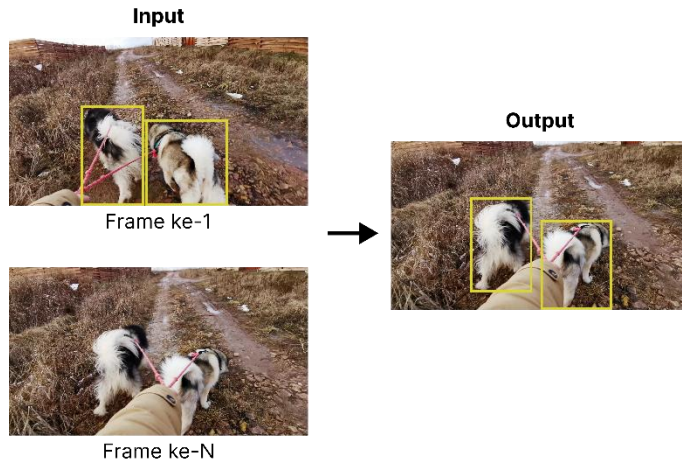
Berdasarkan jumlah objek yang diamati dalam sebuah video, *object tracking* dibagi menjadi dua kategori, yaitu *Single Object Tracking* (SOT) dan *Multi-Object Tracking* (MOT).

Single Object Tracking (SOT). *Single Object Tracking* berfokus pada pelacakan satu objek spesifik secara kontinu di sepanjang urutan video. Pendekatan ini mengutamakan pengembangan *appearance model* dan *motion model* yang andal untuk menangani perubahan skala, rotasi di luar bidang (*out-of-plane rotation*), serta variasi pencahayaan (Luo et al., 2021).



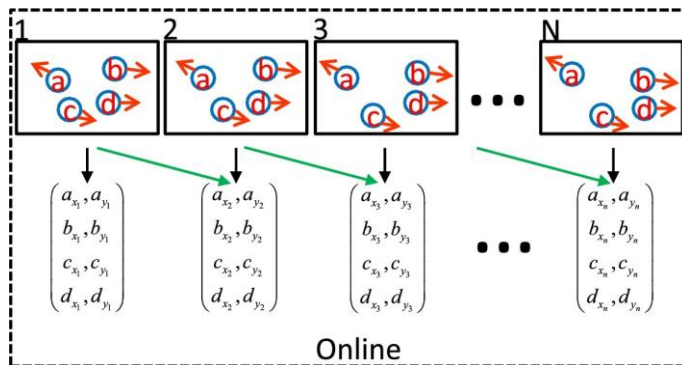
Gambar 7. *Single-object tracking*

Multi-Object Tracking (MOT). *Multi-object tracking* bertujuan melacak seluruh objek dari kelas tertentu yang muncul pada setiap *frame* melalui proses pelokalisasi objek, pemberian ID unik, dan pembentukan lintasan individual. Selain menghadapi tantangan dasar yang juga muncul pada SOT, MOT perlu menangani perubahan jumlah objek secara dinamis melalui proses inisialisasi ketika objek memasuki *frame* dan terminasi ketika objek meninggalkan *frame*, serta menjaga konsistensi ID agar tidak terjadi pertukaran ID (*ID switch*). Kompleksitas ini kerap meningkat akibat oklusi, kemiripan tampilan antara objek, dan interaksi fisik yang rapat antartarget (Luo et al., 2021).

Gambar 8. *Multiple-object tracking*

Paradigma *Tracking-by-Detection* (TBD) merupakan pendekatan yang paling umum digunakan saat ini. Secara umum, TBD memisahkan proses pelacakan menjadi dua subtugas, yaitu deteksi objek dan asosiasi data. Deteksi objek bergantung pada kemampuan model detektor yang digunakan untuk melokalisasi objek serta menentukan label kelas dan skor keyakinan seluruh objek dalam *frame*. Selanjutnya, asosiasi data dilakukan untuk menghubungkan hasil deteksi pada *frame* saat ini dengan lintasan (*tracklet*) yang sudah terbentuk dari *frame-frame* sebelumnya (Zhang et al., 2022).

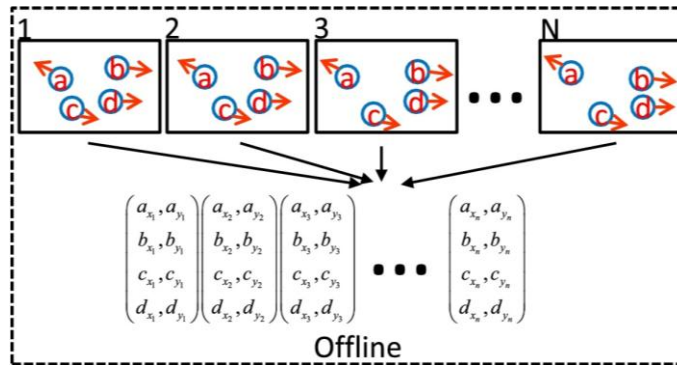
Selain diklasifikasikan berdasarkan jumlah objek yang dilacak, *object tracking* juga dibedakan berdasarkan mode inferensi temporal, yaitu *online tracking* dan *offline tracking*.

Gambar 9. *Online tracking*

Sumber: (Luo et al., 2021)

Online tracking. *Online tracking* melakukan proses pelacakan secara sekuensial dengan memanfaatkan informasi dari *frame-frame* sebelumnya hingga *frame* saat ini untuk memperbarui lintasan objek secara langsung.

Pendekatan ini umumnya diterapkan pada aplikasi pelacakan waktu nyata (*real-time*) karena memiliki latensi yang rendah. Namun, keterbatasan akses terhadap informasi *frame* mendatang kerap menyebabkan penurunan akurasi, terutama pada kondisi oklusi, perubahan skala, atau pergerakan objek yang cepat (Luo et al., 2021).



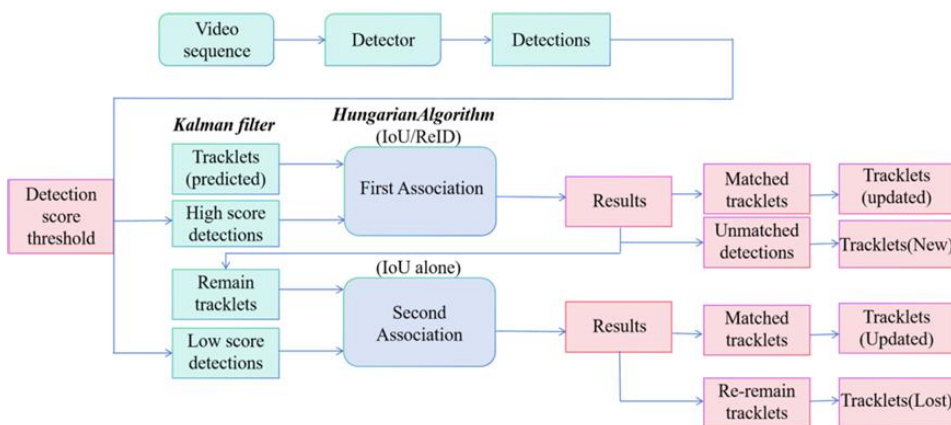
Gambar 10. *Offline tracking*

Sumber: (Luo et al., 2021)

Offline tracking. *Offline tracking* memanfaatkan informasi temporal secara menyeluruh dengan menggunakan *frame* sebelumnya maupun *frame* mendatang untuk memperkirakan lintasan objek. Pendekatan ini memungkinkan optimasi secara global terhadap seluruh urutan video sehingga umumnya menghasilkan performa pelacakan yang lebih stabil dan akurat. Akan tetapi, *offline tracking* memiliki keterbatasan berupa adanya *output delay* serta kebutuhan komputasi dan memori yang relatif besar (Luo et al., 2021).

1.2.9 ByteTrack

ByteTrack merupakan algoritma *Multi-Object Tracking* (MOT) yang mengintegrasikan *bounding box* dengan skor keyakinan rendah (*low-confidence detections*) ke dalam proses pelacakan. Berbeda dari pendekatan konvensional yang hanya mempertahankan deteksi dengan skor tinggi, ByteTrack memanfaatkan *bounding boxes* berskor rendah pada tahap asosiasi sekunder untuk mengevaluasi kemiripannya dengan *tracklets* yang ada, sehingga mampu memulihkan objek yang sesungguhnya (*true object recovery*) sekaligus meningkatkan ketahanan terhadap oklusi dan perubahan fragmentasi lintasan (Zhang et al., 2022).

Gambar 11. *ByteTrack workflow*

Sumber: (Liu et al., 2025)

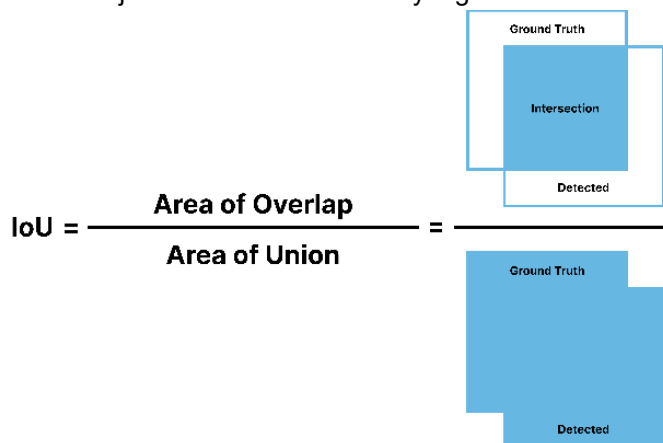
ByteTrack menerapkan skema asosiasi dua tahap melalui metode yang disebut BYTE, dengan membagi hasil deteksi ke dalam kelompok skor tinggi dan skor rendah berdasarkan ambang batas keyakinan. Sebelum setiap tahap asosiasi, *Kalman Filter* terlebih dahulu dijalankan untuk memprediksi posisi seluruh *tracklet* pada *frame* saat ini dengan memperbarui vektor status dan matriks kovarians masing-masing *tracklet* (Zhang et al., 2022). Pada tahap pertama, *bounding box* berskor tinggi diasosiasikan dengan *tracklet* yang telah ada berdasarkan kemiripan gerak (*motion similarity*) menggunakan *Intersection over Union* (IoU) antara posisi prediksi *tracklet* dan *bounding box* deteksi, atau secara opsional menggunakan jarak fitur Re-ID untuk kemiripan tampilan (*appearance similarity*). Penugasan optimal pada tahap ini diselesaikan menggunakan *Hungarian algorithm*. Selanjutnya, pada tahap kedua, *tracklet* yang belum berhasil dipasangkan pada tahap pertama diasosiasikan dengan *bounding box* berskor rendah menggunakan *motion similarity* berbasis IoU. Pendekatan ini memungkinkan pemulihan objek yang mengalami oklusi parsial, yang deteksinya turun di bawah ambang batas skor tinggi, sekaligus menyaring deteksi latar belakang yang bersifat palsu (Zhang et al., 2022).

Arsitektur ByteTrack yang relatif sederhana menjadikannya efisien secara komputasi dan sesuai untuk lingkungan dengan keterbatasan sumber daya perangkat keras. Pemanfaatan deteksi dengan nilai keyakinan rendah terbukti meningkatkan akurasi pelacakan secara keseluruhan tanpa mengorbankan kecepatan secara signifikan, sehingga ByteTrack menawarkan keseimbangan yang baik antara efisiensi komputasi dan performa pelacakan (Bajraktari & Toyman, 2025).

1.2.10 Metrik Evaluasi

Evaluasi model dilakukan untuk menganalisis kinerja sistem secara menyeluruh. Dalam konteks deteksi objek secara *real-time*, metrik evaluasi yang digunakan meliputi *Intersection over Union* (IoU), *Confusion Matrix*, Presisi, Recall, *Average Precision* (AP), serta kecepatan inferensi dalam *Frames Per Second* (FPS).

Intersection over Union (IoU). *Intersection over Union* (IoU) merupakan metrik yang digunakan untuk mengukur tingkat kesesuaian antara *bounding box* hasil prediksi dengan *ground truth* (GT). Nilai IoU dihitung sebagai rasio antara luas irisan (*intersection*) dan luas gabungan (*union*) dari kedua *bounding box*. Nilai IoU berada pada rentang $[0,1]$, di mana nilai yang semakin mendekati 1 menunjukkan akurasi lokalisasi yang semakin baik.



Gambar 12. *Intersection of Union*

Confusion Matrix. *Confusion Matrix* digunakan untuk mengevaluasi hasil prediksi model berdasarkan empat komponen utama, yaitu *True Positive* (TP), *False Positive* (FP), *False Negative* (FN), dan *True Negative* (TN).

- *True Positive* (TP): Prediksi positif dan aktual positif.
- *False Positive* (FP): Prediksi positif dan aktual negatif.
- *False Negative* (FN): Prediksi negatif dan aktual positif.
- *True Negative* (TN): Prediksi negatif dan aktual negatif.

		Predicted Class	
		P	N
Actual Class	P	True Positive	False Negative
	N	False Positive	True Negative

Gambar 13. *Confusion Matrix*

Precision. *Precision* mengukur tingkat ketepatan model dalam menghasilkan prediksi positif yang benar terhadap seluruh prediksi positif yang dihasilkan.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4)$$

Keterangan:

$TP = \text{True Positive}$

$FP = \text{False Positive}$

Recall. *Recall* mengukur kemampuan model dalam menemukan seluruh objek positif yang sesungguhnya ada pada data.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5)$$

Keterangan:

$TP = \text{True Positive}$

$FN = \text{False Negative}$

F1-Score. F1-Score menghitung rata-rata harmonik dari *precision* dan *recall* yang digunakan untuk menilai keseimbangan performa model secara keseluruhan.

$$F1 - \text{Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

Average Precision (AP). *Average Precision* (AP) mengukur performa deteksi objek dengan mempertimbangkan hubungan antara *precision* dan *recall* pada berbagai ambang kepercayaan (*confidence thresholds*).

$$AP = \int_{r=0}^1 p(r) dr \quad (7)$$

Keterangan:

$p(r)$ = nilai *precision* sebagai fungsi dari *recall*

$r = \text{recall}$

Mean Average Precision (mAP). *Mean Average Precision* (mAP) diperoleh dengan menghitung rata-rata nilai AP untuk seluruh kelas objek yang didefinisikan sebagai berikut.

$$mAP = \frac{1}{C} \sum_{C=1}^C AP_C \quad (8)$$

Keterangan:

C = kelas

AP_C = *Average Precision* untuk kelas ke- C

Evaluasi mAP dilakukan pada berbagai ambang IoU, umumnya $mAP@0.5$ dan $mAP@0.5:0.95$. $mAP@0.5$ merepresentasikan kemampuan model dalam mendeteksi objek dengan kriteria bahwa suatu prediksi dinyatakan benar apabila nilai IoU antara *bounding box* prediksi dan *ground truth* lebih besar atau sama dengan 0.5. Sementara itu, $mAP@0.5:0.95$ memberikan evaluasi yang lebih komprehensif dengan mengukur performa model pada rentang ambang IoU dari 0.5 hingga 0.95 dengan interval 0.05, sehingga mencerminkan kemampuan model dalam melakukan deteksi dengan tingkat presisi lokalisasi yang beragam, baik rendah maupun tinggi.

Mean Absolute Error (MAE). *Mean Absolute Error (MAE)* merupakan metrik yang digunakan untuk mengukur rata-rata besar kesalahan prediksi terhadap nilai *ground truth* tanpa memperhatikan arah kesalahan (positif atau negatif). MAE menghitung nilai absolut dari selisih antara prediksi dan *ground truth*, sehingga menunjukkan seberapa besar rata-rata deviasi prediksi dari nilai sebenarnya. Nilai MAE yang semakin kecil menandakan bahwa hasil prediksi semakin mendekati *ground truth*.

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \quad (9)$$

Keterangan:

n = jumlah sampel evaluasi

$\hat{y}_i$ = nilai prediksi pada sampel ke- i

y_i = nilai aktual pada sampel ke- i

Root Mean Square Error (RMSE). *Root Mean Square Error (RMSE)* merupakan metrik yang digunakan untuk mengukur akar dari rata-rata kuadrat kesalahan prediksi terhadap *ground truth*. RMSE mengkuadratkan selisih antara prediksi dan *ground truth* sehingga kesalahan yang besar mendapatkan penalti yang lebih tinggi dibandingkan kesalahan kecil. Nilai RMSE yang semakin kecil menunjukkan prediksi semakin akurat dan dekat dengan *ground truth*.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2} \quad (10)$$

Keterangan:

n = jumlah sampel evaluasi

$\hat{y}_i$ = nilai prediksi pada sampel ke- i

y_i = nilai aktual pada sampel ke- i

Frame Per Second (FPS). *Frame Per Second (FPS)* mengukur seberapa banyak *frame* yang dapat diproses oleh model dalam satu detik, sehingga semakin banyak *frame* yang diproses per detik maka semakin rendah latensi inferensi pada model.

$$FPS = \frac{N_f}{T} \quad (11)$$

Keterangan:

N_f = jumlah *frame* yang diproses

T = total waktu inferensi (detik)

1.3 Rumusan Masalah

Permasalahan utama yang akan diangkat dalam penelitian ini sebagai berikut.

1. Bagaimana cara mengembangkan sistem deteksi dan hitung objek kentang multi-skala secara *real-time*?
2. Bagaimana kinerja sistem deteksi dan hitung objek kentang multi-skala secara *real-time*?

1.4 Tujuan dan Manfaat

1.4.1 Tujuan

Adapun tujuan dari penelitian ini adalah sebagai berikut:

1. Mengembangkan sistem deteksi dan hitung objek kentang multi-skala secara *real-time*
2. Menganalisis kinerja sistem deteksi dan hitung objek kentang multi-skala secara *real-time*.

1.4.2 Manfaat

Manfaat dari penelitian ini adalah sebagai berikut.

1. **Industri Pertanian:** Mendukung *smart farming* dengan sistem otomatisasi, yang dapat meningkatkan produktivitas dan mengurangi ketergantungan pada manusia.
2. **Akademis:** Memberikan pengetahuan terkait penerapan teknologi khususnya di bidang visi komputer untuk deteksi dan perhitungan objek di sektor pertanian secara *real-time*.

1.5 Batasan Masalah

Ruang lingkup dari penelitian ini adalah sebagai berikut.

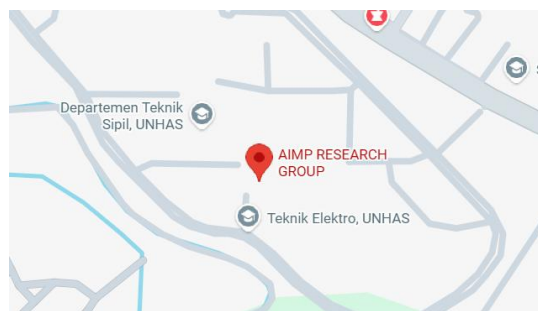
1. Objek penelitian berupa kentang.
2. Berfokus untuk mendeteksi dan menghitung kentang dengan ukuran multi-skala di perkebunan.
3. Pengambilan data dilakukan dalam kondisi cerah.
4. Data diperoleh menggunakan kamera dengan bantuan alat/kerangka yang meniru robot.

BAB II

METODE PENELITIAN

2.1 Tempat dan Waktu Penelitian

Penelitian ini dilaksanakan selama delapan bulan, terhitung sejak bulan Mei 2025 hingga Januari 2026. Lokasi penelitian dilakukan di Laboratorium *Artificial Intelligence* (AI), Departemen Teknik Informatika, Fakultas Teknik, Universitas Hasanuddin. Sementara itu, pengumpulan dataset primer dilakukan dengan mereplikasi kondisi lingkungan riil menggunakan sampel tanah yang diperoleh dari perkebunan kentang yang berlokasi di Desa Erelembang, Kecamatan Tombolo Pao, Kabupaten Gowa, Sulawesi Selatan.



(a)



(b)

Gambar 14. Lokasi Penelitian: (a) Laboratorium *Artificial Intelligence* (AI); (b) Kebun kentang Gowa

2.2 Instrumen Penelitian

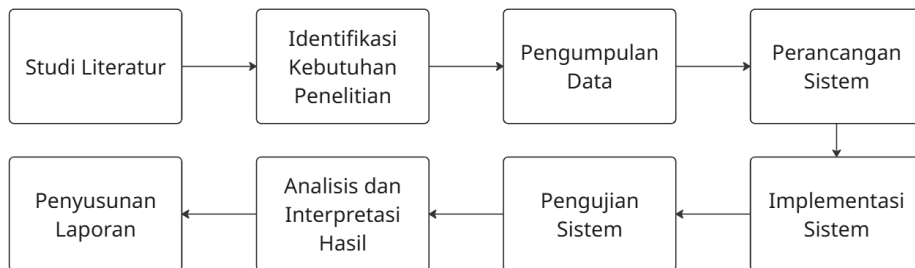
Instrumen-instrumen penelitian yang digunakan dalam penelitian ini, meliputi:

1. Perangkat lunak:
 - a. GoPro Quik
 - b. Roboflow
 - c. Google Colaboratory

- d. Kaggle
 - e. OBS Studio
 - f. Visual Studio Code
 - g. Microsoft Word
 - h. Microsoft PowerPoint
 - i. Google Spreadsheet
 - j. Mendeley Reference Manager
 - k. Sistem Operasi Windows 11 64-bit
2. Perangkat Keras:
 - a. GoPro HERO11 Black
 - b. Lenovo IdeaPad Slim 3 14ITL6, RAM 8 GB, SSD 512 GB, GPU NVIDIA GeForce MX450 2 GB GDDR5
 3. Bahasa Pemrograman
 - a. Python v3.11.13
 4. Algoritma:
 - a. YOLOv5
 - b. ByteTrack
 - c. *Unsharp Masking*
 5. Library
 - a. Roboflow v1.1.66, digunakan sebagai SDK untuk akuisisi, *versioning*, dan pengunduhan dataset berformat YOLO serta manajemen anotasi melalui API.
 - b. PyTorch (Torch) v2.7.1+cu118, digunakan sebagai *backend* komputasi tensor dan inferensi model berbasis GPU (CUDA 11.8).
 - c. Torchvision v0.22.1+cu118, digunakan sebagai pustaka pendukung visi komputer yang menyediakan transformasi data, utilitas dataset, dan fungsi visi yang terintegrasi dengan PyTorch.
 - d. OpenCV (opencv-python) v4.10.0.84, digunakan sebagai pustaka pemrosesan citra/video untuk operasi pra-pemrosesan, transformasi, *filtering*, serta I/O *frame* dan gambar.
 - e. Numpy v2.1.2, digunakan sebagai pustaka komputasi numerik berbasis array untuk operasi matematika vektor/matriks dan manipulasi data berdimensi tinggi.
 - f. Scipy v1.16.0, digunakan untuk proses *matching* pada ByteTrack.
 - g. Pandas v2.2.3, digunakan sebagai pustaka analisis data tabular untuk pengelolaan hasil eksperimen, agregasi metrik, dan ekspor data.
 - h. Matplotlib v3.7.2, digunakan sebagai pustaka visualisasi dasar untuk pembuatan grafik evaluasi, kurva metrik, dan plot analitis.
 - i. Seaborn v0.12.2, digunakan sebagai pustaka visualisasi statistik tingkat lanjut berbasis Matplotlib untuk penyajian distribusi, perbandingan kategori, dan ringkasan pola data.

2.3 Tahapan Penelitian

Tahapan penelitian pada Gambar 15 disusun sebagai alur kerja terstruktur dari tahap studi literatur hingga pelaporan. Uraian teknis setiap tahapan adalah sebagai berikut.



Gambar 15. Tahapan Penelitian

1. Studi literatur

Tahap ini mengkaji penelitian terdahulu terkait deteksi objek pada lingkungan perkebunan dan penghitungan objek, dengan fokus pada pendekatan *real-time* yang mampu menghadapi tantangan visual, khususnya kemiripan warna antara objek target terhadap latar belakang.

2. Identifikasi kebutuhan penelitian

Tahap ini merumuskan kebutuhan penelitian secara sistematis untuk menetapkan permasalahan, tujuan, spesifikasi data, serta metode yang akan digunakan, sehingga ruang lingkup penelitian terdefinisi dengan jelas.

3. Pengumpulan data

Tahap ini melakukan pengumpulan data dengan mereplikasi kondisi operasional proses pemanenan kentang menggunakan media tanah yang bersumber dari kebun kentang Gowa, melalui perekaman video pada variasi sudut kemiringan, ketinggian, dan kecepatan pergerakan kamera guna mensimulasikan kinematika robot pemanen.

4. Perancangan sistem

Tahap ini merancang *pipeline* sistem secara menyeluruh, meliputi skema *preprocessing*, perancangan skenario pelatihan dan konfigurasi *hyperparameter*, mekanisme *post-processing*, penetapan *Region of Interest* (ROI), rancangan mekanisme *counting line*, serta pengembangan antarmuka pengguna (GUI) untuk visualisasi proses dan keluaran sistem.

5. Implementasi sistem

Tahap ini mengimplementasikan rancangan sistem, mencakup integrasi modul deteksi objek, penghitungan, dan visualisasi sesuai spesifikasi yang telah ditentukan.

6. Pengujian sistem

Tahap ini mengevaluasi kinerja sistem secara kuantitatif pada aspek deteksi, penghitungan (*counting*), dan performa *real-time* menggunakan metrik *Precision*, *Recall*, *F1-Score*, *mAP50*, *mAP50–95*, *Mean Absolute*

Error (MAE), *Root Mean Square Error* (RMSE), dan *Frame Per Second* (FPS).

7. Analisis dan interpretasi hasil

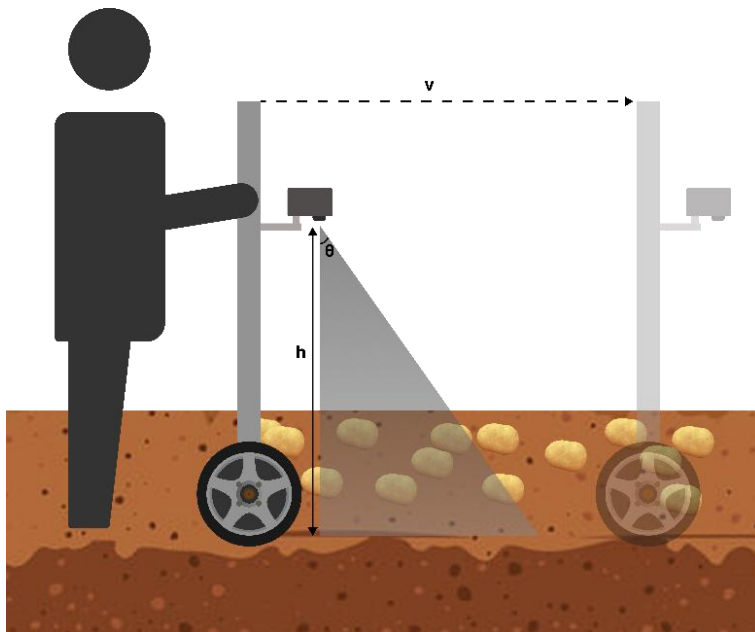
Tahap ini menginterpretasikan hasil evaluasi secara komprehensif melalui analisis kuantitatif dan kualitatif, serta melaksanakan *failure case analysis* untuk mengidentifikasi pola kesalahan dan kondisi yang menyebabkan penurunan performa sistem.

8. Penyusunan laporan

Tahap ini mendokumentasikan seluruh proses dan hasil penelitian secara sistematis dan terstruktur ke dalam bentuk laporan akhir.

2.4 Teknik Pengambilan Data

Data primer diperoleh melalui perekaman lapangan menggunakan kamera *outdoor* GoPro HERO11 Black. Perangkat dikonfigurasi pada resolusi 1920×1080 piksel dan *frame rate* 30 FPS. Fitur *Hypersmooth* dan mode lensa *Linear + Horizon Lock* diterapkan untuk menjaga stabilitas citra serta meminimalkan distorsi. Karakteristik optik sensor mencakup *Field of View* (FOV) horizontal 87° , vertikal 56° dan diagonal 95° untuk memaksimalkan cakupan area observasi.



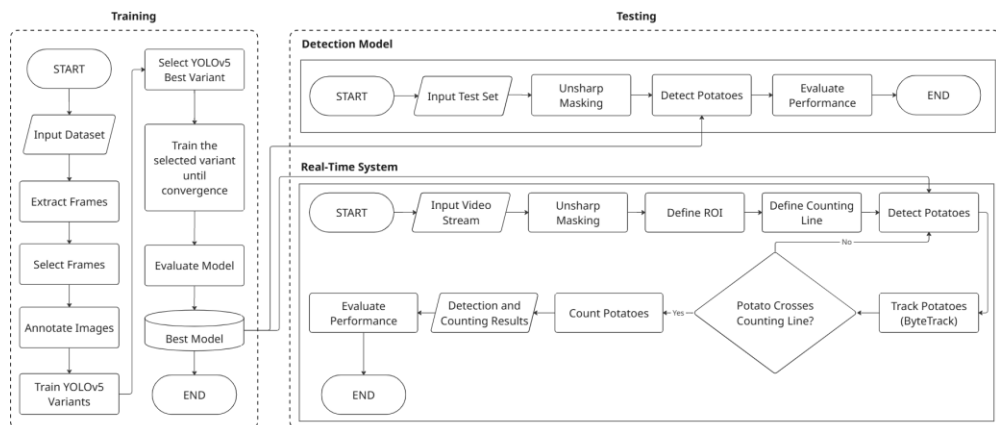
Gambar 16. Skenario pengambilan data

Skenario pengambilan data dirancang untuk mensimulasikan kinematika robot pemanen melalui variasi tiga parameter operasional yang dirincikan pada Tabel 2.

Tabel 2. Parameter skenario pengambilan data

Parameter	Nilai / Rentang	Keterangan
Sudut Kamera (θ)	80°, 90°	Sudut kemiringan kamera terhadap tanah.
Ketinggian (h)	80 – 140 cm	Ketinggian kamera terhadap tanah dengan interval kenaikan 10 cm.
Kecepatan (v)	0.25, 0.5, 0.75, 1 m/s	Kecepatan rata-rata alat simulasi pergerakan robot.

2.5 Perancangan dan Implementasi Sistem

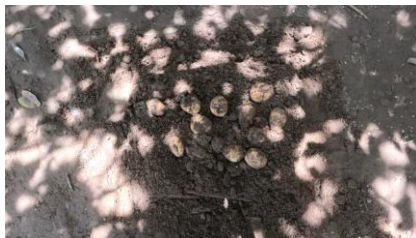


Gambar 17. Diagram Alir Sistem

2.5.1 Tahap Training

1. Input Dataset

Tahap ini diawali dengan menyiapkan data masukan berupa rekaman video mentah (*raw video*) sebagai data primer hasil akuisisi menggunakan kamera GoPro HERO11 Black dengan teknik pengambilan data sebagaimana terlihat pada Gambar 16. Total video yang dihasilkan sebanyak 126 video.

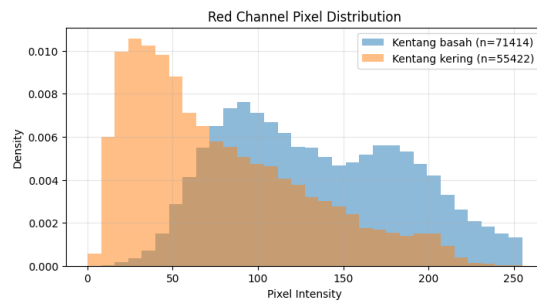


(a)

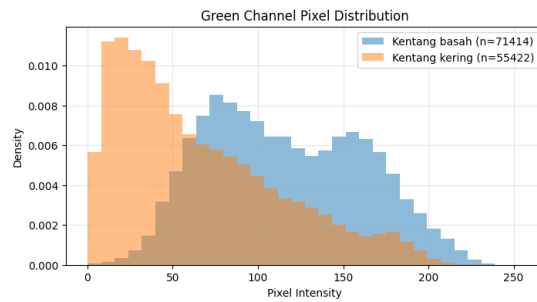


(b)

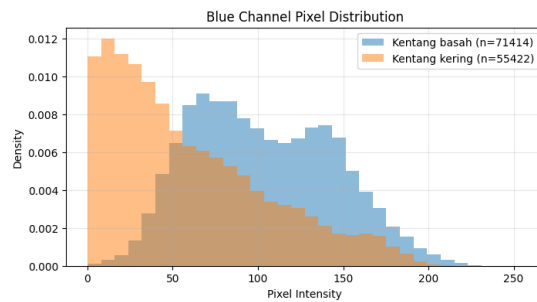
Gambar 18. Tampilan visual kentang: (a) Sesaat setelah dikeluarkan dari tanah; (b) Setelah dikeringkan selama beberapa jam.



(a)



(b)



(c)

Gambar 19. Distribusi piksel untuk citra kentang: (a) *Red channel*; (b) *Green channel*; (c) *Blue channel*.

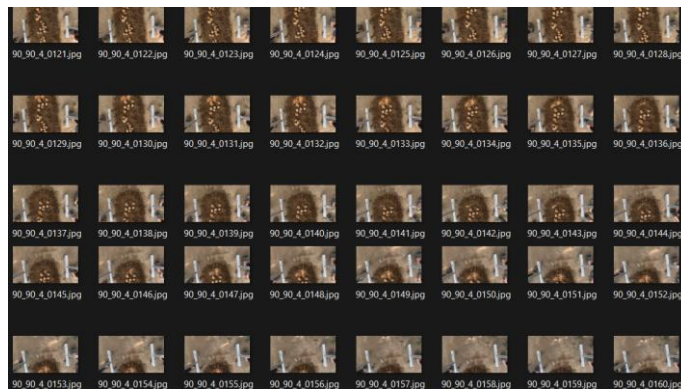
Berdasarkan grafik distribusi piksel pada kanal *Red*, *Green*, dan *Blue*, terlihat adanya perbedaan karakteristik warna antara kentang yang baru dikeluarkan dari tanah (basah) dan kentang yang telah dikeringkan selama beberapa jam. Distribusi piksel pada kentang dengan tanah basah cenderung berada pada rentang intensitas menengah hingga tinggi, yang menunjukkan bahwa permukaan kentang yang masih lembap serta bagian kulit yang lebih terlihat memantulkan cahaya lebih kuat sehingga menghasilkan nilai piksel yang relatif lebih terang. Sebaliknya, distribusi piksel pada kentang kering lebih banyak pada rentang intensitas rendah hingga menengah, yang menunjukkan bahwa lapisan tanah kering yang menutupi permukaan kentang menyebabkan reflektansi cahaya yang lebih rendah sehingga nilai piksel yang dihasilkan cenderung lebih gelap. Pola ini konsisten pada ketiga kanal RGB, sehingga menunjukkan bahwa kondisi kelembapan permukaan kentang memengaruhi karakteristik distribusi intensitas piksel pada citra.



Gambar 20. Kumpulan video dataset

2. Ekstraksi Frame (*Extract Frames*)

Rekaman video diekstraksi menjadi citra statis dengan laju 10 FPS menggunakan OpenCV guna mengonversi data video menjadi kumpulan citra yang dapat dianotasi. Laju ekstraksi 10 FPS dipilih untuk menangkap variasi visual objek antarwaktu secara optimal tanpa menghasilkan redundansi data yang berlebihan akibat pergerakan kamera yang kontinu.



Gambar 21. Hasil ekstraksi video

3. Seleksi Frame (*Select Frames*)

Frame hasil ekstraksi diseleksi secara manual untuk memastikan dataset bersifat representatif dan mencakup keberagaman kondisi yang diperlukan bagi generalisasi model, meliputi variasi ukuran objek, posisi objek, kepadatan kentang, perubahan latar belakang, serta kondisi pencahayaan. Selain citra yang memuat objek kentang, penelitian ini juga menyertakan citra latar belakang tanpa objek kentang sebagai *negative samples* guna meningkatkan kemampuan model dalam membedakan objek kentang dari latar sekitarnya sekaligus menekan peluang terjadinya kesalahan deteksi (*false positive*). Dataset akhir yang digunakan terdiri dari 2.413 citra dengan total 38.049 *instance* objek kentang, yang terbagi atas 2.335 citra kentang dan 78 citra latar belakang.



(a)

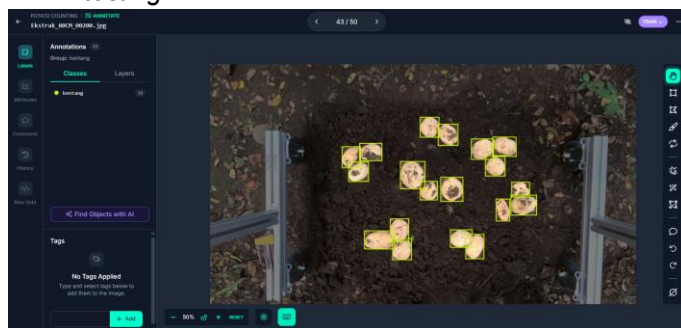


(b)

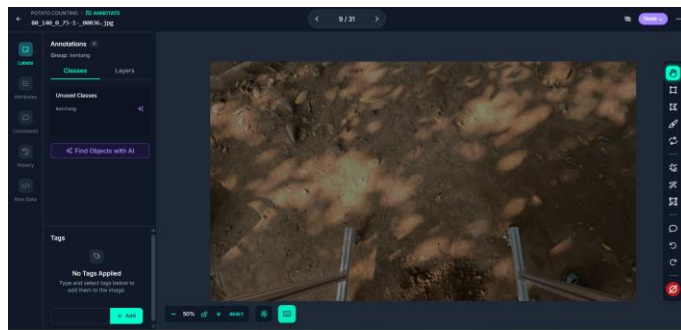
Gambar 22. Sampel dataset: (a) Citra dengan objek kentang; (b) Citra latar belakang.

4. Anotasi Gambar (*Image Annotation*)

Setiap citra dalam dataset dianotasi menggunakan platform Roboflow dengan pemberian *bounding box* untuk melokalisasi objek kentang, guna menghasilkan *ground truth* yang menjadi acuan model dalam mengenali objek selama proses pelatihan. Untuk citra latar belakang, anotasi dibiarkan kosong (*null label*). Dataset yang telah dianotasi dibagi menjadi tiga subset, yaitu data *train*, *valid*, dan *test*. Data *train* digunakan untuk melatih bobot parameter model, data *valid* digunakan untuk memantau performa model selama proses *training*, sedangkan data *test* digunakan untuk evaluasi akhir model secara objektif menggunakan data yang sepenuhnya belum pernah dipelajari oleh model (*unseen data*). Komposisi pembagian dataset pada penelitian ini adalah 2.077 citra untuk *training*, 168 citra untuk *validation*, dan 168 citra untuk *testing*.



(a)



(b)

Gambar 23. Anotasi dataset menggunakan Roboflow: (a) Citra dengan kelas kentang; (b) Citra tanpa kelas kentang.

5. Training YOLOv5 Variants

Proses *training* dilakukan dalam dua tahap berurutan guna memperoleh model deteksi objek dengan kinerja optimal pada skenario operasional *real-time*. YOLOv5 dipilih karena kemampuannya menyeimbangkan akurasi deteksi dan efisiensi komputasi yang menjadikannya relevan untuk aplikasi *real-time* (Terven et al., 2023). Arsitektur YOLOv5 tersedia dalam beberapa varian seperti *nano* (YOLOv5n), *small* (YOLOv5s), *medium* (YOLOv5m), *large* (YOLOv5l), dan *extra-large* (YOLOv5x) yang memiliki arsitektur yang sama. Perbedaan antarvarian tidak terletak pada urutan atau jenis *layer*, melainkan pada dua parameter skala yaitu *depth_multiple* yang mengontrol jumlah repetisi modul *BottleNeck* dalam blok C3, dan *width_multiple* yang mengontrol lebar kanal fitur di setiap *stage*.



Gambar 24. Sampel Dataset

Sebelum masuk ke dalam proses ekstraksi fitur oleh model, representasi citra masukan yang awalnya memiliki rentang nilai piksel 0 hingga 255 perlu dinormalisasi. Intensitas piksel dikompresi ke dalam representasi *floating-point* antara 0 dan 1 menggunakan persamaan matematika dasar:

$$x_{norm} = \frac{x'}{255} \quad (12)$$

Sebagai contoh, diambil sebagian kecil area piksel (5×5) dari citra inputan dengan nilai sebagai berikut.

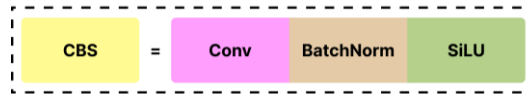
$$x' = \begin{bmatrix} 173 & 171 & 160 & 109 & 73 \\ 177 & 159 & 119 & 79 & 67 \\ 182 & 138 & 76 & 55 & 63 \\ 172 & 109 & 53 & 50 & 66 \\ 137 & 73 & 46 & 54 & 65 \end{bmatrix}$$

Nilai intensitas piksel ini kemudian dinormalisasi menjadi rentang 0 hingga 1 sehingga menghasilkan matriks citra masukan yang dinormalisasi (I):

$$I = \begin{bmatrix} 0.6784 & 0.6706 & 0.6275 & 0.4275 & 0.2863 \\ 0.6941 & 0.6235 & 0.4667 & 0.3098 & 0.2627 \\ 0.7137 & 0.5412 & 0.2980 & 0.2157 & 0.2471 \\ 0.6745 & 0.4275 & 0.2078 & 0.1961 & 0.2588 \\ 0.5373 & 0.2863 & 0.1804 & 0.2118 & 0.2549 \end{bmatrix}$$

a. Blok CBS

Blok CBS merupakan blok dasar pada YOLOv5 yang tersusun atas tiga operasi berurutan, yaitu *Convolution*, *Batch Normalization*, dan aktivasi SiLU. Operasi konvolusi berfungsi mengekstraksi pola spasial dari citra atau *feature map* masukan melalui proses perkalian kernel dengan area lokal masukan.



Gambar 25. Blok CBS

Secara umum, untuk kernel berukuran $K \times K$ keluaran konvolusi pada posisi (i, j) dapat dituliskan sebagai:

$$x_{i,j} = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} I_{(i+m),(j+n)} \cdot W_{m,n} \quad (13)$$

dengan I menyatakan *feature map* masukan dan W menyatakan bobot kernel. Keluaran ini kemudian dinormalisasi menggunakan *Batch Normalization* (BN) untuk menstabilkan distribusi aktivasi per kanal selama *training*, yang dirumuskan sebagai:

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (14)$$

Tahap terakhir adalah fungsi aktivasi *Sigmoid Linear Unit* (SiLU) yang memberikan transisi lebih halus dan aliran gradien yang lebih baik, didefinisikan sebagai:

$$SiLU(y) = y \cdot \sigma(y) \quad (15)$$

dengan

$$\sigma(y) = \frac{1}{1 + e^{-y}} \quad (16)$$

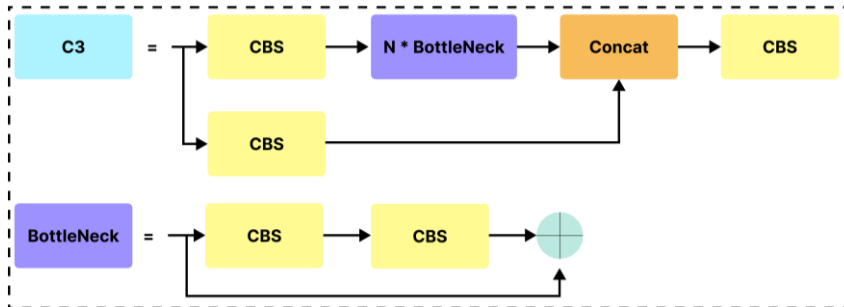
Secara dimensional, keluaran konvolusi dua dimensi dari masukan berukuran $H_{in} \times W_{in} \times C_{in}$ dapat dihitung menjadi:

$$H_{out} = \lfloor \frac{H_{in} - K + 2P}{S} \rfloor + 1, W_{out} = \lfloor \frac{W_{in} - K + 2P}{S} \rfloor + 1 \quad (17)$$

dengan P adalah jumlah *padding* dan S adalah ukuran *stride*.

b. Blok C3

Feature map yang telah melalui proses ekstraksi awal pada Blok CBS kemudian diteruskan ke Blok C3 untuk memperdalam representasi fitur. Blok ini memecah aliran komputasi menjadi dua jalur di mana satu jalur melewati sejumlah blok *BottleNeck*, dan jalur lainnya berperan sebagai *shortcut residual*.



Gambar 26. Blok C3

Blok *BottleNeck* mengadopsi prinsip *residual learning* untuk menambahkan informasi masukan awal ke hasil transformasi fitur:

$$y = F(x, \{W_i\}) + x \quad (18)$$

dengan x sebagai masukan, $F(x, \{W_i\})$ sebagai transformasi non-linear yang dipelajari, dan y sebagai keluaran akhir blok.

Sebagai ilustrasi, jika *feature map* masukan x berdimensi 3×3 diproses menjadi matriks fitur baru $F(x)$, maka keluaran akhir diperoleh dengan penjumlahan *element-wise*:

$$x = \begin{bmatrix} 0.46 & 0.41 & 0.16 \\ 0.56 & 0.14 & -0.26 \\ 0.70 & -0.15 & -0.20 \end{bmatrix}$$

$$F(x) = \begin{bmatrix} 0.10 & 0.20 & 0.05 \\ 0.15 & -0.10 & 0.10 \\ 0.20 & 0.15 & 0.05 \end{bmatrix}$$

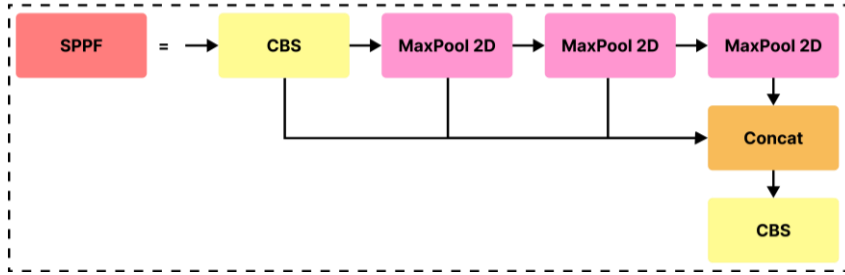
$$y = x + F(x) = \begin{bmatrix} 0.56 & 0.61 & 0.21 \\ 0.71 & 0.04 & -0.16 \\ 0.90 & 0.00 & -0.15 \end{bmatrix}$$

Pada akhir Blok C3, kedua jalur (utama dan *shortcut*) tidak dijumlahkan, melainkan disatukan kembali menggunakan operasi *concatenation* (penumpukan pada sumbu kedalaman/kanal) untuk mempertahankan variansi gradien yang kaya:

$$Y_{concat} = [X_{main}, X_{shortcut}] \quad (19)$$

c. Blok SPPF

Setelah fitur diekstraksi secara hierarkis melalui serangkaian lapisan pada *backbone*, arsitektur YOLOv5 mengimplementasikan modul *Spatial Pyramid Pooling Fast* (SPPF). Modul ini berfungsi untuk memperluas *receptive field* (area pandang jaringan terhadap citra) sehingga model dapat mengagregasi konteks fitur spasial pada berbagai skala tanpa mengubah dimensi resolusi fitur secara drastis. SPPF merupakan bentuk optimasi komputasional dari *Spatial Pyramid Pooling* (SPP) konvensional. Pada SPP tradisional, *feature map* diproses melalui operasi *pooling* paralel dengan berbagai ukuran kernel (misalnya dengan ukuran 5×5 , 9×9 , dan 13×13). Sebaliknya, SPPF menggantikan proses tersebut dengan operasi *pooling* berukuran tunggal 5×5 yang dieksekusi secara sekuensial (berurutan).



Gambar 27. Blok SPPF

Secara matematis, proses ini dirumuskan sebagai berikut.

$$F_0 = CBS_1(x) \quad (20)$$

$$F_1 = MaxPool(F_0) \quad (21)$$

$$F_2 = MaxPool(F_1) \quad (22)$$

$$F_3 = MaxPool(F_2) \quad (23)$$

$$Y_{concat} = [F_0, F_1, F_2, F_3] \quad (24)$$

$$Y_{concat} = CBS_2(Y_{concat}) \quad (25)$$

Operasi inti *MaxPool/2D* mengekstraksi nilai intensitas paling dominan dari area spasial yang dipindai, dirumuskan sebagai:

$$y_{i,j} = \max_{m=0}^{k-1} (\max_{m=0}^{k-1} (x_{(i+m),(j+n)})) \quad (26)$$

dengan s bernilai 1 dan penerapan *padding* agar dimensi *feature map* tidak menyusut.

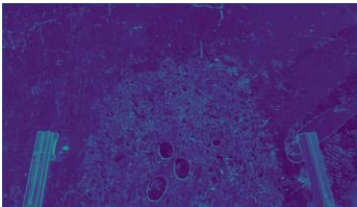
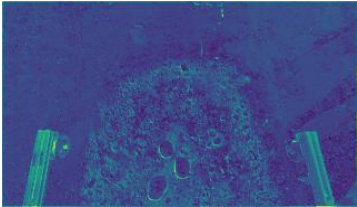
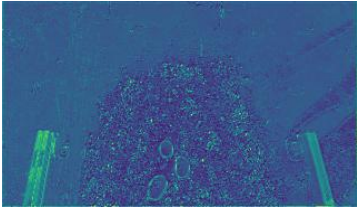
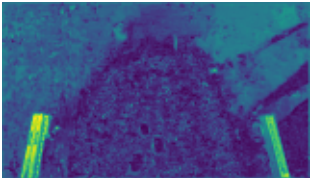
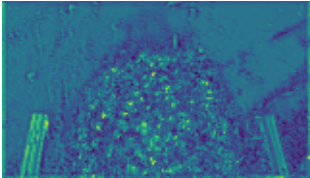
d. Upsampling

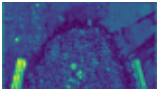
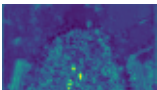
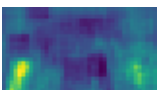
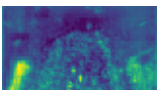
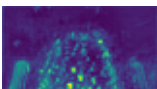
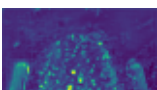
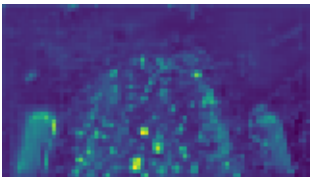
Fitur-fitur beresolusi rendah namun kaya akan informasi semantik dari tahap-tahap sebelumnya perlu digabungkan dengan informasi spasial tingkat rendah guna mendukung deteksi objek *multi-scale*. YOLOv5 memanfaatkan algoritma interpolasi *nearest-neighbor* untuk proses *upsampling*. Metode ini mereplikasi nilai piksel asli ke area terdekatnya. Sebagai ilustrasi, sebuah matriks berdimensi 2×2 yang diperbesar dengan faktor pembesaran dua kali lipat ($scale_factor = 2$):

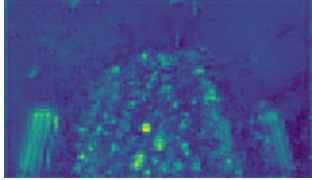
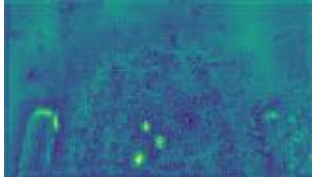
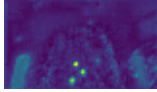
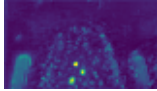
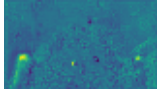
$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{Upsample} \rightarrow Y = \begin{bmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 3 & 3 & 4 & 4 \\ 3 & 3 & 4 & 4 \end{bmatrix}$$

Pendekatan ini memulihkan resolusi secara efisien tanpa mendistorsi struktur numerik bawaan dari fitur aslinya.

Tabel 3. Alur Data pada Arsitektur YOLOv5

Stage	Layer	Input Size	Output Size	Feature Map
0	Conv 6 × 6, s=2	H × W × 3	H/2 × W/2 × C ₁	
1	Conv 3 × 3, s=2	H/2 × W/2 × C ₁	H/4 × W/4 × C ₂	
2	C3 (× n BottleNec k)	H/4 × W/4 × C ₂	H/4 × W/4 × C ₂	
3	Conv 3 × 3, s=2	H/4 × W/4 × C ₂	H/8 × W/8 × C ₃	
4	C3 (× 2n BottleNec k)	H/8 × W/8 × C ₃	H/8 × W/8 × C ₃	

<i>Stage</i>	<i>Layer</i>	<i>Input Size</i>	<i>Output Size</i>	<i>Feature Map</i>
5	Conv 3 × 3, s=2	H/8 × W/8 × C ₃	H/16 × W/16 × C ₄	
6	C3 (× 3n BottleNec k)	H/16 × W/16 × C ₄	H/16 × W/16 × C ₄	
7	Conv 3 × 3, s=2	H/16 × W/16 × C ₄	H/32 × W/32 × C ₅	
8	C3 (× n BottleNec k)	H/32 × W/32 × C ₅	H/32 × W/32 × C ₅	
9	SPPF (MaxPool 5 × 5 × 3)	H/32 × W/32 × C ₅	H/32 × W/32 × C ₅	
10	Conv 1 × 1, s=1	H/32 × W/32 × C ₅	H/32 × W/32 × C ₄	
11	Upsample 2×	H/32 × W/32 × C ₄	H/16 × W/16 × C ₄	
12	Concat (stage 11 + stage 6)	H/16 × W/16 × C ₄ + H/16 × W/16 × C ₄	H/16 × W/16 × 2C ₄	
13	C3 (×n)	H/16 × W/16 × 2C ₄	H/16 × W/16 × C ₄	
14	Conv 1 × 1, s=1	H/16 × W/16 × C ₄	H/16 × W/16 × C ₃	
15	Upsample 2×	H/16 × W/16 × C ₃	H/8 × W/8 × C ₃	

<i>Stage</i>	<i>Layer</i>	<i>Input Size</i>	<i>Output Size</i>	<i>Feature Map</i>
16	Concat (stage 15 + stage 4)	$H/8 \times W/8 \times C_3 + H/8 \times W/8 \times C_3$	$H/8 \times W/8 \times 2C_3$	
17	C3 (xn)	$H/8 \times W/8 \times 2C_3$	$H/8 \times W/8 \times C_3$	
18	Conv 3 x 3, s=2	$H/8 \times W/8 \times C_3$	$H/16 \times W/16 \times C_3$	
19	Concat (stage 18 + stage 13)	$H/16 \times W/16 \times C_3 + H/16 \times W/16 \times C_4$	$H/16 \times W/16 \times (C_3 + C_4)$	
20	C3 (xn)	$H/16 \times W/16 \times (C_3 + C_4)$	$H/16 \times W/16 \times C_4$	
21	Conv 3 x 3, s=2	$H/16 \times W/16 \times C_4$	$H/32 \times W/32 \times C_4$	
22	Concat (stage 21 + stage 9)	$H/32 \times W/32 \times C_4 + H/32 \times W/32 \times C_5$	$H/32 \times W/32 \times (C_4 + C_5)$	
23	C3 (xn)	$H/32 \times W/32 \times (C_4 + C_5)$	$H/32 \times W/32 \times C_5$	

Notasi simbolik $C_1 - C_5$ pada Tabel 3 merepresentasikan lebar kanal fitur yang bervariasi antarvarian sesuai dengan *width_multiple*, dengan nilai *channel* per varian ditunjukkan pada Tabel 4.

Tabel 4. Referensi Nilai *Channel* per Varian YOLOv5

Simbol	YOLOv5n	YOLOv5s	YOLOv5m	YOLOv5l	YOLOv5x
C_1	16	32	48	64	80
C_2	32	64	96	128	160
C_3	64	128	192	256	320
C_4	128	256	384	512	640
C_5	256	512	768	1024	1280

e. Seleksi Varian

Tahap pertama bertujuan memilih varian YOLOv5 terbaik. Kelima varian dilatih menggunakan konfigurasi *hyperparameter* yang sama (Tabel 5) agar perbandingan antarvarian bersifat adil, sehingga perbedaan hasil lebih dapat dikaitkan dengan perbedaan kapasitas arsitektur. Setiap varian menggunakan *pre-trained weights* dari dataset COCO, lalu dilakukan *fine-tuning* pada dataset kentang yang telah dianotasi. Setelah *training*, seluruh varian dievaluasi menggunakan data uji dan metrik yang telah ditentukan, kemudian varian dengan performa terbaik dipilih sebagai model yang dilanjutkan ke tahap berikutnya.

Tabel 5. Konfigurasi *hyperparameter training* untuk seleksi varian

<i>Hyperparameter</i>	Nilai
<i>Epochs</i>	100
<i>Image Size</i>	640 × 640
<i>Batch Size</i>	16
<i>Optimizer</i>	SGD
<i>Learning Rate</i>	0.01

f. Konvergensi Model

Model terpilih dilatih ulang dengan konfigurasi pada Tabel 6 untuk meningkatkan performa. Jumlah *epoch* maksimum dinaikkan menjadi 1.000, dan digunakan *early stopping* dengan parameter *patience* untuk menghentikan *training* secara otomatis ketika metrik validasi tidak membaik dalam beberapa *epoch* berturut-turut. Pendekatan ini membantu menghentikan *training* pada kondisi yang sudah konvergen,

sehingga mengurangi *training* yang tidak perlu dan menekan risiko *overfitting*.

Tabel 6. Konfigurasi *hyperparameter training* untuk konvergensi model

<i>Hyperparameter</i>	<i>Nilai</i>
<i>Epochs</i>	1000
<i>Image Size</i>	640 × 640
<i>Batch Size</i>	16
<i>Optimizer</i>	SGD
<i>Learning Rate</i>	0.01
<i>Patience</i>	100

2.5.2 Tahap *Testing*

1. Evaluasi Model Deteksi

Evaluasi model deteksi dilakukan menggunakan inputan berupa data *test* yang terpisah dan tidak digunakan pada proses *training* maupun validasi (*unseen data*), sehingga hasil yang diperoleh merepresentasikan kemampuan generalisasi model secara objektif. Pada penelitian ini, evaluasi model dilakukan dalam tiga skenario perbandingan dengan prosedur dan metrik yang sama, yaitu *precision*, *recall*, *F1-score* dan *mAP*. Ketiga skenario tersebut meliputi perbandingan performa antarvarian YOLOv5, perbandingan performa varian terbaik sebelum dan sesudah konvergensi model, serta perbandingan performa model terbaik pada citra asli dengan citra yang telah melalui tahap *preprocessing* berupa *unsharp masking*.

2. Evaluasi Sistem *Real-Time*

a. Input Video *Stream*

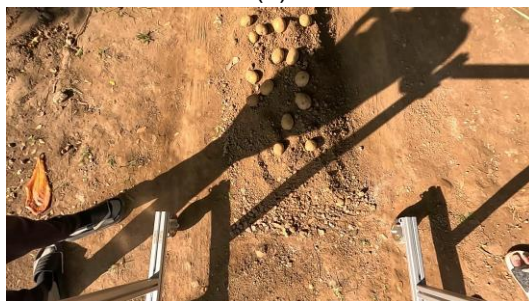
Tahap *testing* dimulai dengan akuisisi video *stream* menggunakan fungsi VideoCapture pada OpenCV dari perangkat kamera yang terhubung ke laptop. Kamera GoPro HERO11 Black dihubungkan ke laptop melalui antarmuka USB to Type-C. Pada beberapa kondisi, *stream* dari GoPro tidak dapat dibaca langsung oleh sistem (*black screen*), sehingga digunakan OBS Studio sebagai perantara untuk menangkap dan menyediakan sumber video yang stabil. Dengan demikian, sumber input sistem dapat berasal dari kamera secara langsung maupun dari *virtual camera* yang disediakan oleh OBS Studio, sesuai dengan konfigurasi perangkat input pada program.

b. *Unsharp Masking*

Setiap frame yang diterima akan melalui *preprocessing* dengan *unsharp masking* sebelum masuk ke tahap deteksi. Tahap ini bertujuan meningkatkan ketajaman detail permukaan kentang yang memiliki kemiripan visual tinggi dengan tanah, sehingga fitur pembeda objek menjadi lebih jelas. Parameter yang digunakan pada penelitian ini adalah *sigma* sebesar 1.5 untuk mengontrol tingkat *blur* pada *Gaussian kernel*, *amount* sebesar 1.5 untuk mengontrol kekuatan penajaman yang diterapkan, dan *threshold* sebesar 6 untuk membatasi penajaman hanya pada piksel yang memiliki perbedaan intensitas di atas nilai ambang batas tersebut guna mencegah amplifikasi *noise* pada area yang relatif seragam.



(a)



(b)

Gambar 28. Perbandingan citra pada tahap *preprocessing*: (a) Citra original; (b) Citra dengan *unsharp masking*.

c. *Define Region of Interest (ROI)*

Sistem menetapkan *Region of Interest (ROI)* sebagai batas area deteksi pada *frame* video. ROI digunakan untuk membatasi proses komputasi hanya pada area bedengan yang umumnya memiliki lebar 75–80 cm, sehingga sistem dapat mengabaikan gangguan visual (*noise*) di luar jalur lintasan dan menekan potensi kesalahan deteksi (*false positive*). ROI pada pengujian ini bersifat statis dan dikalibrasi sesuai parameter skenario pada penelitian.



Gambar 29. Area ROI

d. *Define Counting Line*

Sistem mendefinisikan *counting line* sebagai garis horizontal yang diposisikan pada koordinat tertentu di dalam *frame*, ditetapkan melalui rasio tinggi *frame*. Garis ini berfungsi sebagai acuan *event* penghitungan sehingga proses *counting* memiliki kriteria yang konsisten dan menekan peluang penghitungan ganda. Dalam skenario pengujian, arah gerak kamera mengikuti gerak alat sehingga pergerakan kentang pada bidang citra diperlakukan sebagai pergerakan dari atas menuju bawah.

Gambar 30. Posisi *counting line*

e. *Detect Potatoes*

Setiap *frame* yang berada di dalam area ROI diproses menggunakan model YOLOv5 dengan bobot terbaik (*best weight*) hasil *training*. Model melakukan inferensi pada ROI untuk mendeteksi objek kentang secara *real-time*, dengan keluaran berupa koordinat *bounding box* dan informasi *confidence score* untuk setiap deteksi.



Gambar 31. Visualisasi hasil deteksi kentang

f. *Track Potatoes*

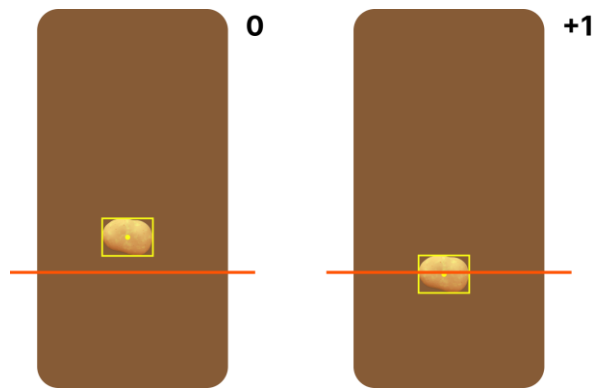
Hasil deteksi diteruskan ke tahap *tracking* menggunakan algoritma ByteTrack untuk menghasilkan ID unik pada setiap objek, sehingga identitas objek dapat dipertahankan antarframe. Sistem menyimpan status setiap ID, termasuk posisi *centroid* pada *frame* sebelumnya dan status apakah objek tersebut sudah dihitung. Mekanisme ini menjaga konsistensi identitas objek dan membantu mencegah penghitungan ganda pada objek yang sama.



Gambar 32. Visualisasi ID pada objek hasil deteksi

g. *Count Potatoes*

Penghitungan dilakukan dengan memantau posisi *centroid* dari *bounding box* setiap ID yang aktif. Jumlah kentang ditambah satu hanya jika terpenuhi empat kondisi: (1) *centroid* pada *frame* sebelumnya berada di atas *counting line*, (2) *centroid* pada *frame* saat ini berada pada atau di bawah *counting line*, (3) arah gerak *centroid* mengarah ke bawah dengan perpindahan minimal satu piksel, dan (4) ID tersebut belum pernah dihitung sebelumnya. Setelah *event counting* terpenuhi, ID tersebut ditandai sebagai sudah dihitung sehingga tidak menambah nilai hitung pada *frame* berikutnya.



Gambar 33. Ilustrasi *event counting*

h. Evaluasi Sistem

Evaluasi sistem dilakukan dalam dua aspek utama, yaitu performa penghitungan dan pemrosesan *real-time*. Performa penghitungan diukur menggunakan *Mean Absolute Error* (MAE) dan *Root Mean Square Error* (RMSE) dengan membandingkan jumlah kentang yang diprediksi sistem terhadap jumlah kentang aktual sebagai *ground truth*. Sementara itu, performa pemrosesan *real-time* diukur menggunakan *Frames Per Second* (FPS) untuk mengetahui kemampuan sistem dalam memproses *frame* video secara berkelanjutan.

BAB III

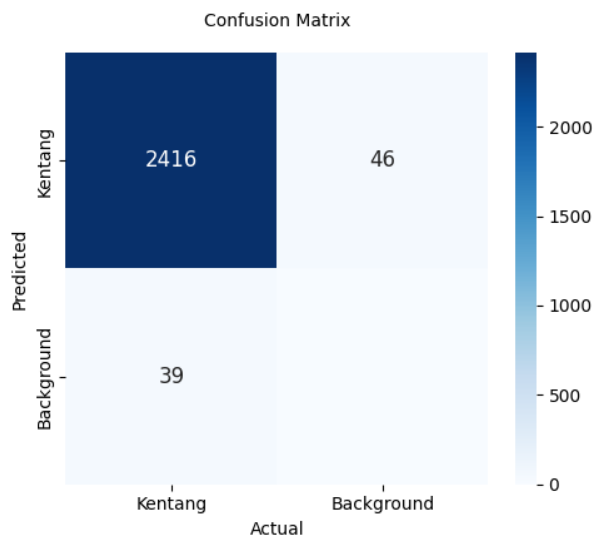
HASIL DAN PEMBAHASAN

3.1 Hasil Pengujian Model YOLOv5

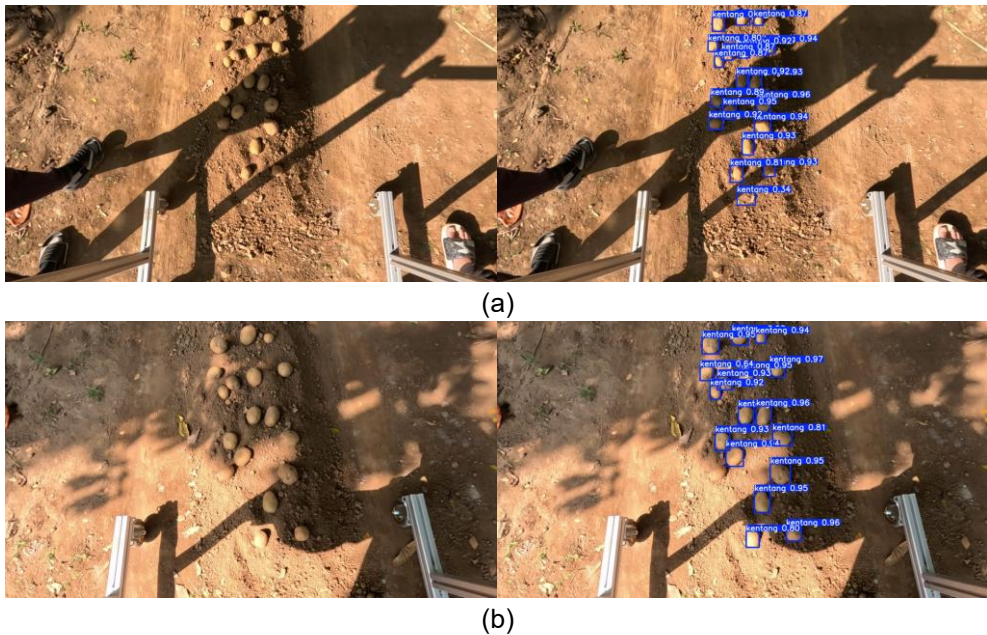
Setelah dilakukan pengujian terhadap lima varian model YOLOv5 yaitu *nano*, *small*, *medium*, *large*, dan *xtra* menggunakan data *test* sejumlah 168 gambar dengan total *Ground Truth* (GT) sebanyak 2.455 objek kentang, dilakukan evaluasi menggunakan metrik deteksi objek sebagai berikut.

3.1.1 YOLOv5 Varian *Nano*

Berdasarkan Gambar 34, model YOLOv5 varian *nano* mengklasifikasikan 2.416 objek kentang secara benar (*true positive*), sementara terdapat 46 kasus *false positive*, yaitu kondisi ketika model mendeteksi objek sebagai kentang padahal sebenarnya merupakan latar belakang. Selain itu, terdapat 39 kasus *false negative*, yaitu objek kentang yang tidak berhasil terdeteksi oleh model. Hasil ini menunjukkan bahwa sebagian besar objek kentang berhasil diidentifikasi dengan benar oleh model, sedangkan jumlah kesalahan deteksi relatif kecil dibandingkan total prediksi.



Gambar 34. *Confusion matrix* YOLOv5 varian *nano*



Gambar 35. Hasil deteksi YOLOv5 varian *nano*: (a) Sampel *false positive*; (b) Sampel *false negative*.

Gambar 35(a) memperlihatkan sampel *false positive*, di mana model memberikan *bounding box* pada beberapa objek yang sebenarnya bukan kentang, seperti daun kering atau tanah yang terkena cahaya matahari memiliki warna yang menyerupai kentang. Hal ini menunjukkan bahwa kemiripan karakteristik visual antara objek kentang dan elemen tertentu pada latar belakang dapat memicu kesalahan deteksi oleh model. Sementara itu, Gambar 35(b) menunjukkan sampel *false negative* yaitu kondisi ketika objek kentang yang berada dalam citra tidak berhasil terdeteksi oleh model sehingga tidak memiliki *bounding box*. Kesalahan ini umumnya terjadi ketika objek tertutup sebagian oleh bayangan, atau memiliki kontras yang rendah terhadap latar belakang, sehingga fitur visualnya tidak cukup kuat untuk dikenali oleh model.

Tabel 7. Performa pengujian YOLOv5 varian *nano*

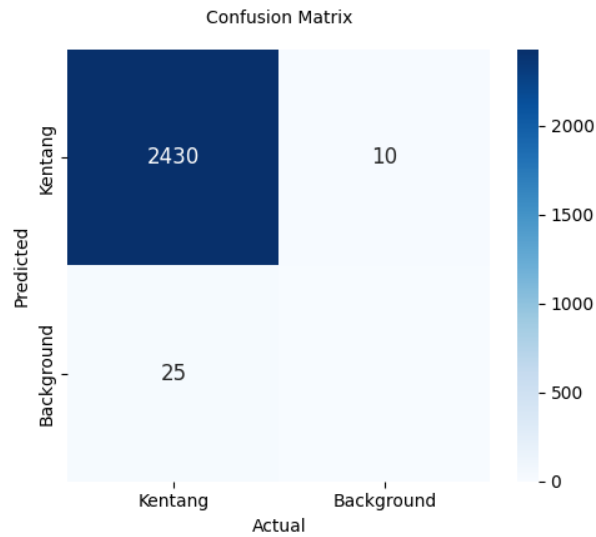
Metrik	Nilai
<i>Precision</i>	0.981
<i>Recall</i>	0.984
<i>F1-Score</i>	0.983
mAP50	0.991
mAP50-95	0.886

Metrik	Nilai
<i>Model Size (MB)</i>	3.81
<i>Preprocessing Time (ms)</i>	0.4
<i>Inference Time (ms)</i>	5.2
<i>Postprocessing Time (ms)</i>	2.1

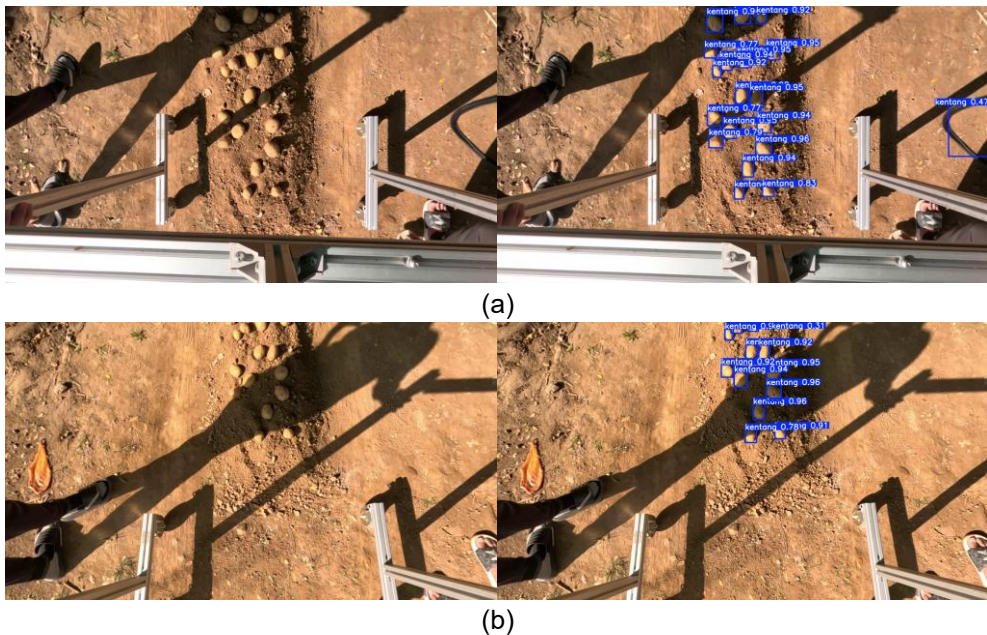
Berdasarkan Tabel 7, model mencapai nilai *precision* sebesar 0.981 dan *recall* sebesar 0.984, yang menunjukkan bahwa sebagian besar deteksi yang dihasilkan model merupakan prediksi yang benar dan sebagian besar objek kentang dalam citra berhasil terdeteksi. Kombinasi kedua metrik tersebut menghasilkan F1-score sebesar 0.983, yang mengindikasikan keseimbangan yang baik antara ketepatan dan kelengkapan deteksi. Selain itu, model mencapai mAP50 sebesar 0.991, yang menunjukkan tingkat akurasi deteksi yang sangat tinggi pada ambang *Intersection over Union* (IoU) sebesar 0.5. Ketika evaluasi dilakukan pada rentang IoU yang lebih ketat (0.5–0.95), nilai mAP50–95 yang diperoleh sebesar 0.886, yang menunjukkan bahwa model tetap mempertahankan performa deteksi yang baik meskipun kriteria evaluasi lebih ketat. Dari sisi efisiensi komputasi, model memiliki ukuran sebesar 3.81 MB, sehingga tergolong ringan dan sesuai untuk implementasi pada sistem *real-time*. Waktu pemrosesan juga relatif singkat, dengan *preprocessing* sebesar 0.4 ms, *inference* sebesar 5.2 ms, dan *postprocessing* sebesar 2.1 ms. Dengan total waktu pemrosesan sekitar 7.7 ms per *frame*, model memiliki potensi untuk diimplementasikan secara *real-time*.

3.1.2 YOLOv5 Varian *Small*

Berdasarkan Gambar 36, model YOLOv5 varian *small* mengklasifikasikan 2.430 objek kentang secara benar (*true positive*), sementara terdapat 10 kasus *false positive*, yaitu kondisi ketika model mendeteksi objek sebagai kentang padahal sebenarnya merupakan latar belakang. Selain itu, terdapat 25 kasus *false negative*, yaitu objek kentang yang tidak berhasil terdeteksi oleh model. Hasil ini menunjukkan bahwa sebagian besar objek kentang berhasil diidentifikasi dengan benar oleh model, sedangkan jumlah kesalahan deteksi relatif kecil dibandingkan total prediksi.



Gambar 36. *Confusion matrix* YOLOv5 varian *small*



Gambar 37. Hasil deteksi YOLOv5 varian *small*: (a) Sampel *false positive*; (b) Sampel *false negative*.

Gambar 37(a) memperlihatkan sampel *false positive*, di mana model memberikan *bounding box* pada beberapa objek yang sebenarnya bukan kentang. Pada kasus ini, kesalahan deteksi dipengaruhi oleh keberadaan objek kabel di atas permukaan tanah yang membentuk pola tepi tertentu. Pola tepi tersebut, ketika berpadu dengan warna tanah yang menyerupai warna kentang, menyebabkan area

tersebut terinterpretasi oleh model sebagai objek kentang. Hal ini menunjukkan bahwa kombinasi antara kemiripan warna dan kemunculan struktur tepi semu pada latar belakang dapat memicu kesalahan deteksi oleh model. Sementara itu, Gambar 37(b) menunjukkan sampel *false negative* yaitu kondisi ketika objek kentang yang berada dalam citra tidak berhasil terdeteksi oleh model sehingga tidak memiliki *bounding box*. Kesalahan ini umumnya terjadi ketika objek tertutup sebagian oleh bayangan, atau memiliki kontras yang rendah terhadap latar belakang, sehingga fitur visualnya tidak cukup kuat untuk dikenali oleh model.

Tabel 8. Performa pengujian YOLOv5 varian *small*

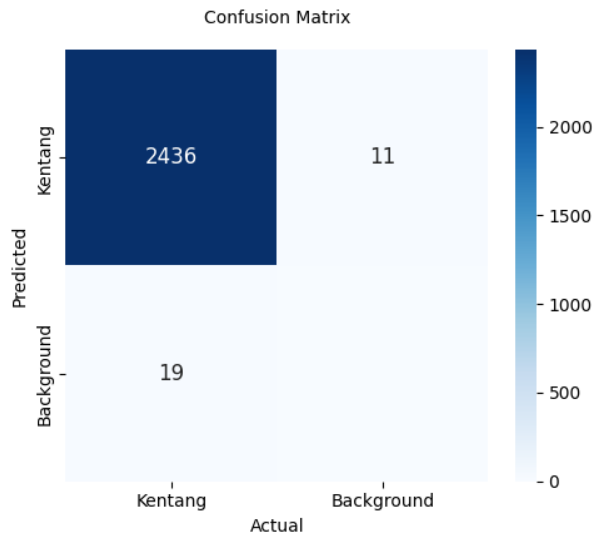
Metrik	Nilai
<i>Precision</i>	0.996
<i>Recall</i>	0.990
<i>F1-Score</i>	0.993
mAP50	0.995
mAP50-95	0.917
<i>Model Size (MB)</i>	14.35
<i>Preprocessing Time (ms)</i>	0.4
<i>Inference Time (ms)</i>	5.9
<i>Postprocessing Time (ms)</i>	2.2

Berdasarkan Tabel 8, model mencapai *precision* sebesar 0.996 dan *recall* sebesar 0.990, yang menunjukkan bahwa hampir seluruh deteksi yang dihasilkan model merupakan prediksi yang benar serta sebagian besar objek kentang yang terdapat dalam citra berhasil terdeteksi. Kombinasi kedua metrik tersebut menghasilkan F1-score sebesar 0.993, yang menunjukkan keseimbangan yang sangat baik antara ketepatan dan kelengkapan deteksi. Dari sisi efisiensi komputasi, model memiliki ukuran sebesar 14.35 MB, yang lebih besar dibandingkan varian Nano, namun masih tergolong cukup ringan. Waktu pemrosesan juga relatif singkat, dengan *preprocessing* sebesar 0.4 ms, *inference* sebesar 5.9 ms, dan *postprocessing* sebesar 2.2 ms. Dengan total waktu pemrosesan sekitar 8.5 ms per *frame*, model tetap memiliki potensi untuk diimplementasikan secara *real-time*.

3.1.3 YOLOv5 Varian *Medium*

Berdasarkan Gambar 38, model YOLOv5 varian *medium* mengklasifikasikan 2.436 objek kentang secara benar (*true positive*), sementara terdapat 11 kasus *false positive*, yaitu kondisi ketika model mendeteksi objek sebagai kentang padahal sebenarnya merupakan latar belakang. Selain itu, terdapat 19 kasus *false negative*,

yaitu objek kentang yang tidak berhasil terdeteksi oleh model. Hasil ini menunjukkan bahwa sebagian besar objek kentang berhasil diidentifikasi dengan benar oleh model, sedangkan jumlah kesalahan deteksi relatif kecil dibandingkan total prediksi.



Gambar 38. *Confusion matrix* YOLOv5 varian *medium*



(a)



(b)

Gambar 39. Hasil deteksi YOLOv5 varian *medium*: (a) Sampel *false positive*; (b) Sampel *false negative*.

Gambar 39(a) memperlihatkan sampel *false positive*, di mana model memberikan *bounding box* pada area yang sebenarnya bukan kentang. Pada kasus ini, kesalahan deteksi dipengaruhi oleh kondisi pencahayaan lapangan, khususnya pantulan atau intensitas cahaya pada permukaan tanah yang menyebabkan area tersebut memiliki karakteristik visual yang menyerupai objek kentang. Kemiripan warna antara tanah yang terkena cahaya dengan permukaan kentang menyebabkan model menginterpretasikan area tersebut sebagai objek kentang. Hal ini menunjukkan bahwa variasi pencahayaan di lingkungan lapangan dapat memengaruhi akurasi deteksi model, terutama ketika menghasilkan pola visual yang menyerupai objek target. Sementara itu, Gambar 39(b) menunjukkan sampel *false negative*, yaitu kondisi ketika objek kentang yang berada dalam citra tidak berhasil terdeteksi oleh model sehingga tidak memiliki *bounding box*. Kesalahan ini umumnya terjadi ketika objek tertutup sebagian oleh bayangan atau memiliki kontras yang rendah terhadap latar belakang, sehingga fitur visualnya tidak cukup kuat untuk dikenali oleh model.

Tabel 9. Performa pengujian YOLOv5 varian *medium*

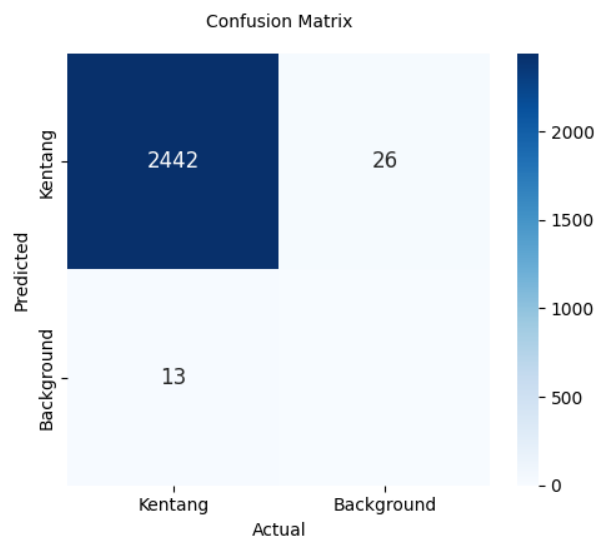
Metrik	Nilai
<i>Precision</i>	0.996
<i>Recall</i>	0.992
<i>F1-Score</i>	0.994
mAP50	0.995
mAP50-95	0.927
<i>Model Size (MB)</i>	42.13
<i>Preprocessing Time (ms)</i>	0.4
<i>Inference Time (ms)</i>	10.3
<i>Postprocessing Time (ms)</i>	2.0

Berdasarkan Tabel 9, model mencapai *precision* sebesar 0.996 dan *recall* sebesar 0.992, yang menunjukkan bahwa hampir seluruh deteksi yang dihasilkan model merupakan prediksi yang benar serta sebagian besar objek kentang yang terdapat dalam citra berhasil terdeteksi. Kombinasi kedua metrik tersebut menghasilkan *F1-score* sebesar 0.994, yang menunjukkan keseimbangan yang sangat baik antara ketepatan dan kelengkapan deteksi. Selain itu, model mencapai mAP50 sebesar 0.995, yang menunjukkan tingkat akurasi deteksi yang sangat tinggi pada ambang *Intersection over Union* (IoU) sebesar 0.5. Ketika evaluasi dilakukan pada rentang IoU yang lebih ketat, model memperoleh mAP50-95 sebesar 0.927, yang menunjukkan bahwa performa deteksi tetap sangat baik meskipun menggunakan kriteria evaluasi yang lebih ketat. Dari sisi efisiensi komputasi, model

memiliki ukuran sebesar 42.13 MB, yang jauh lebih besar dibandingkan varian *Nano* dan *Small*. Waktu pemrosesan model terdiri atas *preprocessing* sebesar 0.4 ms, *inference* sebesar 10.3 ms, dan *postprocessing* sebesar 2.0 ms. Dengan total waktu pemrosesan sekitar 12.7 ms per frame, mengindikasikan bahwa beban komputasi cukup tinggi, sehingga implementasi *real-time* memerlukan perangkat dengan kemampuan pemrosesan yang memadai agar performa tetap stabil.

3.1.4 YOLOv5 Varian *Large*

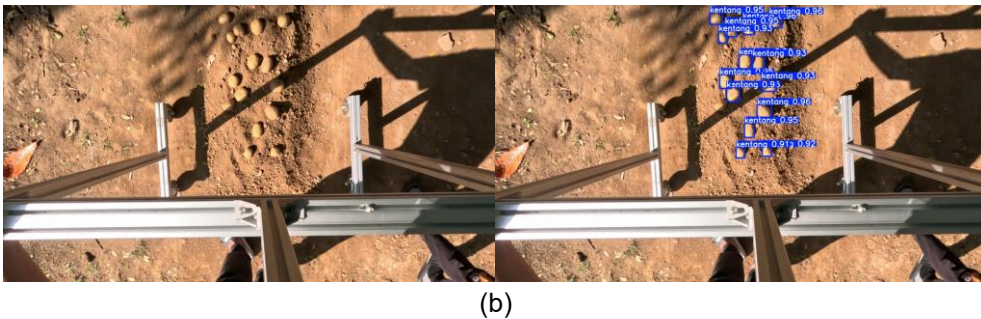
Berdasarkan Gambar 40, model YOLOv5 varian *large* mengklasifikasikan 2.442 objek kentang secara benar (*true positive*), sementara terdapat 26 kasus *false positive*, yaitu kondisi ketika model mendeteksi objek sebagai kentang padahal sebenarnya merupakan latar belakang. Selain itu, terdapat 13 kasus *false negative*, yaitu objek kentang yang tidak berhasil terdeteksi oleh model. Hasil ini menunjukkan bahwa sebagian besar objek kentang berhasil diidentifikasi dengan benar oleh model, sedangkan jumlah kesalahan deteksi relatif kecil dibandingkan total prediksi.



Gambar 40. *Confusion matrix* YOLOv5 varian *large*



(a)



Gambar 41. Hasil deteksi YOLOv5 varian *large*: (a) Sampel *false positive*; (b) Sampel *false negative*.

Gambar 41(a) memperlihatkan sampel *false positive*, di mana model memberikan *bounding box* pada beberapa objek yang sebenarnya bukan kentang. Pada kasus ini, kesalahan deteksi dipengaruhi oleh keberadaan daun kering di permukaan tanah yang secara visual memiliki bentuk dan warna yang menyerupai kentang. Kemiripan karakteristik visual tersebut menyebabkan model menginterpretasikan objek tersebut sebagai kentang. Hal ini menunjukkan bahwa elemen alami pada lingkungan lapangan yang memiliki kesamaan warna maupun tekstur dengan objek target dapat memicu kesalahan deteksi oleh model. Sementara itu, Gambar 41(b) menunjukkan sampel *false negative*, yaitu kondisi ketika objek kentang yang berada dalam citra tidak berhasil terdeteksi oleh model sehingga tidak memiliki *bounding box*. Kesalahan ini umumnya terjadi ketika objek tertutup sebagian oleh bayangan atau memiliki kontras yang rendah terhadap latar belakang, sehingga fitur visualnya tidak cukup kuat untuk dikenali oleh model.

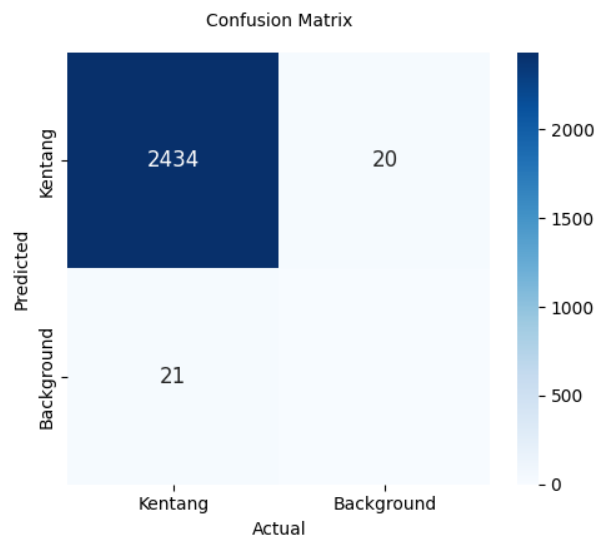
Tabel 10. Performa pengujian YOLOv5 varian *large*

Metriik	Nilai
<i>Precision</i>	0.989
<i>Recall</i>	0.995
<i>F1-Score</i>	0.992
mAP50	0.995
mAP50-95	0.937
<i>Model Size (MB)</i>	92.76
<i>Preprocessing Time (ms)</i>	0.5
<i>Inference Time (ms)</i>	16.2
<i>Postprocessing Time (ms)</i>	2.2

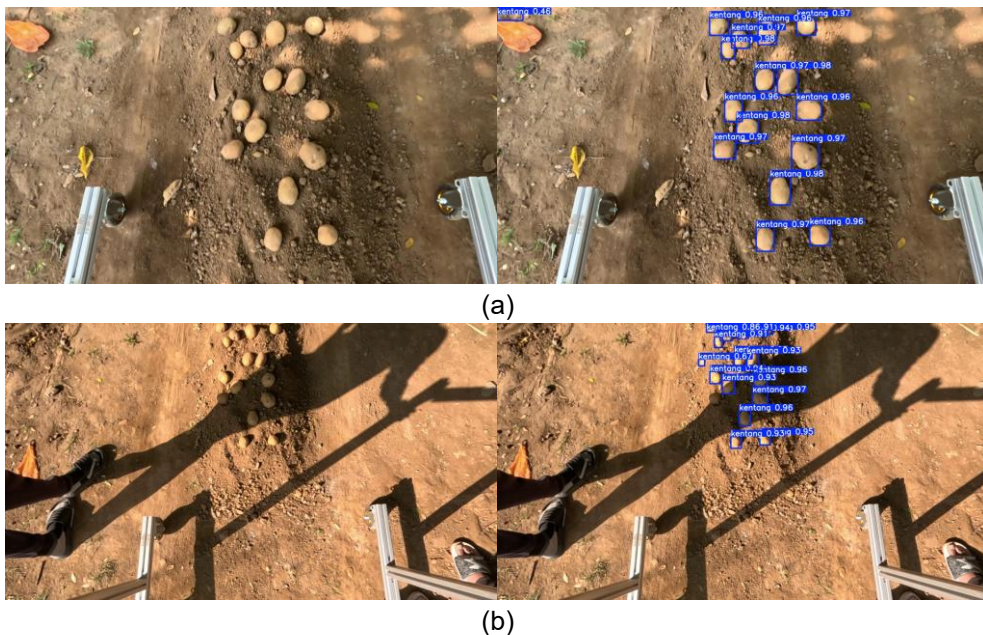
Berdasarkan Tabel 10, model mencapai *precision* sebesar 0.989 dan *recall* sebesar 0.995, yang menunjukkan bahwa sebagian besar objek kentang yang terdapat dalam citra berhasil terdeteksi oleh model, meskipun masih terdapat sejumlah kecil prediksi yang salah. Kombinasi kedua metrik tersebut menghasilkan F1-score sebesar 0.992, yang menunjukkan keseimbangan yang sangat baik antara ketepatan dan kelengkapan deteksi. Selain itu, model mencapai mAP50 sebesar 0.995, yang menunjukkan tingkat akurasi deteksi yang sangat tinggi pada ambang *Intersection over Union* (IoU) sebesar 0.5. Ketika evaluasi dilakukan pada rentang IoU yang lebih ketat, model memperoleh mAP50-95 sebesar 0.937, yang menunjukkan bahwa model mampu mempertahankan performa deteksi yang sangat baik meskipun menggunakan kriteria evaluasi yang lebih ketat. Dari sisi efisiensi komputasi, model memiliki ukuran sebesar 92.76 MB, yang jauh lebih besar dibandingkan varian *nano*, *small*, dan *medium*. Waktu pemrosesan model terdiri atas *preprocessing* sebesar 0.5 ms, *inference* sebesar 16.2 ms, dan *postprocessing* sebesar 2.2 ms. Dengan total waktu pemrosesan sekitar 18.9 ms per *frame*, beban komputasi lebih besar dan kurang ideal diimplementasikan pada sistem *real-time*.

3.1.5 YOLOv5 Varian Xtra

Berdasarkan Gambar 42, model YOLOv5 varian *xtra* mengklasifikasikan 2.434 objek kentang secara benar (*true positive*), sementara terdapat 20 kasus *false positive*, yaitu kondisi ketika model mendeteksi objek sebagai kentang padahal sebenarnya merupakan latar belakang. Selain itu, terdapat 21 kasus *false negative*, yaitu objek kentang yang tidak berhasil terdeteksi oleh model. Hasil ini menunjukkan bahwa sebagian besar objek kentang berhasil diidentifikasi dengan benar oleh model, sedangkan jumlah kesalahan deteksi relatif kecil dibandingkan total prediksi.



Gambar 42. *Confusion matrix* YOLOv5 varian *xtra*



Gambar 43. Hasil deteksi YOLOv5 varian *xtra*: (a) Sampel *false positive*; (b) Sampel *false negative*.

Gambar 43(a) memperlihatkan sampel *false positive*, di mana model memberikan *bounding box* pada beberapa area yang sebenarnya bukan kentang. Pada kasus ini, kesalahan deteksi dipengaruhi oleh area tanah yang terkena cahaya matahari sehingga memiliki karakteristik visual yang menyerupai kentang, serta keberadaan daun kering yang juga memiliki kemiripan warna dan bentuk dengan objek kentang. Kombinasi kemiripan warna, tekstur, dan pola tepi tersebut menyebabkan model menginterpretasikan area tersebut sebagai kentang. Hal ini menunjukkan bahwa variasi kondisi pencahayaan lapangan dan keberadaan objek alami pada latar belakang dapat memicu kesalahan deteksi oleh model. Sementara itu, Gambar 43(b) menunjukkan sampel *false negative*, yaitu kondisi ketika objek kentang yang berada dalam citra tidak berhasil terdeteksi oleh model sehingga tidak memiliki *bounding box*. Kesalahan ini umumnya terjadi ketika objek tertutup sebagian oleh bayangan atau memiliki kontras yang rendah terhadap latar belakang, sehingga fitur visualnya tidak cukup kuat untuk dikenali oleh model.

Tabel 11. Performa pengujian YOLOv5 varian *xtra*

Metrik	Nilai
<i>Precision</i>	0.992
<i>Recall</i>	0.991
<i>F1-Score</i>	0.992

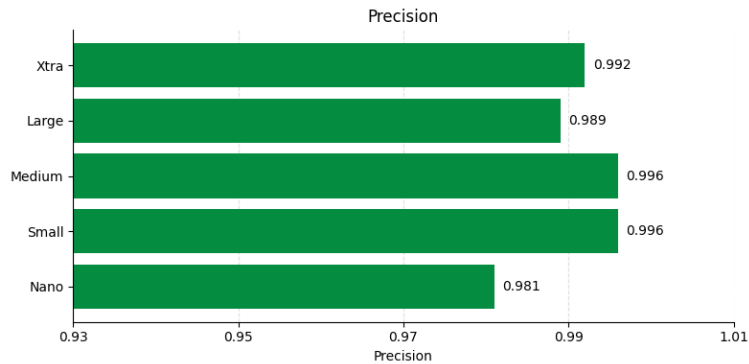
Metrik	Nilai
mAP50	0.995
mAP50-95	0.935
Model Size (MB)	173.02
Preprocessing Time (ms)	0.4
Inference Time (ms)	23.4
Postprocessing Time (ms)	2.1

Berdasarkan Tabel 12, model mencapai *precision* sebesar 0.992 dan *recall* sebesar 0.991, yang menunjukkan bahwa sebagian besar deteksi yang dihasilkan model merupakan prediksi yang benar dan sebagian besar objek kentang yang terdapat dalam citra berhasil terdeteksi. Kombinasi kedua metrik tersebut menghasilkan F1-score sebesar 0.992, yang menunjukkan keseimbangan yang sangat baik antara ketepatan dan kelengkapan deteksi. Selain itu, model mencapai mAP50 sebesar 0.995, yang menunjukkan tingkat akurasi deteksi yang sangat tinggi pada ambang *Intersection over Union* (IoU) sebesar 0.5. Ketika evaluasi dilakukan pada rentang IoU yang lebih ketat, model memperoleh mAP50-95 sebesar 0.935, yang menunjukkan bahwa performa deteksi tetap sangat baik meskipun menggunakan kriteria evaluasi yang lebih ketat. Dari sisi efisiensi komputasi, model memiliki ukuran sebesar 173.02 MB, yang merupakan ukuran terbesar dibandingkan varian sebelumnya. Waktu pemrosesan model terdiri atas *preprocessing* sebesar 0.4 ms, *inference* sebesar 23.4 ms, dan *postprocessing* sebesar 2.1 ms. Dengan total waktu pemrosesan sekitar 25.9 ms per frame, model menunjukkan beban komputasi yang sangat tinggi sehingga kurang ideal untuk implementasi pada sistem *real-time*.

3.1.6 Perbandingan Model

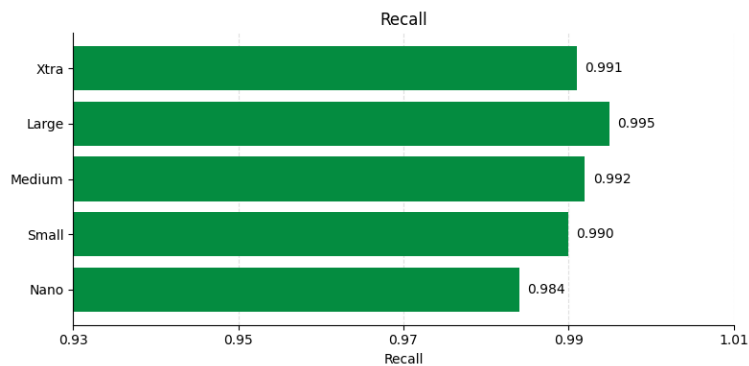
1. *Precision*

Berdasarkan nilai *precision*, model YOLOv5 varian *small* dan *medium* memperoleh nilai tertinggi, yaitu 0.996, yang menunjukkan bahwa kedua model tersebut memiliki tingkat ketepatan prediksi positif paling baik dibandingkan varian lainnya. Artinya, ketika model mendeteksi suatu objek sebagai kentang, probabilitas bahwa prediksi tersebut benar sangat tinggi. Varian *xtra* berada sedikit di bawahnya dengan nilai 0.992, diikuti oleh varian *large* sebesar 0.989, sedangkan varian *nano* memiliki nilai terendah, yaitu 0.981. Hasil ini menunjukkan bahwa varian *small* dan *medium* lebih mampu menekan *false positive* dibandingkan model lain.

Gambar 44. Perbandingan *precision*

2. *Recall*

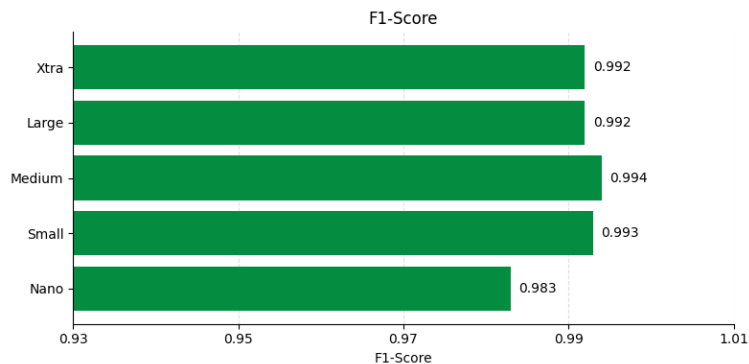
Berdasarkan nilai *recall*, YOLOv5 varian *large* menunjukkan performa terbaik dengan nilai 0.995, yang berarti model ini paling mampu mendeteksi objek kentang yang benar-benar ada pada citra. Dengan kata lain, varian *large* memiliki kemampuan paling baik dalam meminimalkan *false negative*. Selanjutnya, varian *medium* memperoleh nilai 0.992, disusul oleh *xtra* sebesar 0.991, *small* sebesar 0.990, dan *nano* sebesar 0.984. Meskipun seluruh varian menunjukkan *recall* yang tinggi, varian *large* unggul dalam aspek kelengkapan deteksi objek.

Gambar 45. Perbandingan *recall*

3. F1-Score

Berdasarkan nilai F1-score, yang merepresentasikan keseimbangan antara *precision* dan *recall*, YOLOv5 varian *medium* memperoleh nilai tertinggi, yaitu 0.994. Hal ini menunjukkan bahwa varian *medium* memiliki kompromi terbaik antara ketepatan dan kelengkapan deteksi. Varian *small* berada sangat dekat dengan nilai 0.993, sedangkan varian *large* dan *xtra* sama-sama memperoleh 0.992. Adapun varian *nano* memiliki nilai 0.983, yang

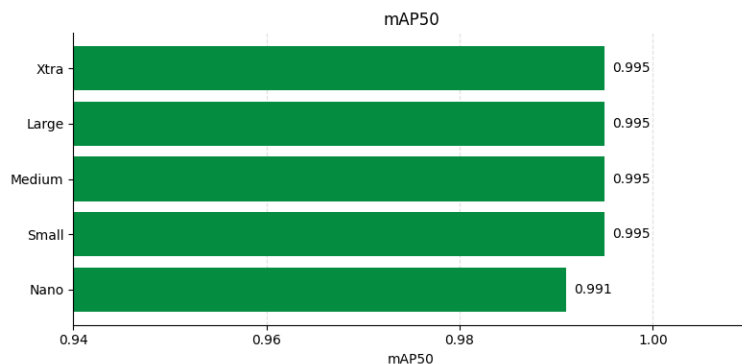
meskipun masih tinggi, tetap berada di bawah varian lain. Dengan demikian, dari sisi keseimbangan performa deteksi, varian *medium* sedikit lebih unggul, namun varian *small* tetap sangat kompetitif.



Gambar 46. Perbandingan F1-score

4. *mAP50*

Berdasarkan metrik *mAP50*, hampir seluruh varian selain *nano* menunjukkan performa yang sama sangat tinggi, yaitu 0.995, meliputi varian *small*, *medium*, *large*, dan *xtra*. Sementara itu, varian *nano* memperoleh nilai 0.991, yang juga tergolong sangat baik. Hasil ini menunjukkan bahwa pada ambang IoU 0,5, seluruh varian YOLOv5 memiliki kemampuan deteksi yang sangat tinggi dan perbedaannya relatif kecil.

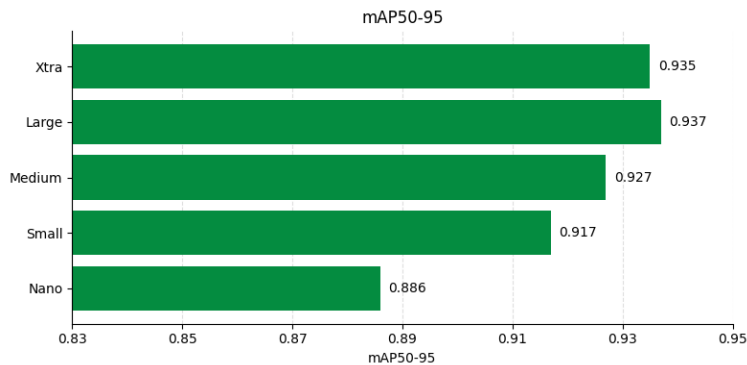


Gambar 47. Perbandingan mAP50

5. *mAP50-95*

Berdasarkan metrik *mAP50-95*, yang merepresentasikan kualitas deteksi pada rentang IoU yang lebih ketat, YOLOv5 varian *large* memperoleh nilai tertinggi, yaitu 0.937, diikuti sangat dekat oleh varian *xtra* sebesar 0.935, *medium* sebesar 0.927, *small* sebesar 0.917, dan *nano* sebesar 0.886. Hasil

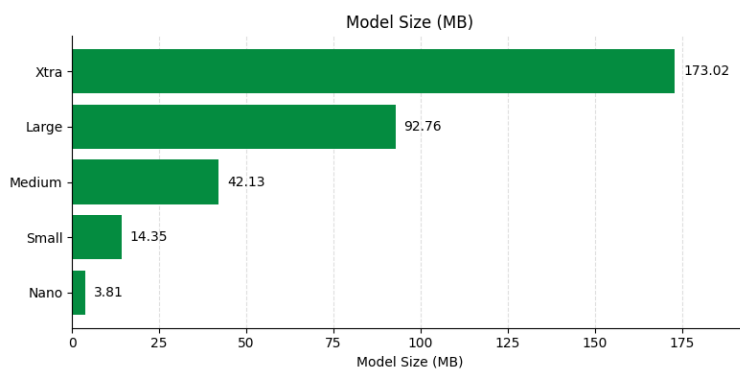
ini menunjukkan bahwa semakin besar kapasitas model, umumnya semakin baik pula ketepatan lokalisasi *bounding box* yang dihasilkan.



Gambar 48. Perbandingan mAP50-95

6. Model Size

Berdasarkan ukuran model, YOLOv5 varian *nano* merupakan model paling ringan dengan ukuran hanya 3.81 MB, diikuti oleh *small* sebesar 14.35 MB, *medium* sebesar 42.13 MB, *large* sebesar 92.76 MB, dan *xtra* sebagai model terbesar dengan ukuran 173.02 MB. Perbedaan ukuran ini menunjukkan peningkatan kompleksitas arsitektur dari tiap varian. Model yang lebih kecil umumnya lebih mudah diimplementasikan pada perangkat dengan sumber daya terbatas, sedangkan model yang lebih besar membutuhkan memori dan kapasitas komputasi yang lebih tinggi.

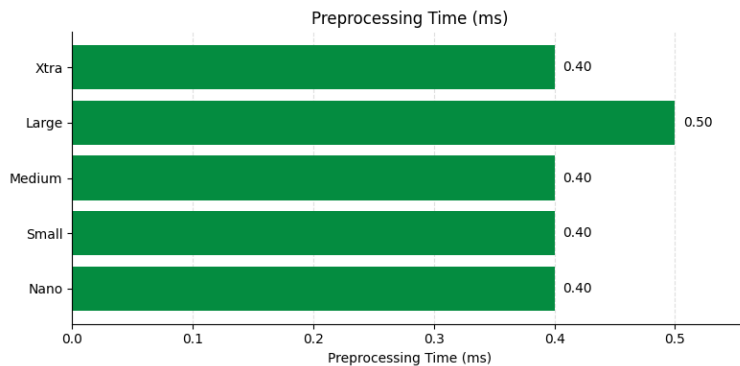


Gambar 49. Perbandingan *model size*

7. Preprocessing Time

Berdasarkan waktu *preprocessing*, hampir seluruh varian menunjukkan waktu yang sama, yaitu 0.4 ms, kecuali varian *large* yang sedikit lebih tinggi, yaitu 0.5 ms. Hasil ini menunjukkan bahwa perbedaan varian model tidak memberikan pengaruh yang signifikan terhadap waktu *preprocessing*.

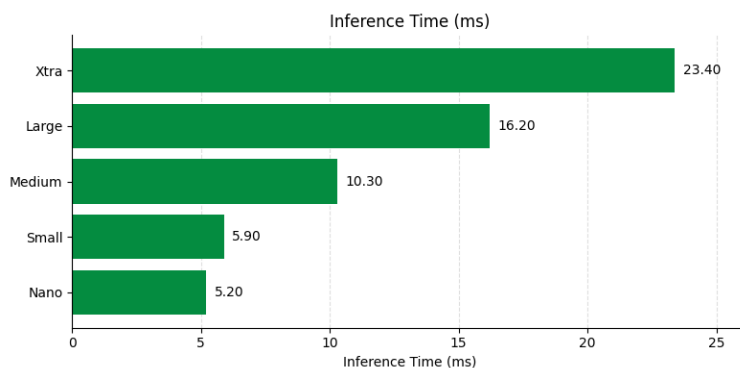
Dengan demikian, tahap ini bukan merupakan faktor pembeda utama dalam evaluasi efisiensi komputasi antarvarian YOLOv5, karena seluruh model bekerja pada rentang waktu yang sangat dekat.



Gambar 50. Perbandingan *preprocessing time*

8. *Inference Time*

Berdasarkan waktu *inference*, YOLOv5 varian *nano* merupakan model tercepat dengan waktu inferensi 5.2 ms, diikuti oleh *small* sebesar 5.9 ms, *medium* sebesar 10.3 ms, *large* sebesar 16.2 ms, dan *xtra* sebesar 23.4 ms. Hasil ini menunjukkan bahwa semakin besar ukuran model, semakin besar pula waktu yang dibutuhkan untuk melakukan inferensi. Dalam konteks sistem *real-time*, metrik ini sangat penting karena secara langsung memengaruhi jumlah *frame* yang dapat diproses per detik. Oleh karena itu, varian *nano* dan *small* lebih unggul dari sisi kecepatan inferensi.

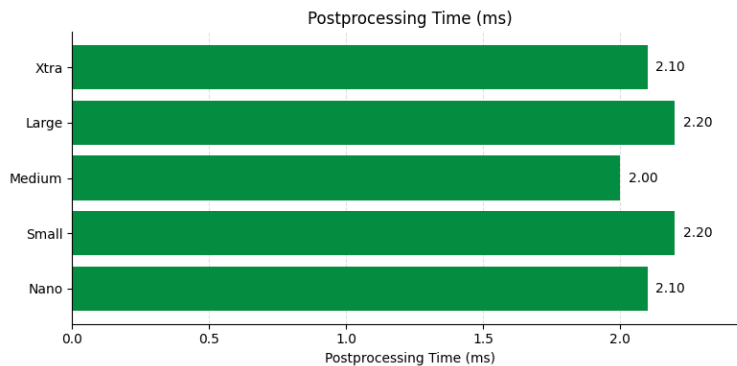


Gambar 51. Perbandingan *inference time*

9. *Postprocessing Time*

Berdasarkan waktu *postprocessing*, seluruh varian juga menunjukkan perbedaan yang sangat kecil. YOLOv5 varian *medium* mencatat waktu tercepat, yaitu 2.0 ms, sedangkan *nano* dan *xtra* masing-masing sebesar 2.1

ms, serta *small* dan *large* sebesar 2.2 ms. Karena selisihnya sangat kecil, tahap *postprocessing* tidak menjadi faktor penentu utama dalam membedakan performa real-time antar model. Namun demikian, hasil ini menunjukkan bahwa seluruh varian memiliki efisiensi yang relatif seragam pada tahap akhir pemrosesan.

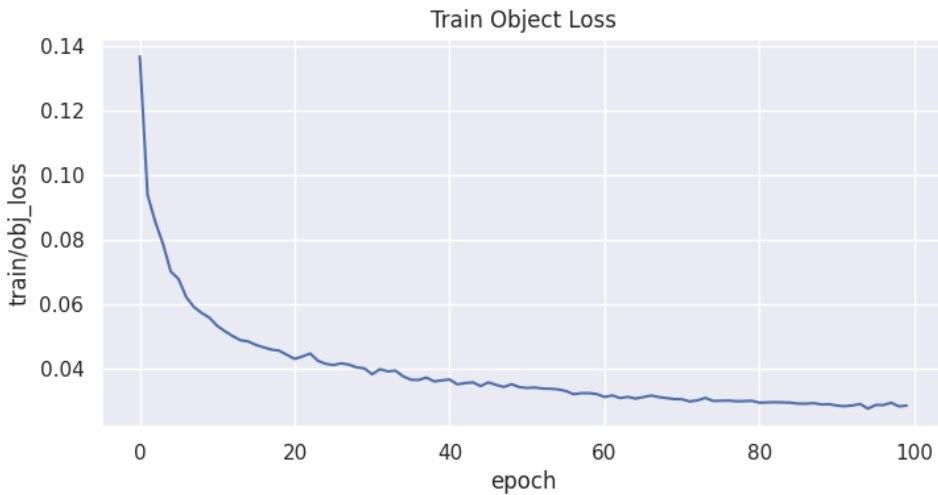


Gambar 52. Perbandingan *postprocessing time*

Secara keseluruhan, meskipun YOLOv5 varian *medium* dan *large* menunjukkan keunggulan pada beberapa metrik akurasi, YOLOv5 varian *small* dapat dianggap sebagai model yang paling optimal untuk sistem deteksi dan hitung objek kentang multi-skala secara real-time. Hal ini disebabkan varian *small* mampu memberikan performa deteksi yang sangat tinggi, dengan *precision* 0.996, *recall* 0.990, *F1-score* 0.993, *mAP50* 0.995, dan *mAP50-95* 0.917, namun tetap mempertahankan ukuran model yang relatif ringan, yaitu 14.35 MB, serta waktu inferensi yang rendah, yaitu 5.9 ms. Dibandingkan varian *nano*, model *small* memberikan peningkatan akurasi yang baik. Sementara itu, dibandingkan varian *medium*, *large*, dan *xtra*, peningkatan akurasi yang diperoleh model-model tersebut tidak terlalu besar jika dibandingkan dengan kenaikan ukuran model dan waktu inferensi yang cukup signifikan. Dengan demikian, YOLOv5 varian *small* memberikan keseimbangan terbaik antara akurasi dan efisiensi komputasi, sehingga paling sesuai untuk implementasi *real-time*.

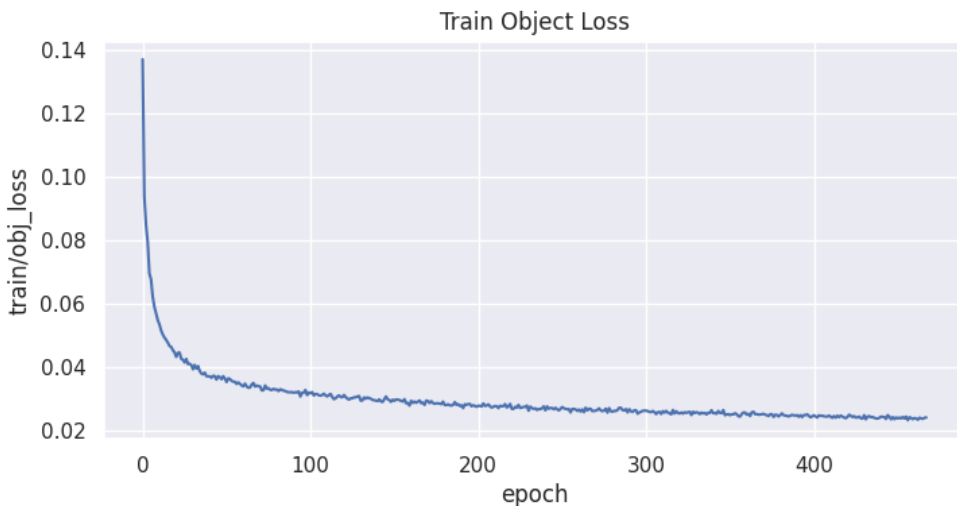
3.2 Hasil Pengujian Konvergensi Model YOLOv5s

Setelah dilakukan analisis komparatif terhadap seluruh varian YOLOv5 pada subbab sebelumnya, model YOLOv5 varian *small* (YOLOv5s) dipilih sebagai model terbaik karena memberikan keseimbangan yang paling baik antara akurasi deteksi dan efisiensi komputasi untuk implementasi *real-time*. Oleh karena itu, pada tahap selanjutnya dilakukan pengujian konvergensi terhadap model YOLOv5s guna memperoleh titik pelatihan yang lebih optimal.



Gambar 53. Grafik metrik *train object loss* YOLOv5s 100 *epoch*

Pengujian konvergensi dilakukan dengan meningkatkan jumlah *epoch* selama *training* dari 100 *epoch* menjadi 1000 *epoch* dengan parameter *patience* sebesar 100. Parameter ini memungkinkan proses *training* dihentikan secara otomatis ketika tidak terjadi peningkatan performa yang signifikan dalam sejumlah *epoch* tertentu. Berdasarkan hasil pelatihan yang diperoleh, proses *training* berhenti secara otomatis pada epoch ke-467, yang menunjukkan bahwa model telah mencapai kondisi konvergen sebelum mencapai batas maksimum *epoch* yang ditentukan.



Gambar 54. Grafik metrik *train object loss* YOLOv5s 1000 *epoch*

Pada Gambar 54 terlihat bahwa nilai *object loss* mengalami penurunan secara bertahap seiring bertambahnya *epoch* yang menunjukkan bahwa model mampu

mempelajari pola pada data pelatihan dengan baik. Dalam arsitektur YOLOv5, *object loss* merupakan fungsi kerugian yang mengukur probabilitas keberadaan suatu objek di dalam *bounding box* yang diprediksi, tanpa memperhitungkan klasifikasi kelas dari objek tersebut. Metrik ini dihitung berbasis *Binary Cross-Entropy* (BCE) untuk mengevaluasi kemampuan model dalam membedakan antara representasi fitur objek target dengan representasi latar belakang (*background*).

Secara operasional, *object loss* bekerja dengan memberikan penalti matematis terhadap prediksi lokalisasi yang keliru. Nilai kerugian ini akan meningkat apabila model memprediksi keberadaan objek pada area latar belakang yang kosong (*false positive*), atau sebaliknya, ketika model gagal mengidentifikasi keberadaan objek pada area yang memuat target (*false negative*). Seiring berjalannya iterasi *epoch*, penurunan nilai metrik ini merepresentasikan kemampuan model yang semakin presisi dalam mengeliminasi *bounding box* yang tidak relevan dan memusatkan prediksi pada area objek yang sebenarnya.

Oleh karena itu, *object loss* dijadikan sebagai parameter krusial untuk menentukan tingkat kematangan model. Penurunan nilai *train object loss* yang drastis pada awal pelatihan (Gambar 53) dan melandai secara asimtotik pada akhir pelatihan (Gambar 54) mengonfirmasi bahwa model telah mengoptimalkan pembobotan jaringannya. Penghentian otomatis pada *epoch* ke-467 menjadi indikator utama bahwa model telah mencapai titik konvergensi, di mana model YOLOv5s telah meminimalkan kesalahan lokalisasi secara maksimal pada data latih. Pada titik ini, model dinilai telah layak dan optimal, karena iterasi pelatihan lebih lanjut tidak akan memberikan perbaikan metrik yang signifikan dan justru berisiko memicu terjadinya *overfitting*.

Selain dievaluasi berdasarkan metrik *loss*, perbandingan performa antara model sebelum dan sesudah konvergensi juga diukur secara menggunakan *confusion matrix*, sebagaimana ditunjukkan pada Tabel 12.

Tabel 12. *Confusion matrix* YOLOv5s 100 *epoch* dan 1000 *epoch*

Model	TP	FP	FN
YOLOv5s, epoch=100	2.430	10	25
YOLOv5s, epoch=1000	2.437	4	18

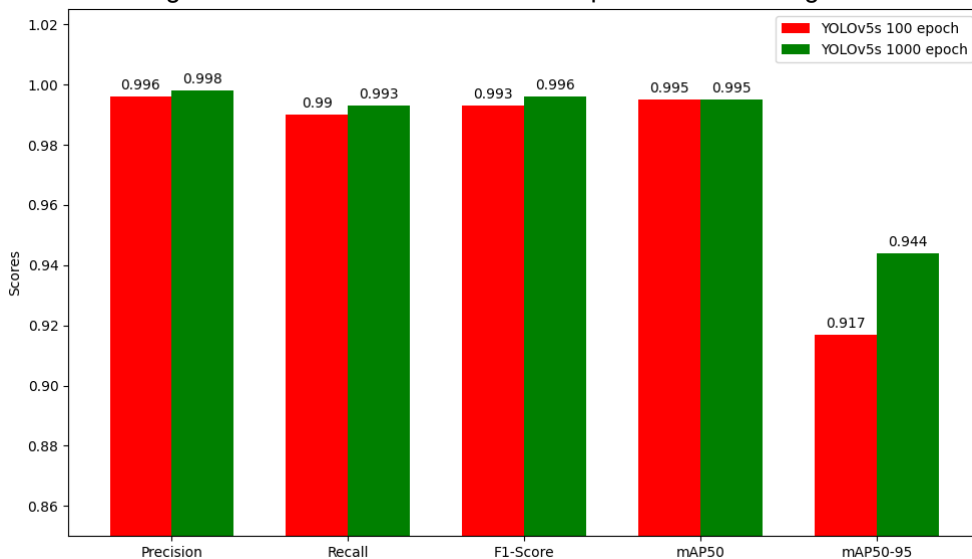


(a)



Gambar 55. Perbandingan hasil deteksi YOLOv5s 100 *epoch* dan YOLOv5s 1000 *epoch*: (a) Sampel *false positive*; (b) Sampel *false negative*.

Berdasarkan Tabel 12, pada konfigurasi 100 *epoch* model menghasilkan 2.430 *true positive* (TP), 10 *false positive* (FP), dan 25 *false negative* (FN). Setelah dilakukan peningkatan jumlah epoch hingga model mencapai kondisi konvergen, performa model mengalami peningkatan dengan menghasilkan 2437 *true positive* (TP), 4 *false positive* (FP), dan 18 *false negative* (FN). Hasil tersebut menunjukkan bahwa peningkatan jumlah *epoch* hingga mencapai kondisi konvergen mampu memperbaiki beberapa kesalahan deteksi yang sebelumnya terjadi pada model dengan 100 *epoch*. Salah satu kasus *false positive* yang sebelumnya muncul akibat area tanah yang dilintasi kabel dan membentuk pola tepi yang menyerupai objek kentang berhasil diatasi oleh model setelah proses *training* yang lebih panjang. Selain itu, beberapa kasus *false negative* yang sebelumnya terjadi karena objek kentang memiliki kontras yang rendah terhadap latar belakang juga berhasil terdeteksi dengan lebih baik setelah model mencapai kondisi konvergen.



Gambar 56. Perbandingan metrik antara YOLOv5s 100 *epoch* dan YOLOv5s 1000

Perbandingan performa model YOLOv5s 100 *epoch* dan 1000 *epoch* juga dapat dilihat pada grafik metrik evaluasi. Berdasarkan grafik tersebut, model YOLOv5s yang dilatih hingga mencapai konvergensi menunjukkan peningkatan pada beberapa metrik utama, yaitu *precision*, *recall*, *F1-score*, serta *mAP50-95*, dibandingkan dengan model yang dilatih selama 100 *epoch*. Peningkatan ini menunjukkan bahwa proses *training* yang lebih panjang memungkinkan model untuk mempelajari representasi fitur objek secara lebih optimal sehingga meningkatkan kemampuan deteksi secara keseluruhan.

Secara keseluruhan, hasil pengujian konvergensi menunjukkan bahwa peningkatan jumlah *epoch* pelatihan hingga model mencapai kondisi konvergen memberikan dampak positif terhadap performa deteksi objek. Model YOLOv5s yang dilatih hingga *epoch* ke-467 menghasilkan tingkat kesalahan deteksi yang lebih rendah serta nilai metrik evaluasi yang lebih tinggi dibandingkan dengan model yang dilatih selama 100 *epoch*. Oleh karena itu, konfigurasi pelatihan ini dipilih sebagai model akhir yang digunakan dalam implementasi sistem secara *real-time*.

3.3 Hasil Pengujian *Preprocessing* dengan *Unsharp Masking*

Tahap pengujian ini bertujuan untuk mengevaluasi secara sistematis pengaruh penerapan *unsharp masking* (UM) sebagai teknik *preprocessing* citra terhadap performa deteksi objek kentang menggunakan model YOLOv5s yang telah mencapai kondisi konvergen pada tahap pelatihan. Evaluasi dilakukan dengan membandingkan performa model pada dua kondisi input, yaitu citra tanpa *unsharp masking* (Tanpa UM) dan citra dengan *unsharp masking* (Dengan UM).

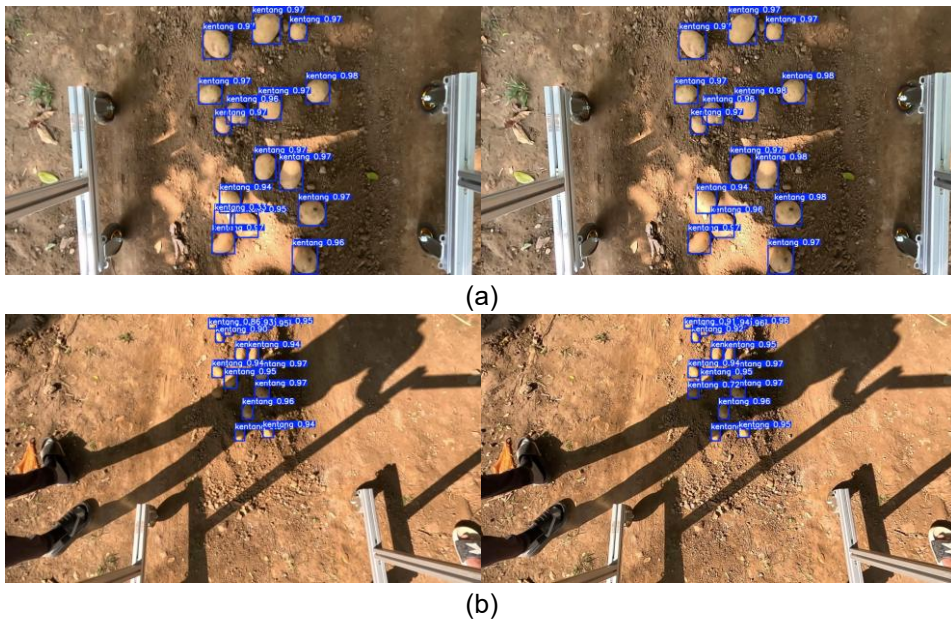
Tabel 13. *Confusion matrix* YOLOv5s tanpa *unsharp masking* dan dengan *unsharp masking*

Model	TP	FP	FN
Tanpa Unsharp Masking	2.437	4	18
Dengan Unsharp Masking	2.448	28	7

Berdasarkan Tabel 13, penerapan *unsharp masking* secara umum mampu meningkatkan jumlah TP dan menekan jumlah FN pada seluruh nilai *threshold* yang diuji. Jumlah TP meningkat dari 2.437 (Tanpa UM) menjadi 2.448 (Dengan UM), sementara FN berkurang dari 18 menjadi 7. Hal ini menunjukkan bahwa peningkatan ketajaman citra yang dihasilkan oleh proses *unsharp masking* mampu memperjelas batas tepi objek kentang sehingga representasi fitur visual objek menjadi lebih mudah dikenali oleh model.

Namun demikian, penerapan teknik ini juga menyebabkan peningkatan jumlah FP. Hal ini disebabkan karena proses penajaman citra tidak hanya memperkuat tepi objek kentang, tetapi juga memperkuat tekstur pada elemen latar belakang seperti tanah, daun kering, maupun objek lain yang memiliki karakteristik visual yang

menyerupai kentang. Akibatnya, beberapa area latar belakang dapat terdeteksi secara keliru oleh model sebagai objek kentang.

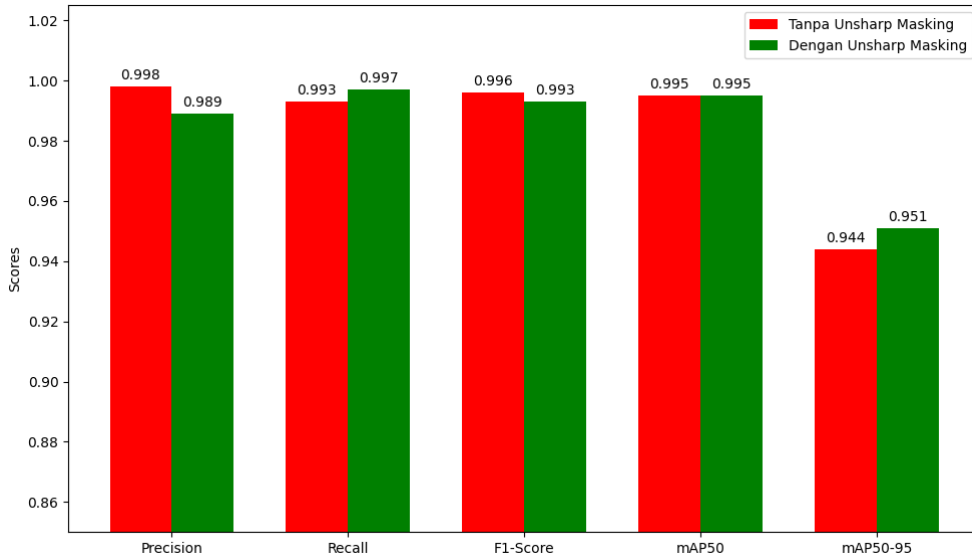


Gambar 57. Perbandingan hasil deteksi YOLOv5s tanpa *unsharp masking* dan dengan *unsharp masking* (a) Sampel *false positive*; (b) Sampel *false negative*.

Pada model tanpa *unsharp masking*, kasus *false negative* umumnya terjadi ketika objek kentang memiliki kontras yang rendah terhadap latar belakang atau sebagian tertutup oleh bayangan sehingga fitur visual objek menjadi kurang jelas. Sementara itu, pada model dengan *unsharp masking*, sebagian kasus *false negative* tersebut berhasil diatasi karena peningkatan ketajaman citra mampu memperkuat perbedaan antara objek dan latar belakang. Namun demikian, peningkatan ketajaman tersebut juga menyebabkan munculnya beberapa *false positive*, terutama pada area latar belakang yang memiliki tekstur atau pola tepi yang menyerupai objek kentang.

Model tanpa UM secara konsisten menunjukkan nilai *precision* yang lebih tinggi pada seluruh *threshold*. Nilai *precision* meningkat dari 0.998 pada *threshold* 0.3 hingga mencapai 1 pada *threshold* 0.5 hingga 0.7. Sebaliknya, model dengan UM memiliki *precision* yang lebih rendah, yakni berkisar antara 0.989 (*threshold* 0.3) hingga 0.998 (*threshold* 0.7), meskipun menunjukkan tren peningkatan yang monoton. Perbedaan ini konsisten dengan temuan pada *confusion matrix*, di mana model dengan UM menghasilkan lebih banyak FP pada *threshold* rendah, sehingga mengurangi nilai *precision*. Hasil ini menunjukkan bahwa tanpa UM, model lebih selektif dalam mengkonfirmasi prediksi objek sebagai kentang, terutama pada kondisi *threshold* yang lebih ketat.

Model dengan UM mempertahankan nilai *recall* yang lebih tinggi dan relatif stabil pada seluruh *threshold*, yaitu dalam rentang 0.995 hingga 0.997. Sementara itu, model tanpa UM menunjukkan penurunan *recall* yang progresif seiring meningkatnya *threshold*, dari 0.993 pada *threshold* 0.3 menjadi 0.978 pada *threshold* 0.7. Penurunan *recall* ini mencerminkan meningkatnya jumlah FN ketika kriteria deteksi diperketat tanpa didukung oleh penguatan fitur visual. Sebaliknya, peningkatan kualitas fitur tepi melalui UM menjadikan model lebih *robust* terhadap perubahan *threshold*, sehingga kemampuan deteksi objek yang sebenarnya (*true detection rate*) tetap terjaga secara konsisten.

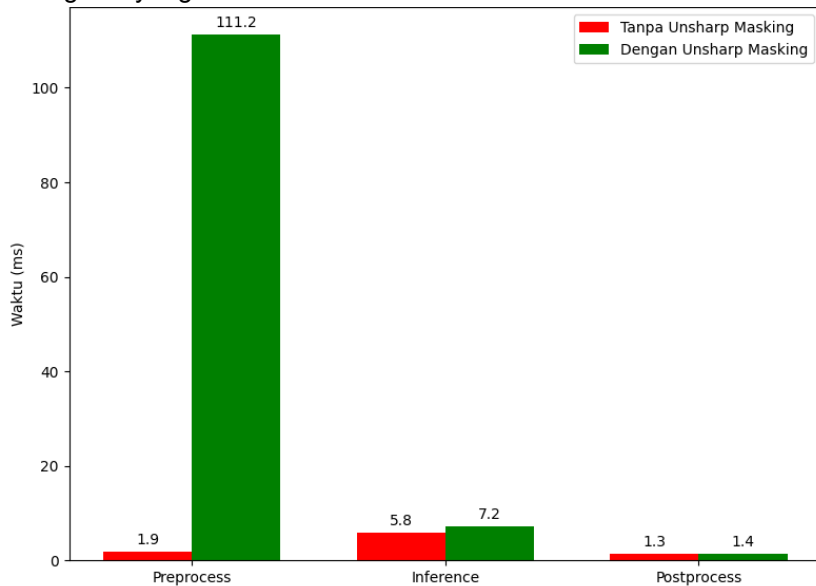


Gambar 58. Perbandingan metrik antara YOLOv5s tanpa *unsharp masking* dan dengan *unsharp masking*

Pada grafik F1-score, model dengan *unsharp masking* juga menunjukkan performa yang lebih baik pada hampir seluruh *threshold*. Pada *threshold* 0.3, nilai F1-score model tanpa *unsharp masking* sedikit lebih tinggi, namun mulai *threshold* 0.4 hingga 0.7 model dengan *unsharp masking* unggul secara konsisten. Nilai F1-score model dengan *unsharp masking* meningkat dari sekitar 0.993 menjadi 0.997, sedangkan model tanpa *unsharp masking* justru menurun dari sekitar 0.997 menjadi 0.989. Hasil ini menunjukkan bahwa *unsharp masking* memberikan keseimbangan yang lebih baik antara *precision* dan *recall*, khususnya pada *threshold* menengah hingga tinggi.

Pada grafik mAP50, model dengan *unsharp masking* mempertahankan nilai yang stabil pada sekitar 0.995 untuk seluruh *threshold*. Sebaliknya, model tanpa *unsharp masking* menunjukkan tren penurunan bertahap dari sekitar 0.995 pada *threshold* 0.3 menjadi sekitar 0.989 pada *threshold* 0.7. Hal ini menunjukkan bahwa penerapan *unsharp masking* membantu menjaga konsistensi performa deteksi pada berbagai tingkat *threshold*.

Hal yang sama juga terlihat pada grafik mAP50–95. Model dengan *unsharp masking* menunjukkan nilai yang relatif stabil pada kisaran 0.951, sedangkan model tanpa *unsharp masking* mengalami penurunan bertahap dari sekitar 0.944 pada *threshold* 0.3 menjadi sekitar 0.939 pada *threshold* 0.7. Hasil ini menunjukkan bahwa *unsharp masking* memberikan keuntungan dalam kualitas deteksi yang dievaluasi pada rentang IoU yang lebih ketat.



Gambar 59. Perbandingan waktu pemrosesan antara YOLOv5s tanpa *unsharp masking* dan dengan *unsharp masking*

Berdasarkan grafik tersebut, terlihat bahwa penerapan *unsharp masking* memberikan dampak yang cukup signifikan pada tahap *preprocessing*. Pada model tanpa *unsharp masking*, waktu *preprocessing* tercatat sekitar 1.9 ms, sedangkan pada model dengan *unsharp masking* meningkat menjadi sekitar 111.2 ms. Peningkatan ini merupakan konsekuensi langsung dari operasi komputasi yang diperlukan dalam algoritma *unsharp masking*, yang melibatkan konvolusi *Gaussian blur* untuk menghasilkan citra terhalus, diikuti oleh operasi pengurangan dan penguatan komponen frekuensi tinggi secara piksel per piksel. Kompleksitas komputasi dari operasi konvolusi 2D ini bersifat $O(N \cdot k^2)$, di mana N adalah jumlah piksel dan k adalah ukuran kernel, sehingga pada resolusi citra yang tinggi, waktu komputasinya menjadi signifikan.

Pada tahap *inference*, waktu pemrosesan relatif tidak mengalami perubahan yang signifikan. Model tanpa *unsharp masking* memerlukan waktu sekitar 5.8 ms, sedangkan model dengan *unsharp masking* memerlukan waktu sekitar 7.2 ms. Selisih waktu yang relatif kecil ini menunjukkan bahwa proses deteksi objek oleh model YOLOv5s tidak dipengaruhi secara langsung oleh penerapan *preprocessing* tersebut.

Hal yang sama juga terlihat pada tahap *postprocessing*, di mana waktu pemrosesan hanya mengalami perbedaan yang sangat kecil. Model tanpa *unsharp masking* memerlukan waktu sekitar 1.3 ms, sedangkan model dengan *unsharp masking* memerlukan waktu sekitar 1.4 ms. Perbedaan ini dapat dianggap tidak signifikan terhadap keseluruhan proses inferensi model yang mana tahap *postprocessing* meliputi operasi *Non-Maximum Suppression* (NMS) dan konversi format keluaran yang lebih dipengaruhi oleh jumlah *bounding box* kandidat dibandingkan karakteristik piksel pada citra input.

Secara keseluruhan, penerapan *unsharp masking* meningkatkan waktu pemrosesan terutama pada tahap *preprocessing* sehingga total waktu pemrosesan per citra menjadi lebih besar dibandingkan model tanpa *preprocessing*. Meskipun demikian, peningkatan waktu komputasi ini memberikan keuntungan dalam peningkatan kemampuan deteksi objek, khususnya dalam mengurangi jumlah *false negative* pada kondisi citra dengan kontras rendah. Oleh karena itu, penggunaan *unsharp masking* dalam sistem yang dikembangkan perlu dipertimbangkan sebagai mekanisme peningkatan kualitas citra dengan mempertimbangkan *trade-off* antara peningkatan performa deteksi dan tambahan beban komputasi yang dihasilkan.

3.4 Hasil Pengujian Counting Model YOLOv5s

Pengujian *counting* model YOLOv5s dilakukan pada 56 skenario melalui kombinasi parameter ketinggian (h), sudut (θ), dan kecepatan (v) menggunakan metrik MAE dan RMSE, di mana nilai MAE dan RMSE yang lebih kecil menunjukkan performa *counting* yang lebih baik. Pengujian dilaksanakan dengan membentuk guludan kecil sepanjang 2 m di atas permukaan tanah menggunakan tanah perkebunan kentang, kemudian kentang disebar di atas guludan dengan posisi yang sama pada setiap skenario. Karena pengujian dilakukan di lingkungan lapangan, kondisi pencahayaan tidak selalu identik pada setiap pengambilan video. Variasi intensitas cahaya dan kemunculan bayangan dapat memengaruhi ketajaman, kontras, serta visibilitas objek pada citra. Perubahan kondisi visual tersebut berpotensi memengaruhi proses deteksi objek dan pada akhirnya berdampak pada nilai MAE dan RMSE yang dihasilkan.

3.4.1 Skenario Ketinggian

1. Ketinggian 80 cm

Tabel 14. Hasil pengujian model YOLOv5s pada ketinggian 80 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	28	9.750	9.798
Video 2	36	27		
Video 3	36	27		

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 4	36	25		
Video 5	36	26		
Video 6	36	26		
Video 7	36	26		
Video 8	36	25		

Berdasarkan Tabel 14, skenario ketinggian 80 cm menunjukkan *undercount* pada seluruh video, dengan jumlah prediksi kentang berada pada rentang 25 hingga 28 dari total 36 objek aktual. Nilai MAE sebesar 9.750 dan RMSE sebesar 9.798 menunjukkan deviasi yang cukup besar antara jumlah prediksi dan jumlah aktual. Kondisi ini menunjukkan bahwa konfigurasi ketinggian 80 cm belum mampu mendukung proses *counting* secara stabil. Salah satu penyebab utama kondisi ini adalah getaran atau guncangan alat selama proses pengambilan video. Pada ketinggian yang relatif rendah, bidang pandang kamera menjadi lebih sempit sehingga objek kentang muncul dalam waktu yang lebih singkat pada *frame*. Ketika guncangan terjadi, posisi objek pada citra dapat berubah secara tiba-tiba sehingga proses pelacakan objek menjadi kurang stabil dan sebagian objek tidak terhitung secara konsisten.

2. Ketinggian 90 cm

Tabel 15. Hasil pengujian model YOLOv5s pada ketinggian 90 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	34		
Video 2	36	33		
Video 3	36	32		
Video 4	36	28		
Video 5	36	32	5.750	6.305
Video 6	36	28		
Video 7	36	28		
Video 8	36	27		

Berdasarkan Tabel 15, skenario ketinggian 90 cm menunjukkan penurunan tingkat kesalahan dibandingkan ketinggian 80 cm, dengan jumlah prediksi berada pada rentang 27 hingga 34 dari total 36 objek aktual. Nilai MAE sebesar 5.750 dan RMSE sebesar 6.305 menunjukkan bahwa deviasi antara jumlah prediksi dan jumlah aktual menjadi lebih kecil. Meskipun demikian, kecenderungan *undercount* masih terlihat pada beberapa video dengan jumlah prediksi turun hingga 27–28 objek. Perbaikan performa pada ketinggian ini kemungkinan disebabkan oleh cakupan bidang pandang yang lebih luas, sehingga objek kentang dapat diamati lebih lama sebelum keluar dari *frame*. Kondisi ini membantu sistem mempertahankan konsistensi deteksi meskipun masih terdapat gangguan akibat getaran alat.

3. Ketinggian 100 cm

Tabel 16. Hasil pengujian model YOLOv5s pada ketinggian 100 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	31	8.125	8.667
Video 2	36	31		
Video 3	36	30		
Video 4	36	31		
Video 5	36	26		
Video 6	36	26		
Video 7	36	23		
Video 8	36	25		

Berdasarkan Tabel 16, skenario ketinggian 100 cm menunjukkan peningkatan eror dibandingkan ketinggian 90 cm, dengan jumlah prediksi berada pada rentang 23 hingga 31 dari total 36 objek aktual. Nilai MAE sebesar 8.125 dan RMSE sebesar 8.667 menunjukkan bahwa hasil *counting* pada skenario ini belum stabil. Variasi prediksi yang lebih lebar mengindikasikan bahwa beberapa video mengalami *undercount* yang cukup besar. Peningkatan eror ini kemungkinan dipengaruhi oleh kombinasi antara getaran alat dan perubahan posisi objek pada citra, yang menyebabkan sebagian objek tidak terdeteksi secara konsisten pada beberapa *frame*.

4. Ketinggian 110 cm

Tabel 17. Hasil pengujian model YOLOv5s pada ketinggian 110 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	34	2.875	3.142
Video 2	36	36		
Video 3	36	32		
Video 4	36	32		
Video 5	36	33		
Video 6	36	33		
Video 7	36	32		
Video 8	36	27		

Berdasarkan Tabel 17, skenario ketinggian 110 cm menunjukkan performa *counting* yang jauh lebih baik dibandingkan skenario sebelumnya. Jumlah prediksi berada pada rentang 27 hingga 36 objek dari total 36 objek aktual. Nilai MAE sebesar 2.875 dan RMSE sebesar 3.142 menunjukkan deviasi yang relatif kecil. Pada salah satu video bahkan diperoleh jumlah prediksi yang sama dengan jumlah aktual. Hasil ini menunjukkan bahwa pada ketinggian 110 cm, sistem mampu melakukan deteksi dan penghitungan objek dengan lebih stabil meskipun masih terdapat getaran alat. Pada ketinggian ini, bidang pandang kamera dan ukuran objek pada citra berada pada kondisi yang lebih seimbang sehingga proses deteksi dan *tracking* dapat berjalan lebih konsisten.

5. Ketinggian 120 cm

Tabel 18. Hasil pengujian model YOLOv5s pada ketinggian 120 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	34	2.750	3.464
Video 2	36	33		
Video 3	36	35		
Video 4	36	31		
Video 5	36	36		
Video 6	36	34		
Video 7	36	34		

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 8	36	29		

Berdasarkan Tabel 18, skenario ketinggian 120 cm menghasilkan performa rata-rata terbaik dengan jumlah prediksi berada pada rentang 29 hingga 36 objek dari total 36 objek aktual. Nilai MAE sebesar 2.750 merupakan yang terendah pada seluruh variasi ketinggian, sedangkan RMSE sebesar 3.464 masih menunjukkan deviasi yang relatif kecil. Pada skenario ini juga terdapat satu video dengan jumlah prediksi yang sama dengan jumlah aktual. Hasil ini menunjukkan bahwa pada ketinggian 120 cm, sistem mampu mempertahankan kestabilan deteksi meskipun terdapat gangguan akibat getaran alat. Kombinasi antara cakupan bidang pandang yang cukup luas dan ukuran objek yang masih cukup jelas pada citra membantu sistem melakukan proses *counting* secara lebih akurat.

6. Ketinggian 130 cm

Tabel 19. Hasil pengujian model YOLOv5s pada ketinggian 130 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	30	7.500	7.842
Video 2	36	28		
Video 3	36	27		
Video 4	36	25		
Video 5	36	31		
Video 6	36	29		
Video 7	36	26		
Video 8	36	32		

Berdasarkan Tabel 19, skenario ketinggian 130 cm menunjukkan penurunan performa kembali dengan jumlah prediksi berada pada rentang 25 hingga 32 objek dari total 36 objek aktual. Nilai MAE sebesar 7.500 dan RMSE sebesar 7.842 menunjukkan bahwa deviasi antara jumlah prediksi dan jumlah aktual meningkat. Kondisi ini menunjukkan bahwa konfigurasi ketinggian 130 cm tidak memberikan kestabilan yang baik pada proses *counting*. Pada ketinggian yang lebih tinggi, ukuran objek pada citra menjadi lebih kecil sehingga sistem lebih sensitif terhadap perubahan posisi akibat

getaran alat, yang dapat menyebabkan sebagian objek tidak terdeteksi secara konsisten.

7. Ketinggian 140 cm

Tabel 20. Hasil pengujian model YOLOv5s pada ketinggian 140 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	32	8.750	9.327
Video 2	36	31		
Video 3	36	30		
Video 4	36	23		
Video 5	36	27		
Video 6	36	26		
Video 7	36	26		
Video 8	36	23		

Berdasarkan Tabel 20, skenario ketinggian 140 cm menunjukkan performa terendah pada variasi ketinggian dengan jumlah prediksi berada pada rentang 23 hingga 32 objek dari total 36 objek aktual. Nilai MAE sebesar 8.750 dan RMSE sebesar 9.327 merupakan yang tertinggi pada skenario ketinggian ini. Hal tersebut menunjukkan bahwa deviasi antara jumlah prediksi dan jumlah aktual cukup besar dan terjadi secara konsisten pada beberapa video. Pada ketinggian ini, ukuran objek dalam citra menjadi lebih kecil sehingga proses deteksi menjadi lebih sensitif terhadap gangguan visual maupun getaran alat. Akibatnya, sebagian objek tidak terdeteksi atau tidak terhitung secara konsisten.

Tabel 21. Perbandingan skenario ketinggian pada model YOLOv5s

Skenario Ketinggian (cm)	MAE	RMSE
80	9.750	9.798
90	5.750	6.305
100	8.125	8.667
110	2.875	3.142
120	2.750	3.464
130	7.500	7.842

Skenario Ketinggian (cm)	MAE	RMSE
140	8.750	9.327

Berdasarkan Tabel 21, nilai MAE dan RMSE menunjukkan kecenderungan yang bergerak searah dan membentuk pola non-linear terhadap perubahan ketinggian kamera. Nilai eror tertinggi terjadi pada ketinggian 80 cm dan kembali meningkat pada 130 cm dan 140 cm, sedangkan performa terbaik berada pada rentang 110 cm hingga 120 cm. Pada ketinggian 110 cm, sistem menghasilkan RMSE terendah sebesar 3.142 dengan MAE sebesar 2.875, sedangkan pada 120 cm sistem menghasilkan MAE terendah sebesar 2.750 dengan RMSE sebesar 3.464. Dengan demikian, rentang ketinggian 110–120 cm dapat dianggap sebagai konfigurasi yang paling stabil dan paling akurat untuk proses *counting* pada pengujian ini. Rentang ini memberikan keseimbangan yang baik antara ukuran objek pada citra, cakupan bidang pandang, serta ketahanan sistem terhadap gangguan getaran alat selama proses pengambilan video.

3.4.2 Skenario Sudut

1. Sudut 80°

Tabel 22. Hasil pengujian model YOLOv5s pada sudut 80°

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	28	5.643	6.475
Video 2	36	27		
Video 3	36	27		
Video 4	36	25		
Video 5	36	34		
Video 6	36	33		
Video 7	36	32		
Video 8	36	28		
Video 9	36	31		
Video 10	36	31		
Video 11	36	30		
Video 12	36	31		
Video 13	36	34		

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 14	36	36		
Video 15	36	32		
Video 16	36	32		
Video 17	36	34		
Video 18	36	33		
Video 19	36	35		
Video 20	36	31		
Video 22	36	30		
Video 22	36	28		
Video 23	36	27		
Video 24	36	25		
Video 25	36	32		
Video 26	36	31		
Video 27	36	30		
Video 28	36	23		

Berdasarkan Tabel 22, evaluasi sistem pada skenario sudut pengambilan gambar 80° menunjukkan hasil *counting* yang bervariasi, dengan jumlah prediksi berada pada rentang 23 hingga 36 kentang dari total 36 objek aktual pada 28 sampel video. Pada skenario ini terdapat satu kasus prediksi yang sesuai dengan jumlah aktual, yaitu pada Video 14, sedangkan beberapa video lainnya menghasilkan jumlah prediksi yang mendekati nilai aktual. Kinerja sistem pada sudut ini ditunjukkan oleh MAE sebesar 5.643 dan RMSE sebesar 6.475. Performa pada sudut 80° cenderung lebih baik karena sudut pengambilan gambar tidak sepenuhnya tegak lurus terhadap permukaan tanah. Kondisi ini membuat bentuk, batas, dan fitur visual kentang lebih mudah dibedakan, sementara objek yang berdekatan tidak terlalu saling menutupi dalam bidang pandang. Selain itu, pada sudut ini pengaruh getaran alat terhadap perubahan posisi objek di dalam citra masih relatif dapat ditoleransi, sehingga proses deteksi dan *counting* dapat berlangsung lebih konsisten.

2. Sudut 90°

Tabel 23. Hasil pengujian model YOLOv5s pada sudut 90°

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	26	7.357	8.185
Video 2	36	26		
Video 3	36	26		
Video 4	36	25		
Video 5	36	32		
Video 6	36	28		
Video 7	36	28		
Video 8	36	27		
Video 9	36	26		
Video 10	36	26		
Video 11	36	23		
Video 12	36	25		
Video 13	36	33		
Video 14	36	33		
Video 15	36	32		
Video 16	36	33		
Video 17	36	36		
Video 18	36	34		
Video 19	36	34		
Video 20	36	29		
Video 22	36	31		
Video 22	36	29		
Video 23	36	26		
Video 24	36	32		
Video 25	36	27		

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 26	36	26		
Video 27	36	26		
Video 28	36	23		

Berdasarkan Tabel 23, evaluasi sistem pada skenario sudut 90° menunjukkan peningkatan tingkat kesalahan dibandingkan skenario sudut 80°, dengan jumlah prediksi kentang berada pada rentang 23 hingga 36 dari total 36 objek aktual. Meskipun terdapat satu kasus prediksi yang sesuai dengan jumlah aktual, yaitu pada Video 17, secara umum lebih banyak sampel menghasilkan prediksi di bawah 30 kentang sehingga *undercount* lebih sering terjadi. Hal ini tercermin pada nilai MAE sebesar 7.357 dan RMSE sebesar 8.185. Penurunan performa pada sudut 90° terjadi karena sudut pandang tegak lurus dari atas membuat objek lebih sering tampak saling menutupi, sehingga batas antarobjek yang berhimpitan menjadi kurang jelas. Kondisi ini membuat sistem lebih sulit membedakan objek secara konsisten. Selain itu, ketika getaran alat terjadi, perubahan posisi objek di dalam *frame* menjadi lebih mudah mengganggu kontinuitas deteksi dan *tracking*, sehingga sebagian objek terlewat dan tidak terhitung.

Tabel 24. Perbandingan skenario sudut pada model YOLOv5s

Skenario Sudut (°)	MAE	RMSE
80	5.643	6.475
90	7.357	8.185

Berdasarkan Tabel 24, sudut 80° memberikan performa *counting* yang lebih baik dibandingkan sudut 90°. Pada sudut 80°, sistem menghasilkan MAE sebesar 5.643 dan RMSE sebesar 6.475, sedangkan pada sudut 90° nilai eror meningkat menjadi MAE sebesar 7.357 dan RMSE sebesar 8.185. Kenaikan kedua metrik pada sudut 90° menunjukkan bahwa deviasi jumlah prediksi terhadap jumlah aktual menjadi lebih besar. Dengan demikian, sudut 80° lebih sesuai digunakan sebagai konfigurasi pengambilan citra karena menghasilkan eror yang lebih rendah dan hasil *counting* yang lebih mendekati jumlah aktual. Hasil ini juga menunjukkan bahwa sudut pengambilan yang sedikit miring lebih membantu sistem mempertahankan kestabilan deteksi dan penghitungan ketika alat mengalami getaran atau guncangan selama pengujian.

3.4.3 Skenario Kecepatan

1. Kecepatan 0.25 m/s

Tabel 25. Hasil pengujian model YOLOv5s pada kecepatan 0.25 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	28	5.000	5.880
Video 2	36	34		
Video 3	36	31		
Video 4	36	34		
Video 5	36	34		
Video 6	36	30		
Video 7	36	32		
Video 8	36	26		
Video 9	36	32		
Video 10	36	26		
Video 11	36	33		
Video 12	36	36		
Video 13	36	31		
Video 14	36	27		

Berdasarkan Tabel 25, evaluasi sistem pada skenario kecepatan 0.25 m/s menunjukkan hasil *counting* terbaik pada variasi kecepatan, dengan jumlah prediksi kentang berada pada rentang 26 hingga 36 dari total 36 objek aktual pada 14 sampel video. Pada skenario ini terdapat satu kasus jumlah prediksi yang sesuai dengan jumlah aktual, yaitu pada Video 12, sedangkan sebagian besar video lainnya menghasilkan jumlah prediksi yang mendekati jumlah aktual. Kinerja sistem pada kecepatan ini ditunjukkan oleh MAE sebesar 5.000 dan RMSE sebesar 5.880. Performa yang lebih stabil pada kecepatan rendah terjadi karena pergerakan alat relatif lambat, sehingga efek getaran atau guncangan yang diterima kamera juga lebih kecil. Akibatnya, citra yang ditangkap lebih stabil dan perpindahan posisi objek antarframe tidak terlalu besar. Kondisi ini membantu proses deteksi, *tracking*, dan *counting* berjalan lebih konsisten.

2. Kecepatan 0.5 m/s

Tabel 26. Hasil pengujian model YOLOv5s pada kecepatan 0.5 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	27	5.929	6.756
Video 2	36	33		
Video 3	36	31		
Video 4	36	36		
Video 5	36	33		
Video 6	36	28		
Video 7	36	31		
Video 8	36	26		
Video 9	36	28		
Video 10	36	26		
Video 11	36	33		
Video 12	36	34		
Video 13	36	29		
Video 14	36	26		

Berdasarkan Tabel 26, evaluasi sistem pada kecepatan 0.5 m/s menunjukkan peningkatan eror dibandingkan kecepatan 0.25 m/s, dengan jumlah prediksi kentang berada pada rentang 26 hingga 36 dari total 36 objek aktual. Terdapat satu kasus prediksi yang sesuai dengan jumlah aktual, yaitu pada Video 4, namun pada banyak sampel hasil prediksi masih berada di bawah jumlah aktual. Kinerja pada skenario ini ditunjukkan oleh MAE sebesar 5.929 dan RMSE sebesar 6.756. Penurunan performa pada kecepatan ini terjadi karena laju gerak alat yang lebih tinggi mulai memperbesar dampak getaran alat terhadap kestabilan citra. Akibatnya, perpindahan posisi objek antarframe menjadi lebih besar dan proses penghitungan menjadi lebih rentan terhadap objek yang terlewat.

3. Kecepatan 0.75 m/s

Tabel 27. Hasil pengujian model YOLOv5s pada kecepatan 0.75 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	27	6.857	7.653
Video 2	36	32		
Video 3	36	30		
Video 4	36	32		
Video 5	36	35		
Video 6	36	27		
Video 7	36	30		
Video 8	36	26		
Video 9	36	28		
Video 10	36	23		
Video 11	36	32		
Video 12	36	34		
Video 13	36	26		
Video 14	36	26		

Berdasarkan Tabel 27, evaluasi sistem pada kecepatan 0.75 m/s menunjukkan penurunan performa yang lebih jelas, dengan jumlah prediksi kentang berada pada rentang 23 hingga 35 dari total 36 objek aktual. Pada beberapa video, jumlah prediksi turun hingga kisaran 23 hingga 26 kentang, yang menunjukkan bahwa *undercount* semakin sering terjadi. Kondisi ini tercermin pada MAE sebesar 6.857 dan RMSE sebesar 7.653. Pada kecepatan ini, pengaruh getaran atau guncangan alat menjadi semakin kuat, sehingga posisi objek pada citra berubah lebih cepat antarframe. Perubahan ini dapat mengganggu kestabilan deteksi dan *tracking*, sehingga sebagian objek tidak berhasil dihitung secara konsisten.

4. Kecepatan 1 m/s

Tabel 28. Hasil pengujian model YOLOv5s pada kecepatan 1 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	36	25	8.214	8.892
Video 2	36	28		
Video 3	36	31		
Video 4	36	32		
Video 5	36	31		
Video 6	36	25		
Video 7	36	23		
Video 8	36	25		
Video 9	36	27		
Video 10	36	25		
Video 11	36	33		
Video 12	36	29		
Video 13	36	32		
Video 14	36	23		

Berdasarkan Tabel 28, evaluasi sistem pada kecepatan 1 m/s menghasilkan tingkat kesalahan terbesar, dengan jumlah prediksi kentang berada pada rentang 23 hingga 33 dari total 36 objek aktual. Pada skenario ini, *undercount* terjadi lebih dominan, dan beberapa video menghasilkan jumlah prediksi yang rendah, yaitu pada kisaran 23 hingga 25 kentang. Kondisi ini tercermin pada MAE sebesar 8.214 dan RMSE sebesar 8.892. Pada laju ini, getaran alat menjadi semakin dominan sehingga kestabilan citra menurun. Perubahan posisi objek antarframe menjadi lebih besar, sementara sistem harus melakukan deteksi dan penghitungan dalam kondisi visual yang kurang stabil. Akibatnya, sebagian objek lebih mudah terlewat dan jumlah prediksi menjadi semakin jauh dari jumlah aktual.

Tabel 29. Perbandingan skenario kecepatan pada model YOLOv5s

Skenario Kecepatan (m/s)	MAE	RMSE
0.25	5,000	5,880
0.5	5,929	6,756
0.75	6,857	7,653
1	8,214	8,892

Berdasarkan Tabel 29, nilai MAE dan RMSE menunjukkan tren peningkatan eror seiring bertambahnya kecepatan alat. Nilai MAE meningkat dari 5.000 pada kecepatan 0.25 m/s menjadi 8.214 pada kecepatan 1 m/s, sedangkan RMSE meningkat dari 5.880 menjadi 8.892. Hasil ini menunjukkan bahwa peningkatan kecepatan berpengaruh langsung terhadap penurunan kemampuan *counting*. Semakin tinggi kecepatan alat, semakin besar pengaruh getaran atau guncangan terhadap kestabilan citra dan kontinuitas pelacakan objek. Akibatnya, sebagian objek lebih mudah terlewat sehingga *undercount* menjadi lebih sering terjadi. Dengan demikian, kecepatan 0.25 m/s merupakan kondisi yang paling baik pada pengujian ini karena menghasilkan eror paling rendah dan hasil *counting* yang paling mendekati jumlah aktual.

Secara keseluruhan, hasil pengujian menunjukkan bahwa performa counting model YOLOv5s dipengaruhi oleh kombinasi parameter ketinggian, sudut, dan kecepatan. Pada skenario ketinggian, performa terbaik diperoleh pada rentang 110–120 cm, yang menunjukkan bahwa sistem memerlukan kompromi antara luas *field of view* dan ukuran objek dalam citra. Pada ketinggian yang terlalu rendah, bidang pandang menjadi sempit sehingga waktu kemunculan objek di dalam *frame* lebih singkat. Sebaliknya, pada ketinggian yang terlalu tinggi, ukuran objek dalam citra menjadi lebih kecil sehingga proses deteksi dan *tracking* menjadi kurang stabil. Pada skenario sudut, 80° memberikan hasil yang lebih baik dibandingkan 90° karena objek tampak lebih terpisah dan batas antarobjek lebih mudah dikenali. Sementara itu, pada skenario kecepatan, performa terbaik diperoleh pada 0.25 m/s, dan eror meningkat seiring bertambahnya kecepatan alat. Meskipun ketiga parameter tersebut berpengaruh terhadap hasil *counting*, faktor yang paling dominan dalam menurunkan performa sistem adalah getaran atau guncangan alat, karena gangguan ini menyebabkan perpindahan posisi objek antarframe, mengganggu kontinuitas *tracking*, dan meningkatkan kemungkinan objek tidak terhitung saat melewati *counting line*.

3.5 Hasil Pengujian Counting Model YOLOv5s dengan Unsharp Masking

Pengujian *counting* model YOLOv5s dengan *preprocessing unsharp masking* dilakukan terhadap 56 skenario yang dibentuk melalui kombinasi parameter ketinggian (h), sudut (θ), dan kecepatan (v). Setiap skenario dijalankan pada guludan

kecil sepanjang 2 m yang dibentuk dari tanah perkebunan kentang, dengan kentang yang disebar di atasnya dalam posisi yang seragam di setiap percobaan. Karena pengujian berlangsung di lingkungan lapangan terbuka, kondisi pencahayaan tidak dapat dikendalikan secara penuh sehingga berpotensi berbeda antar skenario. Variasi intensitas cahaya maupun kemunculan bayangan dapat memengaruhi ketajaman, kontras, dan visibilitas objek dalam citra, yang pada akhirnya berimplikasi pada akurasi deteksi objek dan nilai MAE serta RMSE yang dihasilkan.

3.5.1 Skenario Ketinggian

1. Ketinggian 80 cm

Tabel 30. Hasil pengujian YOLOv5s dengan *unsharp masking* pada ketinggian 80 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19	5.250	7.778
Video 2	19	20		
Video 3	19	14		
Video 4	19	4		
Video 5	19	19		
Video 6	19	19		
Video 7	19	11		
Video 8	19	6		

Berdasarkan Tabel 30, hasil *counting* pada ketinggian 80 cm menunjukkan variasi prediksi yang cukup tinggi, dengan jumlah prediksi per video berkisar antara 4 hingga 20. Meskipun beberapa video, yaitu Video 1, Video 5, dan Video 6, menghasilkan prediksi yang sesuai dengan jumlah aktual, terdapat beberapa video dengan *undercount* yang besar, terutama Video 4 dengan prediksi 4 dan Video 8 dengan prediksi 6. Kondisi ini menghasilkan nilai MAE sebesar 5.250 dan RMSE sebesar 7.778. Nilai RMSE yang lebih tinggi dibandingkan MAE menunjukkan bahwa terdapat beberapa kesalahan dengan besaran yang besar yang memberi pengaruh kuat terhadap rata-rata error.

Pada ketinggian ini, kamera memiliki *field of view* yang relatif sempit sehingga area pengamatan per *frame* menjadi lebih terbatas. Akibatnya, objek kentang hanya muncul dalam waktu yang singkat sebelum melewati *counting line*. Ketika alat bergerak dan menimbulkan guncangan, objek yang sedang dilacak dapat kehilangan ID akibat perubahan posisi yang mendadak. Jika objek kembali terdeteksi setelah kehilangan ID, posisinya

sering kali sudah berada di bawah *counting line*, sehingga tidak memenuhi logika penghitungan dan akhirnya tidak tercatat. Kondisi ini semakin diperburuk oleh tidak adanya *stabilizer* pada kamera, sehingga getaran mekanis alat langsung memengaruhi citra yang diperoleh

2. Ketinggian 90 cm

Tabel 31. Hasil pengujian YOLOv5s dengan *unsharp masking* pada ketinggian 90 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19	2.125	3.142
Video 2	19	18		
Video 3	19	13		
Video 4	19	14		
Video 5	19	19		
Video 6	19	20		
Video 7	19	19		
Video 8	19	15		

Berdasarkan Tabel 31, ketinggian 90 cm menghasilkan performa terbaik dibandingkan seluruh skenario ketinggian yang diuji, dengan MAE sebesar 2.125 dan RMSE sebesar 3.142. Jumlah prediksi per video berada pada rentang 13 hingga 20, dan sebagian besar video menghasilkan prediksi yang mendekati atau sama dengan jumlah aktual.

Pada ketinggian ini, *field of view* kamera memberikan cakupan area yang lebih seimbang. Objek kentang masih memiliki ukuran visual yang cukup jelas, sekaligus berada dalam bidang pandang cukup lama sebelum mencapai *counting line*. Kondisi ini memberi waktu yang lebih memadai bagi sistem *tracking* untuk mempertahankan konsistensi ID objek, meskipun terjadi guncangan ringan. Jika objek sempat kehilangan ID, objek masih berpeluang terdeteksi kembali sebelum melewati garis hitung. Dengan demikian, ketinggian 90 cm dapat dianggap sebagai titik keseimbangan yang baik antara kualitas deteksi, durasi pelacakan, dan keandalan proses *counting*.

3. Ketinggian 100 cm

Tabel 32. Hasil pengujian YOLOv5s dengan *unsharp masking* pada ketinggian 100 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19	5.000	6.982
Video 2	19	27		
Video 3	19	15		
Video 4	19	9		
Video 5	19	19		
Video 6	19	18		
Video 7	19	6		
Video 8	19	9		

Berdasarkan Tabel 32, ketinggian 100 cm menunjukkan penurunan performa dibandingkan ketinggian 90 cm, dengan MAE sebesar 5.000 dan RMSE sebesar 6.982. Jumlah prediksi per video sangat bervariasi, yaitu antara 6 hingga 27, dan pada Video 2 bahkan terjadi *overcount* melebihi jumlah aktual.

Peningkatan ketinggian menyebabkan *field of view* menjadi lebih luas, tetapi ukuran objek dalam piksel menjadi lebih kecil. Kondisi ini membuat representasi visual objek kentang menjadi kurang kuat, sehingga model dan sistem *tracking* lebih rentan terhadap gangguan dari latar belakang maupun guncangan alat. Saat terjadi getaran, posisi objek dapat bergeser cukup besar antarframe sehingga asosiasi ID menjadi tidak stabil. Dalam kondisi tertentu, objek yang terdeteksi ulang masih berada di atas *counting line*, tetapi pada kondisi lain objek sudah melewati garis tersebut. Ketidakstabilan ini menyebabkan eror yang lebih besar sehingga MAE dan RMSE meningkat pada ketinggian ini.

4. Ketinggian 110 cm

Tabel 33. Hasil pengujian YOLOv5s dengan *unsharp masking* pada ketinggian 110 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	20	5.125	7.185

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 2	19	18		
Video 3	19	6		
Video 4	19	7		
Video 5	19	19		
Video 6	19	18		
Video 7	19	15		
Video 8	19	10		

Berdasarkan Tabel 33, pengujian pada ketinggian 110 cm menghasilkan MAE sebesar 5.125 dan RMSE sebesar 7.185. Meskipun nilai MAE tidak jauh berbeda dari skenario 100 cm, variasi jumlah prediksi masih tinggi, yaitu antara 6 hingga 20. Video 3 dan Video 4 menjadi contoh *undercount* yang cukup besar, masing-masing dengan prediksi 6 dan 7.

Pada ketinggian ini, ukuran objek dalam piksel menjadi semakin kecil sehingga deteksi lebih sensitif terhadap gangguan visual. Saat alat bergerak, getaran yang terjadi dapat menyebabkan perubahan posisi objek secara mendadak pada citra. Perubahan ini dapat memutus proses *tracking* dan mengakibatkan hilangnya ID objek. Pada video dengan kondisi pencahayaan kurang baik atau latar belakang yang kurang kontras, masalah ini menjadi lebih besar dan menyebabkan objek gagal dihitung secara konsisten.

5. Ketinggian 120 cm

Tabel 34. Hasil pengujian YOLOv5s dengan *unsharp masking* pada ketinggian 120 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19		
Video 2	19	8		
Video 3	19	12		
Video 4	19	3		
Video 5	19	17		
Video 6	19	20		

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 7	19	9		
Video 8	19	10		

Berdasarkan Tabel 34, ketinggian 120 cm menghasilkan MAE sebesar 5.750 dan RMSE sebesar 7.842, dengan jumlah prediksi per video berkisar antara 3 hingga 20. Pada skenario ini terlihat beberapa kasus *undercount* yang cukup ekstrem, terutama pada Video 4 dengan prediksi 3 dan Video 2 dengan prediksi 8.

Semakin tinggi posisi kamera, ukuran objek dalam citra menjadi semakin kecil. Dalam kondisi tersebut, gangguan kecil akibat guncangan dapat menyebabkan perubahan posisi objek yang cukup besar secara relatif terhadap ukuran objek itu sendiri. Akibatnya, sistem *tracking* menjadi lebih mudah kehilangan ID. Ketika objek terdeteksi kembali, posisinya sering kali sudah terlalu dekat atau bahkan sudah melewati *counting line*, sehingga tidak lagi memenuhi syarat untuk dihitung. Hal ini menjelaskan mengapa performa *counting* pada ketinggian 120 cm menjadi lebih rendah dibandingkan ketinggian optimal.

6. Ketinggian 130 cm

Tabel 35. Hasil pengujian YOLOv5s dengan *unsharp masking* pada ketinggian 130 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	16		
Video 2	19	14		
Video 3	19	5		
Video 4	19	7		
Video 5	19	19	8.125	10.167
Video 6	19	20		
Video 7	19	5		
Video 8	19	3		

Berdasarkan Tabel 35, ketinggian 130 cm menunjukkan penurunan performa yang lebih jelas, dengan MAE sebesar 8.125 dan RMSE sebesar 10.167. Jumlah prediksi per video berada pada rentang 3 hingga 20, dan

terdapat beberapa video dengan hasil prediksi yang sangat rendah, seperti Video 3 dan Video 7 dengan prediksi 5, serta Video 8 dengan prediksi 3.

Pada ketinggian ini, representasi objek kentang dalam citra sudah semakin kecil sehingga model kesulitan mendeteksi objek secara konsisten. Guncangan alat memperburuk kondisi tersebut karena objek yang sedang dilacak dapat dengan mudah keluar dari area prediksi *tracker*. Setelah ID hilang, objek yang terdeteksi ulang biasanya sudah berada di bawah *counting line*, sehingga tidak lagi dapat dihitung. Proses ini terjadi berulang pada banyak video, sehingga menghasilkan nilai eror yang tinggi.

7. Ketinggian 140 cm

Tabel 36. Hasil pengujian YOLOv5s dengan *unsharp masking* pada ketinggian 140 cm

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19	7.375	10.069
Video 2	19	15		
Video 3	19	2		
Video 4	19	9		
Video 5	19	19		
Video 6	19	20		
Video 7	19	10		
Video 8	19	1		

Berdasarkan Tabel 36, ketinggian 140 cm menghasilkan MAE sebesar 7.375 dan RMSE sebesar 10.069, dengan jumlah prediksi per video berkisar antara 1 hingga 20. Beberapa video menunjukkan kegagalan penghitungan yang sangat besar, terutama Video 8 dengan prediksi 1 dan Video 3 dengan prediksi 2.

Pada ketinggian tertinggi ini, ukuran objek dalam piksel menjadi sangat kecil sehingga model tidak lagi dapat mendeteksi objek secara stabil pada semua kondisi. Selain itu, *field of view* yang lebih luas membuat objek terlihat kecil dan lebih mudah terpengaruh oleh guncangan. Dalam kondisi terburuk, objek yang kehilangan ID tidak berhasil terdeteksi kembali sebelum keluar dari bidang pandang. Akibatnya, objek tidak pernah melewati *counting line* dalam kondisi yang dapat dikenali oleh sistem, sehingga menyebabkan *undercount* yang sangat besar.

Tabel 37. Perbandingan skenario ketinggian pada YOLOv5s dengan *unsharp masking*

Skenario Ketinggian (cm)	MAE	RMSE
80	5.250	7.778
90	2.125	3.142
100	5.000	6.982
110	5.125	7.185
120	5.750	7.842
130	8.125	10.167
140	7.375	10.069

Berdasarkan Tabel 37, performa *counting* menunjukkan pola yang tidak linier terhadap variasi ketinggian kamera, dengan titik terbaik berada pada ketinggian 90 cm. Pada ketinggian ini diperoleh MAE sebesar 2.125 dan RMSE sebesar 3.142, yang merupakan nilai terendah di antara seluruh skenario. Pola ini menunjukkan bahwa performa sistem dipengaruhi oleh keseimbangan antara ukuran objek dalam citra, cakupan *field of view*, dan kestabilan proses *tracking* saat terjadi guncangan. Ketinggian yang terlalu rendah mempersempit bidang pandang dan mempersingkat waktu pelacakan, sedangkan ketinggian yang terlalu tinggi membuat objek terlalu kecil untuk dideteksi dan dilacak secara andal. Oleh karena itu, ketinggian 90 cm dapat direkomendasikan sebagai konfigurasi yang paling optimal untuk sistem yang dikembangkan.

3.5.2 Skenario Sudut

1. Sudut 80°

Tabel 38. Hasil pengujian YOLOv5s dengan *unsharp masking* pada sudut 80°

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19	5.179	7.707
Video 2	19	19		
Video 3	19	11		
Video 4	19	6		
Video 5	19	19		
Video 6	19	20		

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 7	19	19		
Video 8	19	15		
Video 9	19	19		
Video 10	19	18		
Video 11	19	6		
Video 12	19	9		
Video 13	19	19		
Video 14	19	18		
Video 15	19	15		
Video 16	19	10		
Video 17	19	17		
Video 18	19	20		
Video 19	19	9		
Video 20	19	10		
Video 22	19	19		
Video 22	19	20		
Video 23	19	10		
Video 24	19	1		
Video 25	19	19		
Video 26	19	20		
Video 27	19	5		
Video 28	19	3		

Berdasarkan Tabel 38, pengujian pada sudut 80° menghasilkan MAE sebesar 5.179 dan RMSE sebesar 7.707. Jumlah prediksi per video bervariasi antara 1 hingga 20. Meskipun beberapa video menghasilkan prediksi yang tepat atau mendekati jumlah aktual, masih terdapat beberapa video dengan *undercount* yang besar, seperti Video 24 dengan prediksi 1, Video 28 dengan prediksi 3, dan Video 27 dengan prediksi 5.

Pada sudut 80°, kamera berada pada posisi sedikit miring terhadap permukaan. Kondisi ini memberikan keuntungan karena pemisahan visual antarobjek menjadi lebih jelas, sehingga kemungkinan terjadinya tumpang tindih antarobjek menjadi lebih kecil. Selain itu, pergerakan objek menuju *counting line* terlihat lebih tegas dalam citra. Ketika terjadi guncangan, objek yang sempat kehilangan ID masih memiliki peluang lebih besar untuk terdeteksi kembali sebelum melewati garis hitung.

2. Sudut 90°

Tabel 39. Hasil pengujian YOLOv5s dengan *unsharp masking* pada sudut 90°

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19	5.893	8.091
Video 2	19	20		
Video 3	19	14		
Video 4	19	4		
Video 5	19	19		
Video 6	19	18		
Video 7	19	13		
Video 8	19	14		
Video 9	19	19		
Video 10	19	17		
Video 11	19	15		
Video 12	19	9		
Video 13	19	20		
Video 14	19	18		
Video 15	19	6		
Video 16	19	7		
Video 17	19	19		
Video 18	19	18		
Video 19	19	12		

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 20	19	3		
Video 22	19	19		
Video 22	19	15		
Video 23	19	2		
Video 24	19	9		
Video 25	19	16		
Video 26	19	14		
Video 27	19	5		
Video 28	19	7		

Berdasarkan Tabel 39, pengujian pada sudut 90° menghasilkan MAE sebesar 5.893 dan RMSE sebesar 8.091. Jumlah prediksi per video berkisar antara 2 hingga 20, dengan beberapa kasus *undercount* yang cukup ekstrem, seperti Video 23 dengan prediksi 2 dan Video 20 dengan prediksi 3.

Pada sudut 90°, kamera memandang objek secara tegak lurus. Walaupun bentuk objek terlihat lebih seragam, arah pergerakan objek terhadap *counting line* menjadi kurang tegas dalam citra. Ketika terjadi guncangan vertikal, posisi objek dapat berubah secara tiba-tiba terhadap garis hitung, sehingga sistem dapat kehilangan momen saat objek seharusnya tercatat. Selain itu, sudut ini juga membuat beberapa objek tampak lebih saling menutupi.

Tabel 40. Perbandingan skenario sudut pada YOLOv5s dengan *unsharp masking*

Skenario Sudut (°)	MAE	RMSE
80	5.179	7.707
90	5.893	8.091

Berdasarkan Tabel 40, sudut 80° menghasilkan performa yang lebih baik dibandingkan sudut 90°, dengan MAE dan RMSE masing-masing sebesar 5.179 dan 7.707. Pada sudut 90°, nilai tersebut meningkat menjadi 5.893 dan 8.091. Meskipun selisihnya tidak terlalu besar, hasil ini menunjukkan bahwa sudut pandang yang sedikit miring lebih mendukung kestabilan proses deteksi, *tracking*, dan *counting*. Oleh karena itu, sudut 80° lebih direkomendasikan untuk sistem ini.

3.5.3 Skenario Kecepatan

1. Kecepatan 0.25 m/s

Tabel 41. Hasil pengujian YOLOv5s dengan *unsharp masking* pada kecepatan 0.25 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	19	0.429	1.000
Video 2	19	19		
Video 3	19	19		
Video 4	19	19		
Video 5	19	19		
Video 6	19	19		
Video 7	19	20		
Video 8	19	19		
Video 9	19	19		
Video 10	19	17		
Video 11	19	19		
Video 12	19	19		
Video 13	19	16		
Video 14	19	19		

Berdasarkan Tabel 41, kecepatan 0.25 m/s menghasilkan performa terbaik pada seluruh skenario kecepatan, dengan MAE sebesar 0.429 dan RMSE sebesar 1.000. Hampir seluruh video menghasilkan prediksi yang tepat, dan hanya beberapa video yang menunjukkan selisih kecil dari jumlah aktual. Pada kecepatan rendah ini, gerakan alat cenderung lebih stabil sehingga guncangan yang diterima kamera menjadi minimal. Akibatnya, posisi objek dalam citra relatif konsisten antarframe, dan sistem *tracking* dapat mempertahankan ID objek dengan baik. Dengan kondisi tersebut, objek dapat melewati *counting line* secara teratur dan proses penghitungan berlangsung dengan sangat akurat.

2. Kecepatan 0.5 m/s

Tabel 42. Hasil pengujian YOLOv5s dengan *unsharp masking* pada kecepatan 0.5 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	20	1.500	1.982
Video 2	19	19		
Video 3	19	18		
Video 4	19	20		
Video 5	19	17		
Video 6	19	18		
Video 7	19	18		
Video 8	19	18		
Video 9	19	18		
Video 10	19	20		
Video 11	19	15		
Video 12	19	20		
Video 13	19	14		
Video 14	19	20		

Berdasarkan Tabel 42, kecepatan 0.5 m/s menghasilkan MAE sebesar 1.500 dan RMSE sebesar 1.982. Nilai ini menunjukkan adanya peningkatan eror, namun performanya masih tergolong baik. Jumlah prediksi berada pada rentang 14 hingga 20, dengan sebagian besar video hanya berbeda 1 hingga 2 objek dari jumlah aktual. Pada kecepatan ini, guncangan mekanis mulai muncul, tetapi belum cukup besar untuk menyebabkan kegagalan *tracking* secara luas. Beberapa objek mungkin mengalami perubahan posisi akibat getaran, namun sistem masih mampu mengaitkan kembali ID objek sebelum mencapai *counting line*. Oleh karena itu, eror yang terjadi masih relatif kecil dan sistem masih dapat bekerja dengan cukup andal.

3. Kecepatan 0.75 m/s

Tabel 43. Hasil pengujian YOLOv5s dengan *unsharp masking* pada kecepatan 0.75 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	14	8.857	10.021
Video 2	19	11		
Video 3	19	13		
Video 4	19	19		
Video 5	19	15		
Video 6	19	6		
Video 7	19	6		
Video 8	19	15		
Video 9	19	12		
Video 10	19	9		
Video 11	19	2		
Video 12	19	10		
Video 13	19	5		
Video 14	19	5		

Berdasarkan Tabel 43, kecepatan 0.75 m/s menyebabkan penurunan performa yang cukup tajam, dengan MAE sebesar 8.857 dan RMSE sebesar 10.021. Kenaikan eror yang besar dibandingkan skenario 0.5 m/s menunjukkan bahwa pada kecepatan ini sistem mulai memasuki kondisi yang sulit untuk mempertahankan keandalan *counting*.

Pada kecepatan tersebut, getaran alat menjadi lebih kuat dan menyebabkan pergeseran citra antarframe semakin besar. Akibatnya, objek yang sedang dilacak sering kehilangan ID karena perpindahan posisinya melampaui toleransi *tracker*. Ketika objek muncul kembali, posisinya sering kali sudah berada di bawah *counting line*, sehingga objek tidak dapat dihitung meskipun sebenarnya telah melewati area penghitungan. Kondisi ini menyebabkan *undercount* yang cukup besar pada banyak video.

4. Kecepatan 1 m/s

Tabel 44. Hasil pengujian YOLOv5s dengan *unsharp masking* pada kecepatan 1 m/s

Video	Jumlah Aktual	Jumlah Prediksi	MAE	RMSE
Video 1	19	4	11.357	12.015
Video 2	19	6		
Video 3	19	14		
Video 4	19	15		
Video 5	19	9		
Video 6	19	9		
Video 7	19	7		
Video 8	19	10		
Video 9	19	3		
Video 10	19	10		
Video 11	19	9		
Video 12	19	1		
Video 13	19	7		
Video 14	19	3		

Berdasarkan Tabel 44, kecepatan 1 m/s menghasilkan performa terburuk, dengan MAE sebesar 11.357 dan RMSE sebesar 12.015. Jumlah prediksi per video sangat rendah, yaitu antara 1 hingga 15, yang menunjukkan bahwa sebagian besar objek gagal dihitung. Pada kecepatan ini, guncangan alat menjadi sangat dominan dan hampir terjadi pada setiap perpindahan frame. Getaran yang tinggi menyebabkan kehilangan ID objek secara berulang, sementara perpindahan objek antarframe juga menjadi semakin besar. Dalam banyak kasus, objek yang terdeteksi kembali sudah melewati *counting line*, sehingga sistem tidak dapat mencatatnya. Selain itu, kualitas visual objek juga menurun akibat gangguan gerak kamera, sehingga kemampuan deteksi model ikut menurun. Kondisi ini yang menyebabkan *counting* pada kecepatan 1 m/s mengalami penurunan performa yang sangat besar.

Tabel 45. Perbandingan skenario kecepatan pada YOLOv5s dengan *unsharp masking*

Skenario Kecepatan (m/s)	MAE	RMSE
0.25	0.429	1.000
0.5	1.500	1.982
0.75	8.857	10.021
1	11.357	12.015

Berdasarkan Tabel 45, terdapat kecenderungan peningkatan eror yang sangat jelas seiring bertambahnya kecepatan. Nilai MAE meningkat dari 0.429 pada kecepatan 0.25 m/s menjadi 11.357 pada kecepatan 1 m/s, sedangkan RMSE meningkat dari 1.000 menjadi 12.015. Kenaikan eror yang sangat tajam antara 0.5 m/s dan 0.75 m/s menunjukkan bahwa terdapat batas operasional di mana sistem mulai kehilangan kestabilan *tracking* akibat guncangan alat. Dengan demikian, kecepatan 0.25 m/s dapat direkomendasikan sebagai kondisi operasional terbaik. Hasil ini juga menunjukkan bahwa penambahan stabilizer kamera pada pengembangan berikutnya akan sangat penting untuk meningkatkan keandalan sistem pada kecepatan yang lebih tinggi.

Secara keseluruhan, hasil pengujian menunjukkan bahwa performa *counting* model YOLOv5s dengan *unsharp masking* juga dipengaruhi oleh kombinasi parameter ketinggian, sudut, dan kecepatan, namun dengan pola optimum yang sedikit berbeda dibandingkan model tanpa *unsharp masking*. Pada skenario ketinggian, performa terbaik diperoleh pada 90 cm, yang menunjukkan bahwa setelah peningkatan ketajaman citra diterapkan, sistem lebih optimal pada posisi kamera yang sedikit lebih rendah. Meskipun demikian, pola umumnya tetap sama, yaitu ketinggian yang terlalu rendah menyebabkan bidang pandang terlalu sempit, sedangkan ketinggian yang terlalu tinggi menyebabkan ukuran objek dalam citra semakin kecil. Pada skenario sudut, 80° tetap memberikan hasil yang lebih baik dibandingkan 90°, sedangkan pada skenario kecepatan, performa terbaik juga dicapai pada 0.25 m/s. Hasil ini menunjukkan bahwa penerapan *unsharp masking* dapat meningkatkan kualitas fitur visual objek, tetapi belum mampu menghilangkan pengaruh utama dari getaran atau guncangan alat. Dalam banyak kasus, guncangan tetap menjadi penyebab utama menurunnya performa *counting*, karena menyebabkan objek kehilangan kontinuitas *tracking*, bergeser secara tiba-tiba antarframe, dan akhirnya tidak tercatat secara konsisten saat melewati *counting line*.

3.6 Performa Real-Time

Pengujian performa real-time dilakukan untuk mengevaluasi kemampuan sistem dalam memproses video secara langsung pada perangkat yang digunakan, yaitu

laptop Lenovo IdeaPad Slim 3 14ITL6 dengan prosesor Intel Core generasi ke-11, RAM 8 GB, SSD 512 GB, dan GPU NVIDIA GeForce MX450 2 GB GDDR5.

Hasil pengujian menunjukkan bahwa sistem tanpa *unsharp masking* menghasilkan rata-rata 19.06 FPS, sedangkan sistem dengan *unsharp masking* menghasilkan rata-rata 16.11 FPS. Dengan demikian, penerapan *unsharp masking* menyebabkan penurunan performa sebesar 2.95 FPS atau sekitar 15.5%. Penurunan ini terjadi karena proses *unsharp masking* menambahkan beban komputasi pada tahap *preprocessing*, yang dikerjakan sebelum *frame* masuk ke tahap inferensi. Akibatnya, waktu pemrosesan per *frame* menjadi lebih besar dan laju pemrosesan video secara keseluruhan menurun.

Meskipun demikian, kedua konfigurasi masih menghasilkan nilai FPS di atas 15 FPS, yang dalam penelitian ini digunakan sebagai batas minimum agar sistem dapat dikategorikan bekerja secara *real-time*. Hal ini menunjukkan bahwa baik sistem tanpa *unsharp masking* maupun sistem dengan *unsharp masking* masih layak digunakan untuk pemrosesan video secara langsung pada perangkat yang digunakan.

Namun, penurunan FPS akibat penerapan *unsharp masking* tetap perlu diperhatikan, terutama ketika sistem dioperasikan pada kecepatan alat yang lebih tinggi. Interval antarframe yang menjadi lebih panjang dapat menyebabkan perpindahan posisi objek antarframe semakin besar, sehingga risiko kehilangan ID pada proses *tracking* ikut meningkat. Kondisi tersebut pada akhirnya dapat memengaruhi kestabilan proses *counting*. Oleh karena itu, penggunaan *unsharp masking* memberikan keuntungan pada sisi kualitas deteksi, tetapi juga menimbulkan konsekuensi berupa penurunan kecepatan pemrosesan yang perlu dipertimbangkan dalam implementasi sistem secara keseluruhan.

BAB IV

KESIMPULAN

Berdasarkan seluruh rangkaian pengujian yang telah dilakukan, penelitian ini menunjukkan bahwa model YOLOv5 mampu digunakan untuk mendeteksi objek kentang multi-skala dengan tingkat akurasi yang tinggi. Dari lima varian yang diuji, yaitu *nano*, *small*, *medium*, *large*, dan *xtra*, varian YOLOv5s dipilih sebagai model terbaik karena memberikan keseimbangan paling baik antara akurasi deteksi dan efisiensi komputasi. Model ini menghasilkan *precision* 0.996, *recall* 0.990, *F1-score* 0.993, *mAP50* 0.995, dan *mAP50-95* 0.917, dengan ukuran model 14.35 MB serta *inference time* 5.9 ms, sehingga sesuai untuk implementasi sistem deteksi objek secara *real-time*.

Hasil pengujian konvergensi menunjukkan bahwa peningkatan jumlah *epoch* pelatihan dari 100 *epoch* menjadi 1000 *epoch* dengan parameter *patience* 100 meningkatkan performa deteksi model. Proses pelatihan berhenti secara otomatis pada *epoch* ke-467, yang menandakan bahwa model telah mencapai kondisi konvergen sebelum mencapai batas maksimum *epoch*. Pada kondisi ini, jumlah *true positive* meningkat dari 2.430 menjadi 2.437, sedangkan *false positive* menurun dari 10 menjadi 4 dan *false negative* menurun dari 25 menjadi 18, yang menunjukkan bahwa pelatihan yang lebih panjang memungkinkan model mempelajari representasi fitur objek secara lebih optimal.

Pengujian *preprocessing* menggunakan *unsharp masking* menunjukkan bahwa teknik ini mampu meningkatkan kemampuan deteksi pada beberapa metrik utama, terutama *recall*, *mAP50*, dan *mAP50-95*. Jumlah *true positive* meningkat dari 2.437 menjadi 2.448, sedangkan *false negative* menurun dari 18 menjadi 7. Selain itu, nilai *mAP50-95* pada model dengan *unsharp masking* berada pada kisaran 0.951, lebih tinggi dibandingkan model tanpa *unsharp masking* yang berada pada 0.944. Namun demikian, peningkatan ketajaman citra juga menyebabkan peningkatan *false positive*.

Dari sisi efisiensi komputasi, penerapan *unsharp masking* meningkatkan waktu pemrosesan terutama pada tahap *preprocessing*. Waktu *preprocessing* meningkat dari sekitar 1.9 ms menjadi 111.2 ms, sedangkan waktu *inference* hanya meningkat dari 5.8 ms menjadi 7.2 ms dan *postprocessing* dari 1.3 ms menjadi 1.4 ms. Akibatnya, sistem tanpa *unsharp masking* menghasilkan rata-rata 19.06 FPS, sedangkan sistem dengan *unsharp masking* menghasilkan 16.11 FPS. Kedua konfigurasi tersebut masih berada di atas ambang 15 FPS, sehingga sistem tetap dapat beroperasi dalam kondisi *real-time*.

Pada pengujian *counting* model YOLOv5s, performa sistem dipengaruhi oleh kombinasi parameter ketinggian, sudut, dan kecepatan. Performa terbaik pada skenario ketinggian diperoleh pada rentang 110–120 cm, dengan MAE terendah sebesar 2.750 pada 120 cm dan RMSE terendah sebesar 3.142 pada 110 cm. Pada skenario sudut, sudut 80° menghasilkan eror yang lebih rendah dibandingkan 90°, sedangkan pada skenario kecepatan, performa terbaik diperoleh pada 0.25 m/s

dengan MAE 5.000 dan RMSE 5.880, yang kemudian meningkat hingga MAE 8.214 dan RMSE 8.892 pada kecepatan 1 m/s.

Pada pengujian *counting* model YOLOv5s dengan *unsharp masking*, pola yang diperoleh secara umum masih serupa, tetapi titik optimum ketinggian bergeser. Pada konfigurasi ini, performa terbaik diperoleh pada ketinggian 90 cm, dengan MAE 2.125 dan RMSE 3.142, sedangkan sudut 80° dan kecepatan 0.25 m/s tetap memberikan performa terbaik dibandingkan konfigurasi lainnya. Hasil ini menunjukkan bahwa peningkatan ketajaman citra memengaruhi kondisi operasional optimal sistem, khususnya pada parameter ketinggian kamera.

Secara keseluruhan, hasil penelitian menunjukkan bahwa faktor yang paling dominan memengaruhi performa *counting* bukan hanya akurasi model deteksi, tetapi juga getaran atau guncangan alat selama proses akuisisi video. Hal ini terlihat dari peningkatan eror yang signifikan pada kecepatan alat yang lebih tinggi. Pada konfigurasi dengan *unsharp masking*, misalnya, peningkatan kecepatan dari 0.25 m/s menjadi 1 m/s menyebabkan MAE meningkat dari 0.429 menjadi 11.357 dan RMSE meningkat dari 1.000 menjadi 12.015. Dengan demikian, keberhasilan sistem deteksi dan penghitungan objek kentang secara *real-time* tidak hanya ditentukan oleh pemilihan model dan teknik *preprocessing*, tetapi juga sangat dipengaruhi oleh kestabilan mekanis sistem saat dioperasikan di lapangan.

Adapun saran pengembangan untuk penelitian selanjutnya dapat mempertimbangkan hal-hal sebagai berikut.

1. Mengimplementasikan dan mengevaluasi sistem pada perangkat *edge computing* seperti seri NVIDIA Jetson atau Raspberry Pi yang dilengkapi akselerator AI, sehingga dapat diperoleh gambaran yang lebih realistis mengenai kelayakan sistem untuk diterapkan secara langsung di lapangan.
2. Mengembangkan prototipe sistem *counting* hasil panen yang terintegrasi secara penuh, yang mencakup integrasi antara mekanisme alat panen atau konveyor, modul kamera, dan unit pemrosesan dalam satu sistem yang siap dioperasikan di lingkungan pertanian.
3. Menggunakan modul kamera dengan spesifikasi yang lebih baik untuk mengurangi kesalahan deteksi akibat kondisi lingkungan yang tidak terkontrol, terutama pada perubahan pencahayaan dan gangguan visual di lapangan.
4. Mempertimbangkan penggunaan kamera dengan kemampuan *auto-exposure* yang lebih responsif serta *polarizing* filter untuk mengurangi efek silau akibat paparan sinar matahari langsung.
5. Menambahkan *stabilizer* optis maupun mekanis pada modul kamera untuk meredam guncangan.

DAFTAR PUSTAKA

- Alhardi, A., & Afeef, M. A. (2024). *Object Detection Algorithms & Techniques 4th International Conference on Innovative Academic Studies Nesne Algılama Algoritmaları ve Teknikleri*. March, 391–399.
- Alif, M. A. R., & Hussain, M. (2024). *YOLOv1 to YOLOv10: A comprehensive review of YOLO variants and their application in the agricultural domain*. <http://arxiv.org/abs/2406.10139>
- Badan Pangan Nasional. (2024a). *Peraturan Badan Pangan Nasional Nomor 13 Tahun 2024 tentang Standar Mutu Produk Pangan Lokal dalam rangka Penganekaragaman Pangan*.
- Badan Pangan Nasional. (2024b). *Peraturan Badan Pangan Nasional Nomor 15 Tahun 2024 tentang Batas Maksimal Residu Pestisida dalam Pangan Segar Asal Tumbuhan*. <https://www.peraturan.go.id/id/peraturan-bapanas-no-15-tahun-2024>
- Badan Pusat Statistik. (2023). *Statistik Hortikultura 2022*. <https://www.bps.go.id/id/publication/2023/06/09/03847c5743d8b6cd3f08ab76/statistik-hortikultura-2022.html>
- Badan Pusat Statistik. (2024). *Statistik Hortikultura 2023*. <https://www.bps.go.id/id/publication/2024/06/10/790c957ba8892f9771aeefb7/statistik-hortikultura-2023.html>
- Badan Pusat Statistik. (2025). *Statistik Hortikultura 2024*. <https://www.bps.go.id/id/publication/2025/06/10/aab67e4d36ea6d7bed30d79f/statistik-hortikultura-2024.html>
- Bajraktari, A., & Toyman, H. (2025). Autonomous Agricultural Robot Using YOLOv8 and ByteTrack for Weed Detection and Destruction. *Machines*, 13(3). <https://doi.org/10.3390/machines13030219>
- Chai, J., Zeng, H., Li, A., & Ngai, E. W. T. (2021). Deep learning in computer vision: A critical review of emerging techniques and application scenarios. *Machine Learning with Applications*, 6(August), 100134. <https://doi.org/10.1016/j.mlwa.2021.100134>
- Cong, X., Li, S., Chen, F., Liu, C., & Meng, Y. (2022). A Review of YOLO Object Detection Based on Deep Learning. *Frontiers in Computing and Intelligent Systems*, 4. <https://doi.org/10.11999/JEIT210790>
- F., M. Y., Yuwono, B., & P., D. B. (2022). *Dasar Pengolahan Citra Digital* (2022 ed.). Lembaga Penelitian dan Pengabdian kepada Masyarakat UPN Veteran Jakarta.
- Farjon, G., Huijun, L., & Edan, Y. (2023). *Deep-Learning-based Counting Methods*,

Datasets, and Applications in Agriculture -- A Review.
<http://arxiv.org/abs/2303.02632>

- Ghazal, S., Munir, A., & Qureshi, W. S. (2024). Computer vision in smart agriculture and precision farming: Techniques and applications. In *Artificial Intelligence in Agriculture* (Vol. 13, hal. 64–83). KeAi Communications Co. <https://doi.org/10.1016/j.aiaa.2024.06.004>
- Gündüz, M. Ş., & Işık, G. (2023). A new YOLO-based method for real-time crowd detection from video and performance analysis of YOLO models. *Journal of Real-Time Image Processing*, 20(1). <https://doi.org/10.1007/s11554-023-01276-w>
- Huang, Y., Qian, Y., Wei, H., Lu, Y., Ling, B., & Qin, Y. (2023). A survey of deep learning-based object detection methods in crop counting. *Computers and Electronics in Agriculture*, 215(November), 108425. <https://doi.org/10.1016/j.compag.2023.108425>
- Hussien, R. M., Al-Jubouri, K. Q., Gburi, M. Al, Hussein Qahtan, A. G., & Duaa Jaafar, A. H. (2021). Computer Vision and Image Processing the Challenges and Opportunities for new technologies approach: A paper review. *Journal of Physics: Conference Series*, 1973(1). <https://doi.org/10.1088/1742-6596/1973/1/012002>
- Jegham, N., Koh, C. Y., Abdelatti, M., & Hendawi, A. (2025). *YOLO Evolution: A Comprehensive Benchmark and Architectural Review of YOLOv12, YOLO11, and Their Previous Versions*. <http://arxiv.org/abs/2411.00201>
- Jiang, Z., Messner, J. I., & Matts, E. (2023). Computer Vision Applications in Construction and Asset Management Phases: a Literature Review. *Journal of Information Technology in Construction*, 28(November 2022), 176–199. <https://doi.org/10.36680/J.ITCON.2023.009>
- Jiménez-Bravo, D. M., Lozano Murciego, Á., Sales Mendes, A., Sánchez San Blás, H., & Bajo, J. (2022). Multi-object tracking in traffic environments: A systematic literature review. *Neurocomputing*, 494, 43–55. <https://doi.org/10.1016/j.neucom.2022.04.087>
- Jocher, G., Stoken, A., Chaurasia, A., Borovec, J., NanoCode012, TaoXie, Kwon, Y., Michael, K., Changyu, L., Fang, J., V, A., Laughing, Tkianai, lmyhxy, Mammana, L., AlexWang1900, Fati, C., Montes, D., Hajek, J., ... Wanghaoyang0106. (2021). *ultralytics/yolov5: v6.0 - YOLOv5n "Nano" models, Roboflow integration, TensorFlow export, OpenCV DNN support (v6.0)*. <https://zenodo.org/records/5563715>
- Johnson, C. M., & Auat Cheein, F. (2023). Machinery for potato harvesting: a state-of-the-art review. In *Frontiers in Plant Science* (Vol. 14). Frontiers Media S.A. <https://doi.org/10.3389/fpls.2023.1156734>
- Joiya, F. (2022). OBJECT DETECTION: YOLO VS FASTER R-CNN. *International*

Research Journal of Modernization in Engineering Technology and Science, 04(09). <https://doi.org/10.56726/irjmets30226>

Kementerian Perdagangan. (2025a). *Peraturan Menteri Perdagangan Nomor 18 Tahun 2025 tentang Kebijakan dan Pengaturan Impor Barang Pertanian dan Peternakan*. <https://www.peraturan.go.id/id/permendag-no-18-tahun-2025>

Kementerian Perdagangan. (2025b). *Peraturan Menteri Perdagangan Nomor 23 Tahun 2025 tentang Kebijakan dan Pengaturan Impor Barang Konsumsi*.

Liu, J., Xie, Y., Zhang, Y., & Li, H. (2025). *Vehicle Flow Detection and Tracking Based on an Improved YOLOv8n and ByteTrack Framework*.

Luo, W., Xing, J., Milan, A., Zhang, X., Liu, W., & Kim, T. K. (2021). Multiple object tracking: A literature review. *Artificial Intelligence*, 293, 103448. <https://doi.org/10.1016/j.artint.2020.103448>

Maulidiyah, R. (2024). *APLIKASI STOCHASTIC FRONTIER DAN STRUCTURAL EQUATION MODELING DALAM MENGANALISIS PENGARUH PENGGUNAAN FAKTOR-FAKTOR PRODUKSI TERHADAP PRODUKSI DAN KEBERHASILAN USAHATANI KENTANG APPLICATION OF STOCHASTIC FRONTIER AND STRUCTURAL EQUATION MODELING IN ANALYZING THE EFFECT OF THE USE OF PRODUCTION FACTORS ON THE PRODUCTION AND SUCCESS OF POTATO FARMING*.

Raja, R., Vats, A., Thawakar, O., & Ashraf, T. (2025). Object Tracking: A Comprehensive Survey From Classical Approaches to Large Vision-Language and Foundation Models. *TechRxiv*, 1–27. <https://doi.org/10.36227/techrxiv.176072570.01421705/v1>

Sari, A. K. (2021). *PERANAN MODAL SOSIAL DALAM USAHATANI KENTANG*.

Shen, L., Su, J., He, R., Song, L., Huang, R., Fang, Y., Song, Y., & Su, B. (2023). Real-time tracking and counting of grape clusters in the field based on channel pruning with YOLOv5s. *Computers and Electronics in Agriculture*, 206(January). <https://doi.org/10.1016/j.compag.2023.107662>

Singh, G., Pidadi, P., & Malwad, D. S. (2023). A Review on Applications of Computer Vision. *International Conference on Hybrid Intelligent Systems*, 647, 464–479. https://doi.org/https://doi.org/10.1007/978-3-031-27409-1_42

Sitepu, S., Hutapea, N., & Atrisiandy, K. (2021). *BUDIDAYA KENTANG*. Balai Pengkajian Teknologi Pertanian.

Sun, Y., Sun, Z., & Chen, W. (2024). The evolution of object detection methods. *Engineering Applications of Artificial Intelligence*, 133(PE), 108458. <https://doi.org/10.1016/j.engappai.2024.108458>

Syahidi, A. A. (2024). *Pengolahan Citra Digital dan Dasar Visi Komputer dengan Python* (R. Fauzan (ed.)). POLIBAN PRESS.

- Syahsir, A. R. R. (2024). *SISTEM DETEKSI BRONDOLAN KELAPA SAWIT MENGGUNAKAN METODE YOU ONLY LOOK ONCE (YOLO)*. Universitas Hasanuddin.
- Tasnim, S., & Qi, W. (2023). Progress in Object Detection: An In-Depth Analysis of Methods and Use Cases. *European Journal of Electrical Engineering and Computer Science*, 7(4), 39–45. <https://doi.org/10.24018/ejece.2023.7.4.537>
- Terven, J., Córdova-Esparza, D. M., & Romero-González, J. A. (2023). A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. In *Machine Learning and Knowledge Extraction* (Vol. 5, Nomor 4, hal. 1680–1716). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/make5040083>
- Wang, J., Zhang, X., Xu, X., & Song, K.-P. (2026). *Underwater Acoustic Point-cloud Filtering via Adaptive Unsharp Masking. February.* <https://doi.org/10.1145/3569966.3570052>
- Wei, J., As'array, A., Anas Md Rezali, K., Zuhri Mohamed Yusoff, M., Ma, H., & Zhang, K. (2025). A Review of YOLO Algorithm and Its Applications in Autonomous Driving Object Detection. *IEEE Access*, 13(May), 93688–93711. <https://doi.org/10.1109/ACCESS.2025.3573376>
- Xiao, C., Song, C., Li, J., Liao, M., Pu, Y., & Du, K. (2025). Potato precision planter metering system based on improved YOLOv5n-ByteTrack. *Frontiers in Plant Science*, 16. <https://doi.org/10.3389/fpls.2025.1563551>
- Yeerjiang, A., Wang, Z., Huang, X., Zhang, J., & Chen, Q. (2024). YOLOv1 to YOLOv10: A Comprehensive Review of YOLO Variants and Their Application in Medical Image Detection. *Journal of Artificial Intelligence Practice*, 7(3), 112–122. <https://doi.org/10.23977/jaip.2024.070314>
- Zaidi, S. S. A., Ansari, M. S., Aslam, A., Kanwal, N., Asghar, M., & Lee, B. (2022). A survey of modern deep learning based object detection models. *Digital Signal Processing: A Review Journal*, 126, 103514. <https://doi.org/10.1016/j.dsp.2022.103514>
- Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. <http://arxiv.org/abs/2110.06864>
- Zharikova, E. P., Grigoriev, Y. Y., & Grigorieva, A. L. (2021). Applications of computer vision in cross-sectoral tasks. *Lecture Notes in Networks and Systems*, 200(9), 415–426. https://doi.org/10.1007/978-3-030-69421-0_45
- Zou, Z., Chen, K., Shi, Z., Guo, Y., & Ye, J. (2023). Object Detection in 20 Years: A Survey. *Proceedings of the IEEE*, 111(3), 257–276. <https://doi.org/10.1109/JPROC.2023.3238524>

LAMPIRAN

Lampiran 1. *Source code*



Tautan: <https://s.unhas.ac.id/D121211005>